Сброс XSCore по debug reset: сброс-уровень, сохранение L1D или её обход, ранняя блокировка A, флаги пустоты

Подробное описание каждого изменения в XiangShan Kunminghu V2 (версия v5). Сброс приходит от Debug Module (ndmreset, hartreset) или от кластера и сбрасывает только ядро с L1; L2 и CHI-порт тайла работают. Параметр CoreResetKeepL1 выбирает: L1D сохраняет строки (Probe ждёт в L2) или перед сбросом отдаёт их все в L2 обходом (Probe отвечает гейт). Сброс — уровень: ядро в сбросе, пока держат запрос, не меньше 100 тактов. Каналы A некогерентных агентов закрываются с первого такта sFlush, канал B — с первого такта sDrain; ожидание в 16 тактов заменено флагами пустоты. Все сигналы из домена сброса ядра изолированы на стороне тайла.

Что	Где
XiangShan kunminghu-v2, база 7b16188	16 файлов `src/`, +998 / −50 строк (git diff 7b16188 HEAD); новый патч от c098f99 даёт то же дерево (изменяемые файлы совпадают)
Порты	`debug_reset` (ndmreset / hartreset), `core_reset_req` (кластер), `core_reset_busy` у XSTile, XSTileWrap и XSNoCTop; `hartResetReq` с `EnableCoreReset` — тоже `debug_reset`. Порты не изменились относительно v4
Параметр `CoreResetKeepL1`	`true` (по умолчанию): L1D сохраняет строки, L2 забирает грязные данные Probe-ом после сброса. `false`: обход (sweep) — L1D отдаёт каждую строку в L2 (Release / ReleaseData), затем сбрасывается вместе с ядром. Параметр один: `CoreResetLevel` удалён, сброс всегда уровень
Probe во время сброса	KEEP: Probe держится в L2 (гейт: `b.ready` = 0, к ядру `b.valid` = 0) весь сброс; с первого такта sDrain. SWEEP: в sDrain держится, в сбросе гейт сам отвечает ProbeAck NtoN (с CoupledL2 такой Probe не приходит: директория L1D не числит)
Длина сброса	`rstCycles` = 100 тактов и до снятия запроса (TileLink [TL 1.8.1 3.2.2], 100 тактов с valid = 0)
Ранний A, быстрый B	канал A некогерентных агентов (ICache, PTW, i_mmio, d_mmio) закрыт с первого такта sFlush; A l1d — с sDrain; канал B закрыт с первого такта sDrain (`holdProbe` — функция регистра состояния); выход sFlush ждёт тишины ранних гейтов (`earlyQuiet`)
Флаги пустоты	ожидание 16 тактов подряд заменено четырьмя точными флагами DCache (`mq_empty`, `wbq_empty`, `mp_empty`, `pq_empty`); выход из sDrain в sReset — в том же такте по `gatesQuiet && l1Done && l1Idle`
Изоляция (RDC)	регистр `isolate` (= `closed`), поднятый на том же фронте, что и сброс ядра: маскирует valid и держит уровни всех сигналов из домена сброса ядра — в гейтах, в `TLResetIsolation`, в XSTile и (KEEP) в записи массивов L1D (`freeze`)
Новые файлы	xiangshan/CoreReset.scala (TLQuiesceGate, TLResetIsolation, CoreResetCtrl), cache/dcache/mainpipe/FlushUnit.scala (L1QuiesceIO, DCacheFlushUnit)
Изменённые файлы	MainPipe (запрос flush, `busy`), Probe.scala (`ProbeQueue.empty`), MissQueue, WritebackQueue, DCacheWrapper (флаги, `tagArray` на сбросе тайла, маски записи), FakeDCache, MemBlock, XSCore, XSTile, XSTileWrap, XSNoCTop, Top, SoC, YamlParser. CoupledL2 не меняется
Итог проверки	формально KEEP: prove 100 — PASS (122,02 с), prove 16 — PASS (8,71 с); cover: нет данных; мутанты: нет данных; формально SWEEP: нет данных; эквивалентность RTL ↔ модели: 5 из 5 миттеров PASS; регрессия: прогонов дизайна 160 / 160 PASS, мутантов 211 / 211 пойманы; длинный прогон: KEEP 490 / 490, SWEEP 490 / 490

Как читать документ. Раздел 1 — задача, один параметр и две конфигурации; что происходит с L1D в каждой; обход и то, как CoupledL2 принимает Release и ответ на Probe; сброс-уровень (1.4); раннее закрытие A, быстрое закрытие B и флаги пустоты (1.5); изоляция домена сброса (1.6). Раздел 2 — понятия TileLink и Debug Module, справочник модулей. Раздел 3 — где изменения.

Раздел 4 — последовательность событий с диаграммами и объяснением, зачем ждать каждого условия. Разделы 5 и 6 — каждое изменение в коде: фрагмент патча (зелёным — добавленные строки, красным — удалённые, серым — неизменённый контекст) и объяснение. Раздел 7 — проверка (в том числе временные диаграммы из VCD), раздел 8 — подключение и выбор конфигурации, раздел 9 — ограничения, раздел 10 — отличия от v4 и состав пакета. Ссылки на TileLink: синие — версия 1.8.1, зелёные — 1.9.3; номера разделов и страниц взяты из документа v4.

Важно: Chisel компилируется; статус полной elaboration

Chisel-код v5 компилируется (`mill xiangshan.compile`; проверено на коммитах, перечисленных в `CLAUDE.md`). **Версия v4 не компилировалась:** в `TLQuiesceGate(val name: String)` член `name` конфликтовал с `LazyModule.name`. В v5 поле называется `gateName`.

Полная elaboration XSNoCTop (все соединения XSTile на реальных модулях) не выполнена: файл результатов `results_elab.txt` отсутствует. Модули `CoreResetCtrl` (KEEP и SWEEP), гейты и изоляция elaborate-ятся стендом (из его вывода получен RTL для проверки эквивалентности, раздел 7.1); соединения XSTile и DCache в целом проверены компиляцией и ревью, но не прогоном всего XSNoCTop, пока результата полной elaboration нет.

1. Задача и общая идея

Нужно сбросить только XSCore — ядро вместе с L1 (DCache, ICache), TLB и PTW, — а L2 (CoupledL2) и CHI-порт тайла оставить работающими. Сброс приходит от Debug Module (сброс системы `ndmreset` или сброс харта `hartreset`) или от контроллера сброса кластера. CHI не затрагивается: кредиты, LINKACTIVE и SYSCO не меняются, кластер сброса не видит.

Ядро и L2 общаются по TileLink (раздел 2.1). Если просто подать сброс работающему ядру, ломаются четыре вещи:

1. **Незавершённые транзакции.** L2 ждёт от ядра GrantAck или ProbeAck, а сброшенное ядро не ответит. Запись MSHR в L2 висит вечно, вместе с ней и снупы HN. По спецификации менеджер ждёт ProbeAck на каждый Probe [TL 1.8.1 9.2, с. 67], агент обязан ответить на каждый принятый запрос [TL 1.8.1 5, с. 28], а Grant — тоже запрос, на него отвечает GrantAck [TL 1.8.1 5.1, с. 29]. Ответ, пришедший после сброса, ядро примет за ответ на свой новый запрос с тем же source ID: идентификатор занят, пока на запрос не ответили [TL 1.8.1 6.4, с. 43].
2. **Сообщение «на середине».** Строка 64 Б идёт по шине 32 Б двумя тактами передачи (beat). Если сброс попал между ними, приёмник ждёт второй beat, которого не будет: у CoupledL2 встаёт SinkC. Beat-ы разных сообщений в канале не перемежаются [TL 1.8.1 4, с. 18], и все beat-ы начатого сообщения должны прийти [TL 1.8.1 5, с. 28].
3. **L1D и директория L2 расходятся.** CoupledL2 инклюзивна: в её директории у каждой строки L1D стоит бит «копия в L1D» и права этой копии. DCache — write-back: строку с правом T она меняет только у себя. Если после сброса DCache пуста, грязные данные потеряны, а L2 продолжает считать, что L1 держит строку. По спецификации права клиента уменьшаются только сообщениями — Release [TL 1.8.1 9.1.1, с. 64] или ProbeAck в ответ на Probe [TL 1.8.1 9.3.5, с. 76]; сброс L1D не посылает ни того, ни другого. После AcquirePerm у L2 нет даже старой копии (раздел 1.3.1).
4. **Выходы ядра в момент сброса.** Установка сброса асинхронная: регистры ядра сбрасываются не по фронту такта, и его выходы в домен тайла (valid TileLink, подтверждения MSI, сигналы состояния) меняются в произвольный момент. Триггер тайла, защёлкнувший такой сигнал, может уйти в метастабильность (раздел 1.6).

Первые две проблемы решаются одинаково во всех конфигурациях: граница ядро / L2 опустошается до сброса, и во время сброса через неё ничего не проходит (гейты `TLQuiesceGate`, автомат `CoreResetCtrl`, автомат `DCacheFlushUnit`). Так спецификация и советует сбрасывать одну сторону связи: через адаптер, у которого две стороны сбрасываются раздельно, и без запросов без ответа в момент сброса [TL 1.8.1 3.2.3, с. 14]. Гейт — такой адаптер. Третья проблема решается по-разному — это и выбирает параметр `CoreResetKeepL1`: L1D либо сохраняет строки, либо перед сбросом отдаёт их все в L2. Четвёртую решает изоляция: на время сброса все выходы ядра заменяются на стороне тайла значениями, не зависящими от ядра.

Ссылки на спецификацию. Пометка вида [TL 1.8.1 3.2.2, с. 14] (синяя) — раздел 3.2.2 на странице 14 спецификации «SiFive TileLink Specification», версия 1.8.1 (2020); [TL 1.9.3 3.2.2, с. 15] (зелёная) — тот же раздел в последней версии 1.9.3, на странице 15. Цитаты из спецификации даны по-английски. Все пункты, на которые ссылается документ, — в таблице раздела 2.1; пункты, на которых держится обход, — ещё и в разделе 1.3.2.

1.1. Что изменилось относительно v4; один параметр и две конфигурации

v5 исправляет и упрощает v4 по запросу: убран импульсный сброс; если ядро в сбросе, гейт либо блокирует транзакцию (L2 ждёт ответ на Probe) при сохранении metaArray, либо сам отвечает при обходе; канал A некогерентных агентов закрывается как можно раньше (в sFlush); ожидание 16 тактов заменено флагами пустоты; в sDrain канал B закрывается максимально быстро.

Что	v4	v5
Компиляция	не компилировалась: `TLQuiesceGate.name` конфликтует с `LazyModule.name`	компилируется; поле называется `gateName`
Режимы сброса	импульс (100 тактов, sHold, второй импульс) или уровень (`CoreResetLevel`)	только уровень: ≥ `rstCycles` = 100 тактов и до снятия запроса; sHold, второй импульс, `releaseCycles` и `CoreResetLevel` удалены
Состояния `CoreResetCtrl`	sIdle, sFlush, sDrain, sReset, sRelease, sHold	sIdle, sFlush, sDrain, sReset, sRelease
Канал A	все 5 гейтов закрывают A только в sDrain	некогерентные (ICache, PTW, i_mmio, d_mmio) — с первого такта sFlush (`earlyBlock`); l1d — с sDrain (`gateBlock`)
Канал B (Probe)	держится / отвечается только с sReset	закрыт с первого такта sDrain (`holdProbe`); в sReset и sRelease: держится (KEEP) или отвечает гейт (SWEEP, `ackProbe`)
Выход из sFlush	по `l1.done`	`l1Done && earlyQuiet`: тишина ранних гейтов (зависимость Get → Probe в CoupledL2, 1.5.1)
Выход из sDrain	16 тактов подряд `gatesIdle && l1Done`; `idle` гейта включал `b.valid` L2	в том же такте `gatesQuiet && l1Done && l1Idle`; `QuiesceGateIO` без `idle`, есть только `quiet`
Опустошение DCache	MissQueue и WBQ пусты 16 тактов подряд	флаги пустоты `mq_empty`, `wbq_empty`, `mp_empty`, `pq_empty` (1.5.3); новое: `MainPipe.busy`, `ProbeQueue.empty` (файл Probe.scala)
sRelease	≥ 8 тактов и `alive`	`l1.alive` и выход frontend из сброса (`resetInFrontend`), без счётчика
Теги L1D при KEEP	стирались при каждом сбросе ядра (`TagSRAMBank.rst_cnt`): дефект, найденный ревью	`tagArray` на сбросе тайла вместе с `metaArray` и `errorArray`
Параметры	`CoreResetKeepL1`, `CoreResetLevel`	только `CoreResetKeepL1`
Порты	`debug_reset`, `core_reset_req`, `core_reset_busy`	без изменений

Одна и та же последовательность подходит обеим конфигурациям; различается подготовка L1D и то, что происходит с Probe, пока ядро в сбросе.

	CoreResetKeepL1 = true (KEEP)	CoreResetKeepL1 = false (SWEEP)
L1D после сброса ядра	все строки с прежними правами и данными («горячая»)	пуста; всё отдано в L2 до сброса
директория L2 после сброса	верна: L1D держит то же, что до сброса	верна: за L1D не числится ничего
домен сброса `metaArray`, `errorArray`, `tagArray`	тайла (порт `l1_meta_reset`)	ядра — как в upstream
подготовка к сбросу	опустошение DCache (флаги пустоты): десятки тактов	опустошение и обход: Release / ReleaseData всех действительных строк (≥ 1024 запросов flush) и ожидание последнего ReleaseAck
Probe в sDrain	держится в L2 с первого такта (`holdProbe`)	держится с первого такта (`holdProbe`)

	CoreResetKeepL1 = true (KEEP)	CoreResetKeepL1 = false (SWEEP)
Probe в sReset и sRelease	держится в L2 весь сброс; после выхода ядра доходит до DCache и получает ответ из «горячей» L1D	отвечает гейт: ProbeAck NtoN (`ackProbe`)
маска записи `freeze` (meta, error, теги, данные)	да (`closed`)	не нужна (`freeze` = 0)
снупы HN во время сброса	к строке из L1D ждут конца сброса; к остальным L2 отвечает сама	не ждут (L2 отвечает сама)
когда выбирать	нужна горячая L1D; при сбросе никто не обращается к кэшируемой этим ядром памяти	отладка через SBA при удержанном сбросе; самая безопасная для отладчика схема

Параметр имеет смысл только при `EnableCoreReset = true`. Формальная и симуляционная проверка (раздел 7) проведены для обеих конфигураций.

1.2. CoreResetKeepL1 = true: что в L1D переживает сброс ядра

L1D ничего не забывает при сбросе ядра. Это возможно благодаря устройству DCache: данные лежат в SRAM без сброса, а состояние строк, флаги ошибок и теги переведены на сброс тайла.

Массив DCache	Что хранит	Реализация	Сброс ядра: KEEP	Сброс ядра: SWEEP
`tagArray`	физические теги + ECC	SRAM (`shouldReset = false`), но обёртка `TagSRAMBank` имеет счётчик инициализации `rst_cnt`, который после сброса пишет нули во все наборы	**на сбросе тайла**: `tagArray.reset := l1_meta_reset`; не меняется; запись под `freeze`	на сбросе ядра, как в upstream (не важно)
`bankedDataArray`	данные строк	SRAM, `shouldReset = false`	не меняется (запись под `freeze`)	не меняется (не важно)
`metaArray` (L1CohMetaArray)	состояние строки N / B / T / Dirty	триггеры, `RegInit`	сохраняется (сброс тайла; запись под `freeze`)	→ N (строки уже отданы)
`errorArray` (L1ErrorMetaArray)	флаги denied / corrupt от L2	триггеры, `RegInit`	сохраняется (сброс тайла; запись под `freeze`)	→ 0
`accessArray`, `prefetchArray`, замещение, WPU	подсказки для предвыборки и замещения	триггеры	→ 0	→ 0

Что нашла проверка: теги стирались (дефект v4, исправлен в v5)

Ревью Chisel выявило дефект, унаследованный от v4: `TagSRAMBank` — обёртка над SRAM — после сброса её домена записывает нули во все наборы (счётчик `rst_cnt`, он в домене сброса ядра). При KEEP `metaArray` сохранялась, а теги стирались при каждом сбросе ядра: строка оставалась «действительной» с тегом 0 — порча данных. Исправление: `tagArray` — на сброс тайла (`tagArray.reset := r`, DCacheWrapper), как `metaArray` и `errorArray`. В моделях добавлен мутант `TAG_MUTANT_CORE_RESET` (раздел 7.3).

Исходный код счётчика (файл не менялся, `TagArray.scala`):

```
55   class TagSRAMBank(index: Int)(implicit p: Parameters) extends AbstractTagArray {
     ...
66     val rst_cnt = RegInit(0.U(log2Up(nSets + 1).W))
67     val rst = rst_cnt < nSets.U
68     val rstVal = 0.U
69     val waddr = Mux(rst, rst_cnt, io.write.bits.idx)
70     val wdata = Mux(rst, rstVal, io.write.bits.asECCTag())
71     val wmask = Mux(rst || (DCacheWayDiv == 1).B, (-1).asSInt, io.write.bits.way_en.asSInt).asBools
72     val rmask = Mux(rst || (DCacheWayDiv == 1).B, (-1).asSInt, io.read.bits.way_en.asSInt).asBools
73     when (rst) {
74       rst_cnt := rst_cnt + 1.U
75     }
76
```

«Холодной» DCache делает только сброс 2048 бит состояния и счётчика тегов. Если эти биты (и флаги ошибок) сбрасывать только вместе с тайлом, после сброса ядра DCache видит все свои строки с прежними правами, в том числе грязные. Директория L2 по-прежнему верна, данные не

теряются, кэш остаётся тёплым, а L2 забирает грязные данные из L1D обычным Probe.

Сбрасываются управляющая логика DCache (MainPipe, MissQueue, WritebackQueue, ProbeQueue) и всё ядро. Поэтому в момент сброса эта логика должна быть свободна (флаги пустоты, 1.5.3), а Probe от L2, пришедший во время сброса, должен подождать: ответить на него без L1D нельзя — данные строки, возможно, только там. Разрешения записи во все четыре массива формирует управляющая логика ядра, поэтому на время сброса и его снятия они закрыты `freeze` из домена тайла (разделы 1.6 и 5.5). Запись в массивы данных и meta проходит одну регистровую стадию: этот инвариант нулевого запаса описан в 1.5.3.

1.3. CoreResetKeepL1 = false: обход L1D (sweep), затем сброс вместе с ядром

Перед сбросом DCache отдаёт в L2 каждую действительную строку — тем же путём, каким вытесняет строку при промахе: MainPipe читает тег и состояние строки, посылает добровольный Release (чистая строка) или ReleaseData (грязная) и в том же такте пишет в `metaArray` состояние N. Обход идёт по всем парам (набор, путь): 256 наборов × 4 пути = 1024 запроса. Затем DCache сбрасывается вместе с ядром — `metaArray` и `errorArray` на сбросе ядра, как в upstream. Значит:

- **Данные не теряются.** Грязная строка уходит в L2 сообщением ReleaseData с параметром TtoN, чистая — Release TtoN (была T) или BtoN (была B) [TL 1.9.3 9.3.10, с. 81], [TL 1.9.3 9.3.11, с. 82]. Строки в состоянии N обход пропускает: Release с NtoN запрещён [TL 1.9.3 9.3.10, с. 81].
- **Директория L2 точна.** Каждый Release снимает бит клиента; после последнего ReleaseAck CoupledL2 не числит за L1D ни одной строки (раздел 1.3.3).
- **Probe во время сброса по-прежнему отвечает гейт:** ProbeAck NtoN [TL 1.9.3 9.3.5, с. 78] — правдивый отчёт пустой L1D. С CoupledL2 такой Probe не приходит: он посылается только строке с битом клиента, а их не осталось. Ответчик — страховка для менеджера, который пробует клиента без копии (TileLink это допускает, раздел 1.3.2).
- **Опустошение границы остаётся** — оно нужно протоколу, а не строкам: начатый Acquire должен получить GrantAck, ReleaseData не должна оборваться между beat-ами, ответ не должен прийти в сброшенное ядро [TL 1.9.3 3.2.3, с. 15].
- **Цена — время:** около 1024 тактов на запросы flush (один запрос за такт, если MainPipe принимает), Release только для действительных строк (данные — только у грязных) и ожидание последнего ReleaseAck. После сброса L1D пуста, первые обращения ядра — промахи; данные L2 получила при обходе.

	CoreResetKeepL1 = true (keep)	CoreResetKeepL1 = false (sweep)
данные грязных строк	остаются в L1D, L2 заберёт их Probe	в L2 (ReleaseData при обходе)
Probe, пока ядро в сбросе	ждёт в L2 (гейт держит канал B)	гейт отвечает NtoN (с CoupledL2 не приходит)
что переживает сброс ядра	`metaArray`, `errorArray`, `tagArray` (сброс тайла); данные (SRAM, запись под `freeze`)	ничего (как в upstream)

1.3.1. Почему полный сброс L1D невозможен: AcquirePerm

Если L1D просто сбросить вместе с ядром (без обхода), строки и грязные данные теряются, а устаревшую директорию L2 исправлял бы первый Probe (ответ NtoN). Это корректно, только если у L2 есть действительная, пусть и старая, копия каждой строки L1D. DCache XiangShan этого не гарантирует: запись целой строки при промахе получает права без данных — сообщением AcquirePerm.

XiangShan, cache/dcache/mainpipe/MainPipe.scala: промах записи, покрывающей всю строку

```
894     miss_req.full_overwrite := s2_req.isStore && s2_req.store_mask.andR
```

XiangShan, cache/dcache/mainpipe/MissQueue.scala: AcquirePerm вместо AcquireBlock

```
259       val acquirePerm = edge.AcquirePerm(
260         fromSource = mshr_id,
261         toAddress = get_block_addr(req.addr),
262         lgSize = (log2Up(cfg.blockBytes)).U,
263         growPermissions = grow_param
264       )._2
265       acquire := Mux(req.full_overwrite, acquirePerm, acquireBlock)
266       // resolve cache alias by L2
      ...
```

```
848     io.mem_acquire.bits := Mux(full_overwrite, acquirePerm, acquireBlock)
```

`full_overwrite` — запись из sbuffer покрывает все 64 байта строки (`store_mask.andR`). Тогда MissEntry посылает AcquirePerm (право T без данных), получает Grant без данных и пишет строку целиком: в L1D строка Dirty, а у L2 действительных данных нет. CoupledL2 поступает так же: AcquirePerm превращается в CHI MakeUnique (права без данных), и в DataStorage ничего не пишется:

coupledL2/tl2chi/MSHR.scala: AcquirePerm → MakeUnique; Grant без записи DataStorage

```
380         *  AcquirePerm NtoT     |  MakeUnique
381         *  AcquirePerm BtoT     |  MakeUnique
        ...
803       mp_grant.tagWen := !cmo_cbo && !dirResult.hit && !denied
804       mp_grant.dsWen := (gotGrantData || probeDirty && (req_get || req.aliasTask.getOrElse(false.B))) && !denied
```

`dsWen` задачи Grant равен 1 только при `gotGrantData` (данные пришли от HN) — у MakeUnique их нет. Тег пишется (`tagWen`), директория получает TRUNK с битом клиента, а в DataStorage остаётся содержимое пути от прежней строки — данные другого адреса. Спецификация прямо требует от менеджера помнить об этом:

Пункт TileLink 1.9.3	Текст
[TL 1.9.3 9.3.2, с. 75] AcquirePerm	«AcquirePerm must be only used in situations where no copy of the data is required to complete the initiating operation. The primary example is the case where the block is being acquired in order to be entirely overwritten.» «For a Slave Agent to maintain coherence on a block after AcquirePerm, the Slave Agent must track "data not valid" until the Master Agent performs an evict using ReleaseData, responds with ProbeAckData to a ProbeBlock, or writes all bytes dirty in the whole block using PutFullData.»
[TL 1.9.3 9.3.6, с. 79] ProbeAckData	«It is illegal to use the NtoN permission transition in a ProbeAckData message, as it is illegal for a Master Agent to hold dirty data in the N state.»

Полный сброс L1D нарушает этот договор. Единственная действительная копия строки — грязная строка в L1D — исчезает без ReleaseData и без ProbeAckData. L1D после сброса действительно в N и на Probe честно отвечает NtoN, но L2 ждёт данных («data not valid»). По ProbeAck NtoN CoupledL2 переводит строку TRUNK → TIP (шаг 6 раздела 1.3.4) и с этого момента считает своей копией то, что лежит в DataStorage, — данные другого адреса. Дальше они уйдут HN в ответе на снуп или при вытеснении либо ядру в GrantData: тихая порча памяти, а не просто потеря последних записей. Поэтому вариант без KeepL1 — обход: строка после AcquirePerm грязная и уходит в L2 ReleaseData, как и требует [TL 1.9.3 9.3.2, с. 75].

Почему так: обход, а не полный сброс или «сброс с правкой директории»

Исправить это на стороне L2 (например, считать NtoN на строку TRUNK признаком потери данных) нельзя: данных всё равно нет нигде. Обход сохраняет и данные, и протокол: он пользуется только обычными сообщениями TileLink (Release, ReleaseData) и обычным путём вытеснения MainPipe, без изменений в CoupledL2. Цена — около тысячи тактов до сброса и небольшое изменение MainPipe (раздел 5.5). Проверка v4 этого не ловила: модель DCache промахивалась только AcquireBlock. Теперь модель DCache шлёт AcquirePerm на промах записи всей строки, а модель L2 после такого Grant помечает свою копию недействительной (poison) и сообщает об ошибке, если когда-нибудь её отдаст.

1.3.2. На какие пункты спецификации TileLink опирается обход

Пункты TileLink 1.9.3 — последней версии (зелёные ссылки; страницы — по PDF), в скобках — тот же пункт версии 1.8.1 (синие ссылки), если он там есть. Цитаты дословные.

Утверждение	Пункт	Текст спецификации
Release — добровольный запрос мастера на понижение прав; параметр — Shrink или Report без NtoN	[TL 1.9.3 9.3.10, с. 81], табл. 41	«A Release message is a request message used by a Master Agent to voluntarily downgrade its permissions on a cached data block.» c_param: «Shrink (TtoB, TtoN, BtoN) or Report sans NtoN (TtoT, BtoB).»
Строку в N обход не трогает: Release NtoN запрещён	[TL 1.9.3 9.3.10, с. 81]	«It is illegal to use the NtoN permission transition in a Release message, as a Master Agent with no permission to a block should have no reason to report on its permission to that block.»

Утверждение	Пункт	Текст спецификации
Грязная строка — ReleaseData, тоже без NtoN	[TL 1.9.3 9.3.11, с. 82]	«A ReleaseData message is a request message used by a Master Agent to voluntarily downgrade its permissions on a cached data block. and write back dirty data to the managing Slave Agent.»
Строка после AcquirePerm перестаёт быть «data not valid» именно по ReleaseData	[TL 1.9.3 9.3.2, с. 75]	«until the Master Agent performs an evict using ReleaseData»
На каждый Release менеджер отвечает ReleaseAck; обход ждёт последнего	[TL 1.9.3 9.3.12, с. 82]	«A ReleaseAck message is a response message used by a Slave Agent to acknowledge the receipt of a Release[Data], and is in turn used to ensure global serialization of operations by the Slave Agent.»
После Release мастер не отвечает на Probe к этой строке и не шлёт по ней Acquire до ReleaseAck	[TL 1.9.3 9.2, с. 70] ([TL 1.8.1 9.2, с. 69])	«Once the Release is issued, the master should not issue a ProbeAck, ProbeAckData, Acquire, Get, Put, ArithmeticData, LogicalData, or further Release on the block until it receives a ReleaseAck from the slave acknowledging completion of the writeback.»
Гонка Release и Probe: менеджер принимает и Release, и обогнавший ProbeAck	[TL 1.9.3 9.2, с. 71] ([TL 1.8.1 9.2, с. 69])	«The slave must accept the voluntary Release as well as any overtaken ProbeAcks resulting from Probe messages that have already been processed by the master before the Release was sent»
Ответ гейта в сбросе: NtoN — Report, допустим на любой Probe	[TL 1.9.3 9.3.5, с. 78], табл. 36	c_param: «Shrink (TtoB, TtoN, BtoN) or Report (TtoT, BtoB, NtoN).» «the latter reports what they were and continue to be.»
Сбросить одну сторону связи можно через адаптер и без запросов без ответа	[TL 1.9.3 3.2.3, с. 15] ([TL 1.8.1 3.2.3, с. 14])	«a TileLink-to-TileLink adapter is needed which acts as a slave in one domain and a master in the other domain.» «ensure that there are no unanswered TileLink requests when one half of the crossing is reset or depowered.»
Сброс связи: valid = 0 не меньше 100 тактов	[TL 1.9.3 3.2.2, с. 15] ([TL 1.8.1 3.2.2, с. 14])	«The valid signals must be driven LOW for at least 100 cycles while reset is asserted.» «Ready, control, and data signals are free to take any value during reset.»

1.3.3. Как CoupledL2 принимает Release и ReleaseData обхода

Для L2 обход — серия обычных добровольных вытеснений из L1D. Путь: SinkC (канал C) → RequestArb → MainPipe банка; ответ ReleaseAck уходит через GrantBuffer в канал D. Код — CoupledL2 55d8e69, вариант tl2chi (номера строк — по этой ревизии).

coupledL2/SinkC.scala: Release(Data) — в буфер задач, данные — в dataBuf

```
 47    val isRelease = io.c.bits.opcode(1)
 48    val hasData = io.c.bits.opcode(0)
     ...
107    when (io.c.fire && isRelease) {
108      when (hasData) {
109        when (first) {
110          dataBuf(nextPtr)(beat) := io.c.bits.data
111          beatValids(nextPtr)(beat) := true.B
112        }.otherwise {
113          assert(last)
114          dataBuf(nextPtrReg)(beat) := io.c.bits.data
115          beatValids(nextPtrReg)(beat) := true.B
116        }
117      }
118    }
119
     ...
120    when (io.c.fire && isRelease && last) {
121      when (hasData) {
122        taskValids(nextPtrReg) := true.B
123        taskBuf(nextPtrReg) := toTaskBundle(io.c.bits)
124        taskBuf(nextPtrReg).bufIdx := nextPtrReg
125      }.otherwise {
126        taskValids(nextPtr) := true.B
127        taskBuf(nextPtr) := toTaskBundle(io.c.bits)
128        taskBuf(nextPtr).bufIdx := nextPtr
129      }
130    }
```

coupledL2/tl2chi/MainPipe.scala: директория, DataStorage и ReleaseAck для Release(Data)

```
460     }.otherwise { // req_s3.fromC
461       sink_resp_s3.bits.opcode := ReleaseAck
462       sink_resp_s3.bits.param := 0.U // param of ReleaseAck must be 0
    ...
482     val wen_c = sinkC_req_s3 && isParamFromT(req_s3.param) && req_s3.opcode(0) && dirResult_s3.hit
    ...
573     val metaW_s3_c = MetaEntry(
574       dirty = meta_s3.dirty || wen_c,
575       state = Mux(isParamFromT(req_s3.param), TIP, meta_s3.state),
576       clients = Fill(clientBits, !isToN(req_s3.param)),
577       alias = meta_s3.alias,
578       accessed = meta_s3.accessed,
579       tagErr = Mux(wen_c, req_s3.denied, meta_s3.tagErr),
580       dataErr = Mux(wen_c, req_s3.corrupt, meta_s3.dataErr) // update error when write DS
581     )
```

- `isRelease = opcode(1)` (Release 6, ReleaseData 7): сообщение становится задачей MainPipe, данные ReleaseData — в `dataBuf` (оба beat-а). Ответы на Probe (opcode 4, 5) идут другим путём — откликом MSHR (раздел 1.3.4).
- `metaW_s3_c` — новая запись директории: `clients = !isToN(param)` — для TtoN и BtoN бит L1D снимается; `state` — TIP, если L1D отдаёт T, иначе прежнее; `dirty` — если пришли данные (`wen_c`).
- `wen_c` — запись в DataStorage: ReleaseData от владельца T с попаданием в директорию. Так L2 получает последние данные строки, в том числе строки, взятой AcquirePerm («data not valid» снят, [TL 1.9.3 9.3.2, с. 75]).
- Ответ — `sink_resp_s3` с `opcode = ReleaseAck`, `param = 0`. По нему DCache освобождает запись WritebackQueue; обход `DCacheFlushUnit` ждёт, пока WBQ пуста (`idle`).
- После обхода у каждой строки, которую держала L1D, бит клиента снят — директория L2 точна, и снупы HN к этим строкам L2 обслуживает сама, без Probe.

1.3.4. Как и в каких модулях CoupledL2 обрабатывает ответ на Probe

Ответы на Probe нужны обеим конфигурациям: во время опустошения и обхода Probe обслуживает DCache (ProbeAck или ProbeAckData), а пока ядро в сбросе, Probe либо ждёт (KEEP), либо получает ответ гейта (SWEEP, NtoN). L2 состоит из нескольких банков (Slice); Probe и ответ на него обрабатывает банк, которому принадлежит адрес строки.

Модуль (файл)	Роль в обработке ProbeAck
MainPipe (tl2chi/MainPipe.scala)	по записи директории решает, нужен ли Probe, и выделяет MSHR; заранее копирует строку из DataStorage в releaseBuf; в конце пишет директорию и данные и отправляет ответ HN
MSHR (tl2chi/MSHR.scala, 16 штук)	ведёт транзакцию: выставляет Probe, ждёт ProbeAck, по нему меняет свою копию записи директории (meta), затем ставит задачу в MainPipe
SourceB (SourceB.scala, внутри MSHRCtl)	очередь Probe в канал B на 4 записи; не пускает Probe к строке, на Grant которой ещё нет GrantAck
SinkC (SinkC.scala)	приёмник канала C: ProbeAck и ProbeAckData превращает в отклик `resp`, данные ProbeAckData — в запись releaseBuf
MSHRCtl (tl2chi/MSHRCtl.scala)	по адресу находит MSHR, которая ждёт ответа, передаёт ей отклик и даёт номер записи releaseBuf
releaseBuf (MSHRBuffer.scala)	по строке данных на каждую MSHR: сначала копия из DataStorage, ProbeAckData её перезаписывает; отсюда берутся данные для ответа HN, вытеснения или Grant

Пример: снуп SnpUnique от HN к строке, которую директория числит за L1D, и ответ NtoN

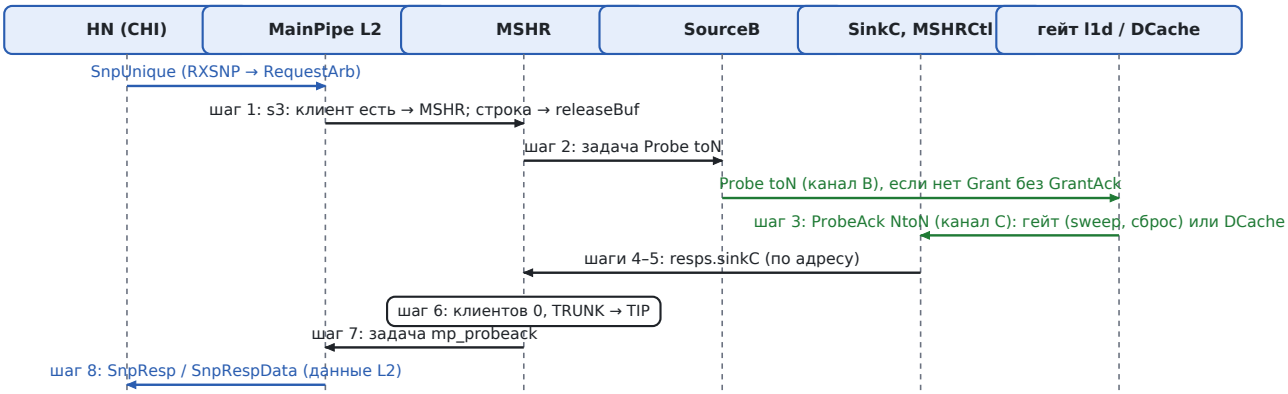


Рис. 1. Путь Probe и ответа на него в банке CoupledL2. Остальные источники Probe (вытеснение из L2, Get от ICache, alias, CBO) идут тем же путём; отличается задача MSHR на шаге 7.

Шаг 1. Нужен ли Probe — MainPipe, стадия s3.

```
242     val need_probe_s3_a = dirResult_s3.hit && meta_has_clients_s3 && (
243       req_get_s3 && (meta_s3.state === TRUNK) ||
244       req_cbo_clean_s3 && (meta_s3.state === TRUNK) ||
245       req_cbo_flush_s3 ||
246       req_cbo_inval_s3
247     )
      ...
283     val need_pprobe_s3_b_snpToN = req_s3.fromB && (
284       isSnpUniqueX(req_s3.chiOpcode.get) ||
285       req_s3.chiOpcode.get === SnpCleanInvalid ||
286       isSnpMakeInvalidX(req_s3.chiOpcode.get)
287     ) && dirResult_s3.hit && meta_has_clients_s3
288     val need_pprobe_s3_b_snpNDERR = req_s3.fromB && tagError_s3 && dirResult_s3.hit
```

- `need_pprobe_s3_b_snpToN` — снуп HN, который отнимает строку: Probe нужен, если строка есть в директории и у неё есть клиент. Снупы, понижающие до B, и SnpOnce / Query / Stash шлют Probe только строке TRUNK.
- `need_probe_s3_a` — Get (ICache, PTW) или CBO к строке с клиентом; для Get и CBOClean — только TRUNK: L2 нужна последняя копия. Это зависимость Get → Probe, из-за которой выход sFlush ждёт тишины ранних гейтов (1.5.1).
- Вытеснение из L2: MSHR промаха получает от директории жертву; если у жертвы есть клиент — `s_rprobe = 0`.
- `need_write_releaseBuf`: если клиент может ответить без данных, MainPipe заранее читает строку из DataStorage и пишет её в releaseBuf своей MSHR: на ProbeAck без данных releaseBuf уже держит копию L2.

Шаги 2–3. Probe в канал B — MSHR, MSHRCtl, SourceB; ответ — гейт или DCache.

```
281     io.tasks.source_b.valid := !state.s_pprobe || !state.s_rprobe
      ...
430     val b_task = {
431       val ob = io.tasks.source_b.bits
432       ob.tag := dirResult.tag
433       ob.set := dirResult.set
434       ob.off := 0.U
435       ob.opcode := Probe
436       ob.param := Mux(
437         !state.s_pprobe,
438         Mux(
439           snpToB,
440           toB,
441           Mux(snpToN, toN, toT)
442         ),
443         Mux(
444           // *NOTICE: CBOClean derives upper Probe toB for now.
445           (req_get || req_cboClean) && dirResult.hit && meta.state === TRUNK,
446           toB,
447           toN
448         )
449       )
450       ob.alias.foreach(_ := meta.alias.getOrElse(0.U))
451       ob
452     }
```

```
  79    val conflictMask = io.grantStatus.map(s =>
  80      s.valid && s.set === io.task.bits.set && s.tag === io.task.bits.tag
  81    )
  82    val conflict     = Cat(conflictMask).orR
  83
      ...
 115      when(p.valid && !io.grantStatus(p.waitG).valid) {
 116        p.rdy := RegNext(true.B) // cuz GrantData has 2 beats, can move RegNext elsewhere
 117      }
```

- `b_task.param`: для снупа — по типу снупа (toB, toN, иначе toT); для Get и CBOClean к TRUNK — toB; для вытеснения, alias и остальных CBO — toN.
- SourceB держит Probe (`rdy = 0`), пока по той же строке есть Grant без GrantAck: «Once the Grant is issued, the slave should not issue Probes on that block until it receives a GrantAck.» [TL 1.8.1 9.2, c. 68].
- Ответ: вне сброса отвечает DCache (ProbeQueue → MainPipe → WritebackQueue; для строки, которой нет, — ProbeAck NtoN); в сбросе (SWEEP) гейт l1d формирует `edgeOut.ProbeAck(probe, NtoN)` по [TL 1.9.3 9.3.5, c. 78]. CoupledL2 source ответа не смотрит (шаги 4 и 5), поэтому оба варианта равноценны.

Шаги 4–5. Приём ответа — SinkC; чей это ответ — MSHRCtl.

```
 147    io.resp.valid := io.c.valid && (first || last) && !isRelease
 148    io.resp.mshrId := 0.U // DontCare
 149    io.resp.tag := parseAddress(io.c.bits.address)._1
 150    io.resp.set := parseAddress(io.c.bits.address)._2
 151    io.resp.respInfo := 0.U.asTypeOf(io.resp.respInfo.cloneType)
 152    io.resp.respInfo.opcode := io.c.bits.opcode
 153    io.resp.respInfo.param := io.c.bits.param
      ...
 161    val probeAckDataBuf = RegEnable(io.c.bits.data, 0.U((beatBytes * 8).W),
 162      io.c.valid && io.c.bits.opcode === ProbeAckData && first)
 163
 164    io.releaseBufWrite.valid := io.c.valid && io.c.bits.opcode === ProbeAckData && last
 165    io.releaseBufWrite.bits.id := 0.U(mshrBits.W) // id is given by MSHRCtl by comparing address to the MSHRs
 166    io.releaseBufWrite.bits.data.data := Cat(io.c.bits.data, probeAckDataBuf)
 167    io.releaseBufWrite.bits.beatMask := Fill(beatSize, true.B)
      ...
 186    io.c.ready := !isRelease || !first || !full
```

```
 125    val resp_sinkC_match_vec = mshrs.map { mshr =>
 126      val status = mshr.io.status.bits
 127      val tag = Mux(status.needsRepl, status.metaTag, status.reqTag)
 128      mshr.io.status.valid && status.w_c_resp && io.resps.sinkC.set === status.set && io.resps.sinkC.tag === tag
 129    }
      ...
 184    io.releaseBufWriteId := ParallelPriorityMux(resp_sinkC_match_vec, (0 until mshrsAll).map(i => i.U))
```

- `io.c.ready` для ProbeAck всегда 1 (`!isRelease`): SinkC никогда не задерживает ответы на Probe.
- `io.resp` комбинационный: из адреса — tag и set; opcode, param; `last`; `dirty = opcode(0)`; `denied` и `corrupt` — из `c.corrupt`. Поля source и size не используются.
- MSHR ищется по адресу, не по source: она занята, ждёт ответа на канале C (`w_c_resp`), совпадают набор и тег. Данные ProbeAckData пишутся в releaseBuf записи, найденной тем же вектором совпадений.

Шаг 6. Что меняет ответ — MSHR, обработка c_resp.

```
1094    val c_resp = io.resps.sinkC
1095    val rxrsp = io.resps.rxrsp
1096    val rxdat = io.resps.rxdat
1097    // Probe core response
1098    when (c_resp.valid) {
1099      when (c_resp.bits.opcode === ProbeAck || c_resp.bits.opcode === ProbeAckData) {
1100        state.w_rprobeackfirst := true.B
1101        state.w_rprobeacklast := state.w_rprobeacklast || c_resp.bits.last
1102        state.w_pprobeackfirst := true.B
1103        state.w_pprobeacklast := state.w_pprobeacklast || c_resp.bits.last
1104      }
1105      when (c_resp.bits.opcode === ProbeAckData) {
1106        probeDirty := true.B
1107        meta.dirty := true.B
1108      }
1109      when (isToN(c_resp.bits.param)) {
1110        meta.state := Mux(isT(meta.state), TIP, meta.state)
1111        meta.clients := Fill(clientBits, false.B)
1112      }
1113      when (isToB(c_resp.bits.param)) {
1114        meta.state := Mux(isT(meta.state), TIP, meta.state)
1115      }
1116      when (TLPermissions.isReport(c_resp.bits.param)) {
```

```
1117          meta.tagErr := c_resp.bits.denied
1118          meta.dataErr := c_resp.bits.corrupt
```

```
  52    def isToN(param: UInt): Bool = {
  53      param === TLPermissions.TtoN || param === TLPermissions.BtoN || param === TLPermissions.NtoN
  54    }
  55    def isToB(param: UInt): Bool = {
  56      param === TLPermissions.TtoB || param === TLPermissions.BtoB
  57    }
  58    def isToT(param: UInt): Bool = {
  59      param === TLPermissions.TtoT
  60    }
  61    def isT(state: UInt): Bool = state(1)
```

- Любой ProbeAck(Data) ставит `w_*probeackfirst/last`. ProbeAckData: `probeDirty = 1`, `meta.dirty = 1`.
- `isToN(param)` — TtoN, BtoN и NtoN: бит клиента снимается, TRUNK → TIP (L2 — единственный владелец). После NtoN L2 продолжает со своей копией — поэтому эта копия обязана быть верной; после AcquirePerm без ReleaseData это не так (раздел 1.3.1).

Ответ клиента	meta.clients	meta.state	Данные в releaseBuf
ProbeAck NtoN (гейт, пустая DCache)	0	TRUNK → TIP, иначе как было	копия из DataStorage
ProbeAck TtoN (чистая T)	0	TRUNK → TIP	копия из DataStorage
ProbeAck BtoN	0	как было (TIP / BRANCH)	копия из DataStorage
ProbeAckData TtoN (грязная)	0	TRUNK → TIP; `probeDirty = 1`	данные L1D
ProbeAck(Data) TtoB, BtoB	1	TRUNK → TIP	с данными — L1D, иначе копия

Главное следствие. Для MSHR ответ NtoN неотличим от ответа чистой L1D, которая отдала строку T без данных (TtoN): одна и та же ветка кода — L2 продолжает со своей копией. Это безопасно, только если копия L2 актуальна. В режиме sweep так и есть: всё, что L1D меняла, она отдала ReleaseData до сброса, а после обхода директория её вообще не числит.

Шаги 7–8. Задача MSHR в MainPipe и запись директории. Ни одна задача, зависящая от ответа, не начнётся до него: ответ HN (`mp_probeack_valid`) ждёт `w_pprobeacklast`; вытеснение и Grant — `w_rprobeacklast`. Для отнимающего снупа строка становится INVALID, клиентов нет; для понижающего — BRANCH. Грязность после NtoN — только грязность L2; `dsWen = 0` без `probeDirty`, DataStorage не пишется; данные для HN читаются из releaseBuf (копия L2). Ответ — SnpResp или SnpRespData с данными L2.

```
 284    val mp_probeack_valid = !state.s_probeack && state.w_pprobeacklast
       ...
 680      mp_probeack.dsWen := !(snpToN || tagErr) && probeDirty && !releaseDirty
```

Почему гейт по-прежнему отвечает NtoN, если с CoupledL2 Probe не придёт

Гейт — адаптер на границе сброса [TL 1.9.3 3.2.3, с. 15]: он не должен полагаться на то, как устроен менеджер. TileLink разрешает Probe клиенту без копии, ответ — Report NtoN [TL 1.9.3 9.3.5, с. 78]; держать такой Probe весь сброс (а сброс держат сколько угодно) значило бы задерживать снуп HN без причины. Ответ пустой L1D известен заранее, и гейт даёт его сам. В проверке L2 нарочно иногда пробует строки, которых нет в директории (раздел 7.3).

1.4. Сброс-уровень: ядро в сбросе, пока держат запрос

Debug Module держит `ndmreset` или `hartreset` столько, сколько решит отладчик: бит в `dmcontrol` остаётся 1, пока отладчик не запишет 0 (это могут быть миллионы тактов). Контроллер держит `coreReset`, пока запрос высокий, но не меньше `rstCycles` = 100 тактов, и отпускает ядро один раз — после снятия запроса. Состояния sHold и второго импульса нет.

- **Probe с KEEP** держится в L2 — не только в сбросе, а с первого такта sDrain — и доходит до DCache после выхода ядра из сброса. Грязная строка отвечает ProbeAckData из «горячей» L1D.
- **Probe со SWEEP** в sDrain держится (DCache ещё есть, но новый Probe ей уже не нужен: строки отданы), в sReset и sRelease гейт сам отвечает ProbeAck NtoN. Снупы HN L2 обслуживает сама (директория L1D не числит); риска блокировки нет.
- **Что исчезло:** состояние sHold (ядро вне сброса, но стоит), второй импульс после снятия запроса, `releaseCycles` (≥ 8 тактов в sRelease), `CoreResetLevel`, счётчики 16 тактов тишины.

- Ядро стартует ровно после снятия запроса и с текущим reset vector.

Почему так: с KEEP ожидание Probe в сбросе допустимо, но у него есть цена

Пока `ready` = 0, beat просто не принят [TL 1.8.1 4.1, с. 18]; длину ожидания спецификация не ограничивает, но требует, чтобы предложенный beat когда-нибудь был принят, если его не задерживает сообщение с более высоким приоритетом (правило 3.i, [TL 1.8.1 5, с. 28]). Здесь оно выполняется, если запрос когда-нибудь снимут. Есть и системное ограничение: снуп HN, ждущий Probe, держит запись в HN, а с ней — всех, кто обращается к той же строке. Если снятие запроса зависит от такой строки (запись через SBA отладчиком), ожидание бесконечно — взаимная блокировка: отладчик держит сброс и пишет программу в память через системную шину DM; адрес лежит в L1D; HN шлёт снуп, L2 — Probe; Probe ждёт в гейте, пока ядро в сбросе; запись SBA ждёт снуп; отладчик ждёт запись и не снимает сброс. Поэтому с KEEP ответственность за то, чтобы никто не ждал строку L1D во время сброса, лежит на системе; для отладки через SBA есть SWEEP (раздел 8).

1.5. Раннее закрытие A, быстрое закрытие B, флаги пустоты

Три решения v5, которые сокращают подготовку к сбросу и делают её точной. Диаграмма — в 4.1; сигналы автомата — в 4.4; временные диаграммы реальных прогонов — в 7.4.

1.5.1. Некогерентные агенты: A закрыт с первого такта sFlush; зависимость Get → Probe

ICache, PTW, `i_mmio` и `d_mmio` — некогерентные клиенты (TL-UL: только каналы A и D). Их запросы после сброса никому не нужны, а начатый запрос должен получить ответ до сброса. Поэтому гейты этих рёбер (`early = true`) закрывают канал A с первого такта sFlush (`earlyBlock` = состояние ≠ sIdle) — на границе сообщения, чтобы не оборвать многобитное Put. Гейт l1d (TL-C) закрывает A только с sDrain: в sFlush DCache ещё делает Release (обход) и отвечает на Probe.

Зависимость Get → Probe в CoupledL2. Get некогерентного агента к строке, которую L1D держит в TRUNK / TIP, ждёт в L2 Probe toB к L1D (`need_probe_s3_a`, 1.3.4). Значит, пока у некогерентного агента открыт хоть один Get, канал B закрывать нельзя: Probe не дойдёт, Get не получит ответа, а сброс без ответа на Get недопустим. Поэтому выход из sFlush ждёт `earlyQuiet` (все ранние гейты quiet) — и только после этого B закрывается (в sDrain). Это одна из причин, по которой B нельзя закрыть вместе с ранним A.

Почему так: A некогерентных — рано, B — только после их тишины

Закрыть A некогерентных с первого такта безопасно и полезно: новые запросы только удлиняли бы опустошение. Нельзя то же самое сделать с B и с A l1d: L1D ещё должна ответить на Probe (в том числе по требованию незавершённого Get) и вернуть Release. Мутант `NO_EARLY_BLOCK` (A не закрыт рано) ловится свойством P9 и симуляцией; мутант `NO_EARLY_QUIET` (выход из sFlush без тишины ранних гейтов) — P10.

1.5.2. B закрыт с первого такта sDrain; решение по quiet текущего такта

`holdProbe` — комбинационная функция регистра состояния (`state === sDrain` или `closed`), поэтому канал B закрывается в первом же такте sDrain, а не на такт позже, когда «решение» дошло бы через регистр. Гейт снимает `b.ready` к L2 и `b.valid` к ядру в том же такте; снятие `valid` непринятого beat-а законно: «a sender may raise valid and then lower it on the following cycle, even if the message was not accepted on the previous cycle.» [TL 1.8.1 4.1, с. 19]. L2 продолжает предлагать Probe (SourceB держит его), и он будет принят после сброса (KEEP) или получит ответ гейта (SWEEP).

Решение sDrain → sReset принимается по `quiet` **текущего такта** (`gatesQuiet && l1Done && l1Idle`, комбинационно из счётчиков гейтов), а не по регистровому значению: иначе Probe, принятый в последнем такте sFlush, проскакивает (мутант `STALE_QUIET`). Что остаётся открытым в sDrain: только Probe, принятый гейтом l1d в последнем такте sFlush — его ProbeAck идёт через буфер `l1d_to_l2_buffer` (между DCache и гейтом), счётчик `probes` гейта его видит, и sDrain ждёт его (диаграмма 7.4, случай 5).

Из длинных прогонов (soak), число	KEEP	SWEEP
фаз sDrain	5869	5869
самая длинная sDrain, тактов	85	84
Probe удержано в sDrain	354	315
Get, ждавших Probe (get-probes)	68129	65798
направленный Probe в 1-м такте sDrain: предложен в sDrain / позже	250 / 0	250 / 0
направленный Probe в последнем такте sFlush: попал / позже	249 / 1	249 / 0

Обычно sDrain длится 1 такт (флаги пустоты уже подняты в sFlush); с Probe последнего такта sFlush — пока его ProbeAck(Data) пройдёт через буфер C; с медленной L2 — до десятков тактов. Числа — из файлов `coverage_*.json` (сумма по прогонам сценариев 1–7 в модели).

1.5.3. Флаги пустоты вместо 16 тактов

В v4 DCache и контроллер ждали 16 тактов подряд тишины, чтобы поймать «окна», в которых признак пустоты мог на такт оказаться ложным. В v5 каждая часть управляющей логики DCache даёт **точный** признак: что-то находится в ней ровно тогда, когда признак низкий. Тогда ждать нечего.

Флаг	Структура и выражение	Почему точен
`mq_empty`	MissQueue: `!busy_i` по 16 записям (`busy = req_valid`), `!miss_req_pipe_reg.reg_valid()`, `cmo_unit.io.req.ready`	запись занята с приёма промаха до установки строки и GrantAck; регистр между приёмом и записью (в нём запрос между «разрешён» и «занял запись») учтён отдельно; CMO — в unit
`wbq_empty`	WritebackQueue: `!block_addr.valid_i` по записям (`state =/= s_invalid`)	запись живёт от приёма Release / ProbeAck до ReleaseAck (Release) или последнего beat (ProbeAck): пуста — значит все Release подтверждены, все ProbeAck отправлены
`mp_empty`	MainPipe: `!(s1_valid || s2_valid || s3_valid)` (`MainPipe.io.busy`)	запрос, принятый на s0, покидает свою очередь на том же фронте, на котором встаёт `s1_valid`: между очередью и стадиями запроса нет; значит, запись из sbuffer / AMO / Probe / flush учтена либо очередью, либо `busy`
`pq_empty`	ProbeQueue: `!block_addr.valid_i` по записям и `!selected_req_valid` (`ProbeQueue.empty`)	Probe находится в записи или в регистре выбора перед MainPipe — других мест нет

`DCacheFlushUnit.idle` = все четыре флага. Выходы: sDrain → sSweep / sDone по `idle`; sWait → sDone по `inflight === 0 && idle`; `l1.idle` = `idle` — в контроллере. Новые места состояния в DCache, которые держат что-либо до сброса, требуют собственного флага (раздел 9).

Инвариант нулевого запаса: запись в массивы данных и meta и момент coreReset

Флаги не покрывают одну регистровую стадию записи массивов данных (`BankedDataArray`) и meta (`AsynchronousMetaArray`): запись, вызванная `s3_fire` в предыдущем такте, фиксируется в массиве на фронте, который заканчивает такт решения. Это тот же фронт, на котором поднимается регистр `coreReset`, а установка сброса асинхронна: запись успевает пройти до сброса. Следствие: между решением sDrain → sReset и поднятием `coreReset` нельзя добавлять задержку (регистр, ожидание) — запись попала бы под сброс (KEEP: испорченная строка переживает сброс). Инвариант записан в комментариях `CoreResetCtrl` и `DCacheFlushUnit`.

`l1.idle` в контроллере с нынешним DCache избыточен (в sDone занят может быть только путь Probe, а каждый Probe считает гейт): он оставлен как прямое требование — свойство P19; мутант `NO_L1_IDLE` ожидаемо эквивалентен в симуляции.

1.6. Изоляция домена сброса ядра (пересечение доменов сброса)

Сброс ядра формирует ResetGen: `withClockAndReset(clock, (coreBaseReset || coreReset).asAsyncReset) { ResetGen() }`. Его установка асинхронная: регистр `coreReset` контроллера переключается по фронту такта, а дальше сброс идёт по дереву без участия такта — через асинхронные входы триггеров ResetGen XSTile и следующих ResetGen внутри ядра (MemBlock → Backend → Frontend). Синхронизировано только снятие. Значит, после фронта, на котором поднялся `coreReset`, регистры XSCore сбрасываются в разные, заранее не известные моменты, а вместе с ними меняются — и могут дребезжать — выходы ядра. Статический анализ проверяет для асинхронного сброса только recovery / removal на снятии; путь «установка сброса → выход ядра → триггер тайла» им не покрыт. Это классическая задача пересечения доменов сброса (RDC); решают её изоляцией: на стороне принимающего домена сигнал заменяется значением, которое не зависит от сбрасываемого домена.

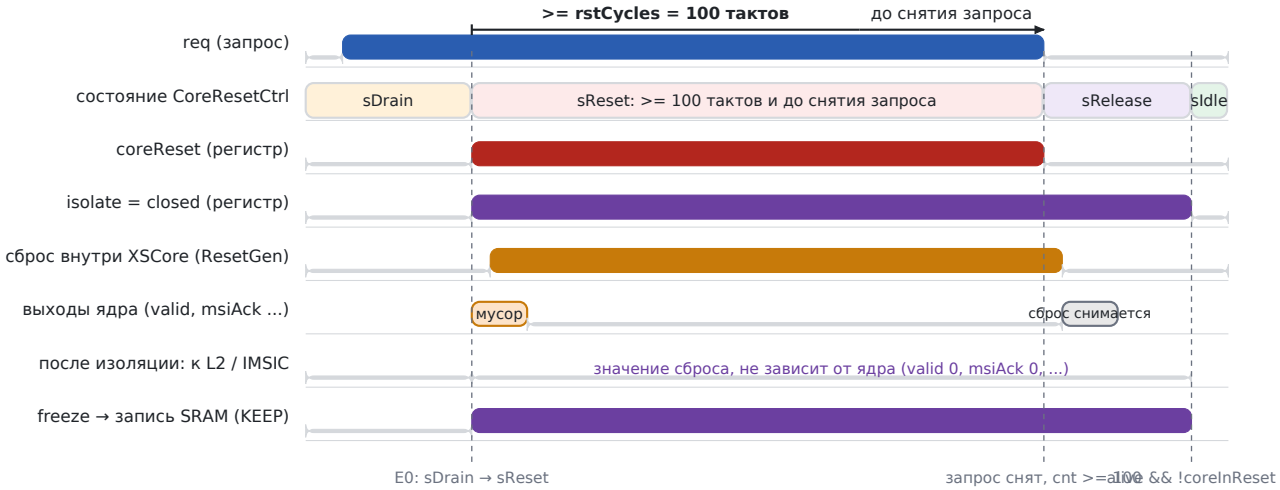


Рис. 2. Сброс ядра и изоляция: sReset длится не меньше 100 тактов и до снятия запроса. `coreReset` и `isolate` поднимаются на одном фронте (E0); сброс внутри ядра приходит позже и в неизвестный момент, выходы ядра в это время произвольны («мусор»), после изоляции тайл видит значение сброса без глитчей.

Принцип. `isolate` — регистр `closed` контроллера: ставится на том же фронте, что и `coreReset` (переход sDrain → sReset), и снимается при выходе из sRelease (`l1.alive && !coreInReset`), когда DCache и frontend уже вне сброса. Это регистр тайла, поэтому сам он не дребезжит. Асинхронный путь начинается с того же регистра `coreReset`, поэтому раньше E0 + t_cq ни один выход ядра измениться не может; `isolate` к этому времени тоже переключился, путь от него до маски — обычный синхронный. Снятие сброса синхронно, поэтому выходы ядра после него меняются только по фронтам, и `isolate` можно снять после выхода DCache и frontend из сброса.

Какое значение подставлять. Для каждого управляющего выхода ядра — одно из двух: значение в сбросе (`isoReset`): valid — 0, подтверждения рукопожатий (`msiAck`) — 0, «frontend в сбросе» — 1, признаки состояния (`cpu_halt`, `cpu_critical_error`, `l2_flush_en`) — 0; или значение до сброса (`isoHold`, `RegEnable(x, !isolate)`) — для уровней конфигурации без рукопожатия (настройки предвыборщика L2, `mstatus` для трассы, `robTrueCommit`). Поле данных, которое читают только вместе с valid, отдельной изоляции не требует. Входы ядра (прерывания через IntBuffer, `msiInfo`, `clintTime`, reset vector, `hartId`, `l2_hint`, `perfEvents`) изоляции не требуют: их формирует домен тайла синхронно.

> **Почему msiAck — значение сброса, а не удержание**
>
> Доставка MSI в IMSIC ядра — четырёхфазное рукопожатие уровнями: мост поднимает `vld_req`, IMSIC в CSR ядра отвечает `vld_ack = vld_req` (регистр), мост ждёт фронта `vld_ack` и снимает запрос. Если бы `msiAck` удерживался равным 1 весь сброс, а запрос к этому моменту уже снова стоял, ядро после сброса подняло бы `vld_ack` из 0 в 1 за один такт — ещё под удержанием, — и после снятия изоляции мост увидел бы постоянную 1 без фронта и ждал бы вечно. Значение сброса (0) показывает мосту то же, что показал бы сброс ядра. (Прерывания, уже принятые IMSIC ядра, теряются при сбросе ядра в любом случае: файл прерываний — часть ядра.)

Где стоит изоляция. Все сигналы из домена сброса ядра в домен тайла (сверено с XSTile v5):

Сигнал (из ядра)	Вид	Изоляция, пока isolate = 1	Где
TileLink l1d: A valid, C valid, E valid, D ready, B ready	TL-C, ядро — клиент	A — `block` и `isolate`; C, E — valid = 0 (собственный ProbeAck гейта проходит); D ready = 0; B — `holdB` (KEEP) или `ackB` (SWEEP)	TLQuiesceGate (6.1)
TileLink l1i, ptw, i_mmio, d_mmio: A valid, D ready	TL-UL, ядро — клиент	A valid = 0, D ready = 0; к ядру D valid = 0	TLQuiesceGate
TileLink icachectrl: A ready, D valid	TL-UL, ядро — менеджер	к ядру A valid = 0; к тайлу A ready = 0, D valid = 0	TLResetIsolation (6.2)
предвыборка L1 → L2, → L3: `addr_valid`	valid + данные	0 (адрес, источник, `l2_pf_en` — данные)	XSTile (6.5)
`msiAck`, `teemsiAck`	рукопожатие (уровень)	0 — значение сброса	XSTile
`cpu_halt`, `cpu_critical_error`	состояние	0 — значение сброса	XSTile
`resetInFrontend` → `hartIsInReset`	состояние	1 — значение сброса; плюс `hartInReset` контроллера	XSTile
`l2_flush_en`	запрос (CSR)	0 — значение сброса	XSTile
`beu_errors.{icache,dcache,uncache}.ecc_error.valid`	valid + адрес	0	XSTile
трасса: `groups(i).valid`; `mstatus`	valid + данные; уровень	valid = 0; `mstatus` удерживается	XSTile
`l2_tlb_req`: `req.ready`, `resp.valid` (ядро — ответчик)	рукопожатие	0, 0; ответ (`resp.bits`, `l2_pmp_resp`) — данные	XSTile
`l2PfCtrl` → `pfCtrlFromCore`	конфигурация (CSR)	удерживается	XSTile
`debugTopDown.robHeadPaddr.valid`; `robTrueCommit`	valid + адрес; счётчик	valid = 0; счётчик удерживается	XSTile
`l1.done`, `l1.idle`, `l1.alive` (DCache → контроллер)	рукопожатие / состояние	`done`, `idle` смотрятся только при `closed = 0`, `alive` и `resetInFrontend` — только в sRelease (снятие синхронно)	CoreResetCtrl
записи в `tagArray`, `bankedDataArray`, `metaArray`, `errorArray`	разрешения записи (KEEP)	`freeze` = `closed`: valid записи = 0	DCacheWrapper (5.6)

Что не нуждается в изоляции. Регистр `isoHold` и маски стоят в домене тайла и сбрасываются только сбросом тайла. Интерфейс `l1_quiesce` в обратную сторону (`req`, `freeze`) и все остальные входы ядра идут из домена тайла. Внутренние связи ядра сбрасываются целиком и пересечения доменов не образуют.

Почему маски и на A, и на B, если канал A закрывает block, а B — holdB / ackB

Канал A гейт закрывает с sDrain (`block`, на границе сообщения), а опустошение гарантирует, что к сбросу A стоит на границе. Значит, маска `isolate` на A функционально лишняя — формальная проверка подтверждает: мутант без неё (`GATE_MUTANT_NO_ISO_A`) эквивалентен (7.2). Её оставили, чтобы ни один выход гейта к L2 во время сброса не зависел от выхода ядра даже теоретически, без опоры на опустошение. Для B отдельной маски нет: пока `isolate` = 1, всегда поднят `holdB` (KEEP) или `ackB` (SWEEP) — ровно та же маска; это проверяют assert в гейте и свойство P17.

Как это связано с правилом 100 тактов TileLink. «The valid signals must be driven LOW for at least 100 cycles while reset is asserted.» [TL 1.9.3 3.2.2, с. 15] ([TL 1.8.1 3.2.2, с. 14]). Внутри XSCore все TileLink-связи (DCache, ICache, PTW, их крестовины) сбрасываются вместе с ядром — для них правило выполняет длина сброса: `rstCycles` по умолчанию 100, и сброс держится не меньше. На границе изоляция даёт то же с обеих сторон: гейты и `TLResetIsolation` держат valid к ядру равным 0 всё время `isolate` (sReset — не меньше 100 тактов, и sRelease), а к L2 valid из ядра не проходит. «Ready, control, and data signals are free to take any value during reset» — поэтому ready ядра во время сброса может быть любым (у ProbeQueue в сбросе он 1), и изоляция не даёт ему дойти до L2.

2. Понятия и модули

2.1. TileLink между ядром и L2

TileLink — протокол шины из rocket-chip. Связь «клиент → менеджер» (здесь: кэш L1 → L2) состоит из пяти независимых каналов, у каждого свой valid/ready. TL-UL — только A и D (простые чтения и записи); TL-C — все пять, с кэшируемыми копиями и правами N (нет), B (чтение), T (запись). Приоритет каналов: «A < B < C < D < E» [TL 1.8.1 2.2, с. 12], и ответ всегда идёт по каналу с более высоким приоритетом, чем запрос [TL 1.8.1 5.1, с. 30]: ответ на Probe (C) не может ждать канала A.

Канал	Направление	Сообщения, которые встречаются здесь
A	ядро → L2	Get, PutFullData (ICache, PTW, MMIO); AcquireBlock / AcquirePerm (промах DCache); CBOClean / CBOFlush / CBOInval (cbo.*)
B	L2 → DCache	Probe (toN — отдать строку, toB — понизить до чтения)
C	DCache → L2	ProbeAck / ProbeAckData (ответ на Probe, отчёт TtoN, BtoN, TtoB, NtoN ...); Release / ReleaseData (добровольное вытеснение)
D	L2 → ядро	AccessAck / AccessAckData, Grant / GrantData, ReleaseAck, CBOAck
E	DCache → L2	GrantAck (подтверждение Grant)

Транзакция завершена, только когда прошли все её сообщения: запрос требует ответа, Grant — одновременно ответ и запрос [TL 1.8.1 5.1, с. 29]. Гейт считает именно эти пары (счётчики внутри `quiet`):

Пара	Что значит незавершённость	Счётчик гейта
запрос A → ответ D	менеджер (L2, MMIO-мост) обрабатывает запрос, ответ придёт в ядро	`reqs`
Grant → GrantAck	MSHR L2 ждёт подтверждения; до GrantAck L2 не шлёт Probe по этой строке [TL 1.8.1 9.2, с. 68]	`grants`
Probe → ProbeAck	MSHR L2 ждёт ответа L1, чтобы ответить на снуп HN или вытеснить строку [TL 1.8.1 9.2, с. 67]	`probes`
Release → ReleaseAck	L2 записывает данные и директорию; до ReleaseAck DCache не отвечает на Probe к этой строке и не шлёт по ней Acquire [TL 1.8.1 9.2, с. 69]	`releases`

Отчёт NtoN. ProbeAck несёт параметр из категорий Shrink или Report [TL 1.8.1 9.3.5, табл. 36, с. 76]: Shrink — права уменьшились (TtoB, TtoN, BtoN), Report — не изменились, ответ лишь сообщает текущие (TtoT, BtoB, NtoN) [TL 1.8.1 9.1.4, табл. 31, с. 65–66]. NtoN значит «прав нет». Probe с уровнем выше, чем права клиента, законен и прав не меняет [TL 1.8.1 9.3.3, с. 74], поэтому клиент без копии на любой Probe отвечает NtoN. NtoN от клиента, которого директория ещё числит владельцем, бывает и в обычной работе — в гонке Release и Probe [TL 1.8.1 9.2, с. 69] (ProbeAck NtoN законен). Этим ответом пользуется гейт в режиме SWEEP. В Release и ReleaseData NtoN запрещён [TL 1.9.3 9.3.10, с. 81], [TL 1.9.3 9.3.11, с. 82], в ProbeAckData — тоже [TL 1.9.3 9.3.6, с. 79].

Многотактные сообщения. Сообщения с данными строки (GrantData, ProbeAckData, ReleaseData, AccessAckData) идут двумя beat по 32 Б; на MMIO-рёбрах шина 8 Б. Beat-ы разных сообщений в канале не перемежаются [TL 1.8.1 4, с. 18], [TL 1.8.1 4.2, с. 21]. `edge.firstlast(канал)` из rocket-chip даёт `first` и `last`.

Удержание канала. Пока `ready` = 0, beat не принят: «If ready is LOW, the receiver must not process the beat and the sender must not consider the beat processed.» [TL 1.8.1 4.1, с. 18]. Получатель может держать так beat долго, но не бесконечно: предложенный beat должен когда-нибудь быть принят [TL 1.8.1 5, с. 28]. Так ждёт Probe с KEEP, пока ядро в сбросе. Отправитель же может снять `valid` непринятого beat-а: «a sender may raise valid and then lower it on the following cycle, even if the message was not accepted on the previous cycle.» [TL 1.8.1 4.1, с. 19] — так гейт закрывает канал A и снимает Probe к ядру (`holdB`).

Пункты спецификации TileLink 1.8.1, на которые ссылается документ (цитаты дословные); пункты 1.9.3 — в 1.3.2:

Пункт	Текст спецификации	Где
[TL 1.8.1 2.2, с. 12] приоритет каналов	«The prioritization of messages across channels is A < B < C < D < E, in order of increasing priority.»	2.1

Пункт	Текст спецификации	Где
[TL 1.8.1 3.2.2, с. 14] сброс	«Before deasserting reset, a_valid, c_valid, and e_valid must be driven LOW by the master, while b_valid and d_valid must be driven LOW by the slave.» «The valid signals must be driven LOW for at least 100 cycles while reset is asserted.» «Ready, control, and data signals are free to take any value during reset.»	1.6, 4.4
[TL 1.8.1 3.2.3, с. 14] пересечение доменов	«a TileLink-to-TileLink adapter is needed which acts as a slave in one domain and a master in the other domain.» «It is highly recommended that a crossing carefully coordinates with the rest of the SoC to ensure that there are no unanswered TileLink requests when one half of the crossing is reset or depowered.»	1, 4.3, 6.1
[TL 1.8.1 4, с. 18] управление потоком	«It is forbidden in TileLink to interleave the beats of different messages on a channel.»	1, 2.1
[TL 1.8.1 4.1, с. 18–19] ready / valid	«If ready is LOW, the receiver must not process the beat and the sender must not consider the beat processed.» «Note that a sender may raise valid and then lower it on the following cycle, even if the message was not accepted on the previous cycle.»	1.5, 4.4, 6.1
[TL 1.8.1 5, с. 28] свобода от взаимных блокировок	правило 2: «Agents must eventually present all beats of a received message»; правило 3: «Unless they have a higher priority message in flight or unanswered» — 3.i «Agents must eventually accept a presented beat», 3.ii «Agents must eventually answer a received request message»	1.4, 4.4
[TL 1.8.1 5.1, с. 29–30] термины	«A request message requires a response message.» «Notice that Grants are both request and response messages.»	1, 2.1
[TL 1.8.1 6.4, с. 43] идентификаторы	«Each outstanding request identifier of a channel in a particular link must be unique»	1, 4.2, 4.3
[TL 1.8.1 9.2, с. 67–69] транзакции TL-C	«The slave waits to receive a ProbeAck for every Probe that was sent.» «Once the Grant is issued, the slave should not issue Probes on that block until it receives a GrantAck.» «Masters must continue to process and respond to Probes even with an outstanding Grant pending in the network.» «The slave must accept the voluntary Release as well as any overtaken ProbeAcks»	1.3, 2.1, 6.1
[TL 1.8.1 9.3.3, с. 74] Probe	«Probing Master Agents to cap their permissions at a more permissive level than they currently have is allowed, and does not result in a permissions change.»	2.1
[TL 1.8.1 9.3.5, табл. 36, с. 76] ProbeAck	«A ProbeAck message is a response message used by a Master Agent to acknowledge the receipt of a Probe.» c_param: «Shrink or Report (TtoB, TtoN, BtoN, TtoT, BtoB, NtoN).»	1.3, 6.1
[TL 1.8.1 9.3.9, с. 79] GrantAck	«The GrantAck response message is used by the Master Agent to provide a final acknowledgment of transaction completion»	4.2

Diplomacy. Механизм rocket-chip, которым соединяются TileLink-узлы: `a := b` создаёт ребро, параметры согласуются автоматически. `TLAdapterNode` — узел-адаптер, вставляемый в ребро; на нём построены гейт и `TLResetIsolation`. `TLTempNode` — пустой узел-заглушка, который diplomacy убирает (без `EnableCoreReset` рёбра соединяются как в upstream).

2.2. Debug Module и его сбросы

Понятие	Что это
Debug Module (DM)	блок отладки RISC-V (спецификация RISC-V Debug): управляется отладчиком через JTAG (DTM), умеет останавливать харты, выполнять абстрактные команды, читать и писать память
`dmcontrol.ndmreset`	сброс всего, кроме DM: отладчик пишет 1, затем 0. Что именно сбрасывается, решает платформа; здесь — XSCore каждого тайла (L2 и CHI работают)
`dmcontrol.hartreset`	сброс выбранных хартов, по той же схеме 1 → 0
`hartIsInReset`	вход DM от харта: «харт в сбросе». По нему DM выставляет липкий бит havereset (`dmstatus.anyhavereset` / `allhavereset`), отладчик сбрасывает его записью `ackhavereset`
`haltreq`, `resethaltreq`	просьба остановить харт; с `resethaltreq` харт останавливается на первой команде после сброса. DM доводит это до ядра прерыванием отладки (`debug_int`); с этим патчем не связано
SBA (system bus access)	доступ DM к памяти через системную шину, без участия харта. Обычно так загружают программу; запись в строку, которая есть в L1D, вызывает снуп
`hartResetReq` (XSTileWrap, XSNoCTop)	в upstream — вход сброса всего тайла от DM hartreset. С `EnableCoreReset` он запускает этот механизм (объединяется с `debug_reset`)

2.3. Домены сброса

Домен сброса — множество регистров, сбрасываемых одним сигналом. Такт везде один, но есть пересечение доменов сброса: сигналы из домена, который сейчас сбрасывается, в домен, который работает. Запрос `debug_reset` приходит из домена DM — он асинхронный и проходит синхронизатор.

Домен	Что в нём	Сбрасывается
тайла	L2Top (L2, MMIO-мост, CHI-порт), гейты, `TLResetIsolation`, `CoreResetCtrl`, синхронизаторы запросов, регистры изоляции в XSTile; с KeepL1 — ещё `metaArray`, `errorArray` и `tagArray` DCache (со счётчиком инициализации тегов)	только сбросом тайла
ядра	всё остальное в XSCore: Frontend, Backend, MemBlock, управляющая логика DCache (MainPipe, MissQueue, WBQ, ProbeQueue, DCacheFlushUnit); в режиме SWEEP — и `metaArray`, `errorArray`, `tagArray`	сбросом тайла или `coreReset`
SRAM	массивы данных DCache и ICache (и теги — сами макросы SRAM)	никогда (содержимое не сбрасывается)

Сброс в XiangShan формирует ResetGen: установка асинхронная (регистры сбрасываются сразу), снятие синхронное — через цепочку триггеров, чтобы все регистры вышли из сброса в одном такте. Из-за асинхронной установки сигналы ядра в домен тайла на время сброса изолируются (раздел 1.6).

2.4. Справочник модулей

Модуль	Что это и зачем нужен
XSTileWrap	Обёртка тайла: синхронизирует сбросы и асинхронные входы, выводит порты в XSNoCTop. Здесь `hartResetReq` объединяется с `debug_reset`, и он больше не сбрасывает весь тайл.
XSTile	Тайл: XSCore + L2Top. Сюда вставлены гейты, контроллер, синхронизаторы запросов, формирование сброса ядра (и с KeepL1 — сброса массивов L1D) и `hartIsInReset`.
XSCore	Ядро: Frontend (выборка, ICache), Backend (ROB, исполнение, CSR), MemBlock. Новое: `l1_quiesce`, `l1_meta_reset`.
L2Top	Обёртка L2: CoupledL2, MMIO-крестовина и MMIO-мост в CHI, логгеры и буферы TileLink, CHI-порт.
CoupledL2	Кэш L2, инклюзивный по отношению к L1D. Директория хранит состояние строки (INVALID, BRANCH, TRUNK — право записи у L1, TIP — L2 владелец) и биты клиентов. MSHR — записи незавершённых транзакций L2. Не меняется.
MemBlock	Подсистема памяти ядра: LoadUnit / StoreUnit, LSQ, sbuffer, DCache, TLB, PTW, Uncache, предвыборка L1. Содержит `l1d_to_l2_buffer` между DCache и гейтом (домен сброса ядра).
DCache (L1D)	Кэш данных L1: 64 КБ, 4 пути × 256 наборов, строка 64 Б, write-back, VIPT. TL-C-клиент L2. Внутри — MainPipe, MissQueue, WritebackQueue, ProbeQueue и массивы.
`metaArray`, `errorArray`, `tagArray`	Состояние каждой строки (N, B, T, Dirty), её флаги ошибок и теги; запись — только из s3 MainPipe. С KeepL1 — на сбросе тайла.
`bankedDataArray`	SRAM данных; сбросом не очищается.
MainPipe	Конвейер DCache (s0–s3): запись из sbuffer, атомики, установка строки после промаха, Probe, вытеснение. Новое: запрос flush (вытеснение заданного пути набора) и выход `busy` = s1..s3.
MissQueue / MissEntry	16 записей промахов: Acquire → Grant(Data) → установка строки → GrantAck. Новое: `empty`.
WritebackQueue (WBQ)	18 записей: Release / ReleaseData и ProbeAck / ProbeAckData; запись Release живёт до ReleaseAck. Новое: `empty`.
ProbeQueue	8 записей: принимает Probe из канала B и передаёт их в MainPipe через регистр выбора. Новое: `empty`.
sbuffer, AtomicsUnit	Буфер зафиксированных записей и исполнитель AMO / LR / SC; пишут через MainPipe.
ICache, PTW / L2TLB	Кэш команд и обход таблиц страниц; к L2 обращаются только Get. ICache после сброса ядра пуст (`valid_array` — RegInit).
Uncache, instr uncache	Некэшируемые обращения данных (`d_mmio`) и выборка команд из MMIO (`i_mmio`).
BPU	Предсказатель переходов; в сбросе записывает reset vector в начальный адрес выборки.
L2-предвыборщик, `l2_tlb_req`	Предвыборщик BOP в CoupledL2 переводит виртуальные адреса через TLB ядра.
ResetGen	Генератор сброса XiangShan: асинхронная установка, синхронное снятие. Цепочка ядра: MemBlock → Backend → Frontend (frontend выходит последним: `resetInFrontend`).
AsyncResetSynchronizerShiftReg	Синхронизатор асинхронного уровня в домен такта (3 триггера, rocket-chip).
WFI clock gate (XSNoCTop)	При `WFIClockGate` такт Core + L2 отключается, пока ядро в WFI; запрос сброса и `core_reset_busy` его будят.
`TLQuiesceGate` (новый)	TileLink-адаптер на ребре ядро → L2Top: закрывает A (рано для некогерентных, с sDrain для l1d), держит B (`holdB`) или сам отвечает на Probe NtoN (`ackB`), изолирует сторону ядра, выдаёт `quiet`: через ребро ничего не идёт и не ожидается.
`TLResetIsolation` (новый)	TileLink-адаптер на ребре L2Top → регистры ICache ctrl (менеджер в ядре): изоляция и счётчик запросов без ответа; выход `quiet`.
`DCacheFlushUnit` (новый)	Автомат в DCache: блокирует новые операции, ждёт флаги пустоты и (SWEEP) обходит все строки запросами flush через MainPipe.
`L1QuiesceIO` (новый)	Интерфейс контроллер ↔ DCache: `req`, `freeze` → / ← `done`, `idle`, `alive`.
`CoreResetCtrl` (новый)	Автомат последовательности: sFlush → sDrain → sReset → sRelease. Регистры `closed` (= `isolate`), `coreReset`, `hartInReset`. Параметры `keepL1`, `rstCycles`.

3. Где находятся изменения

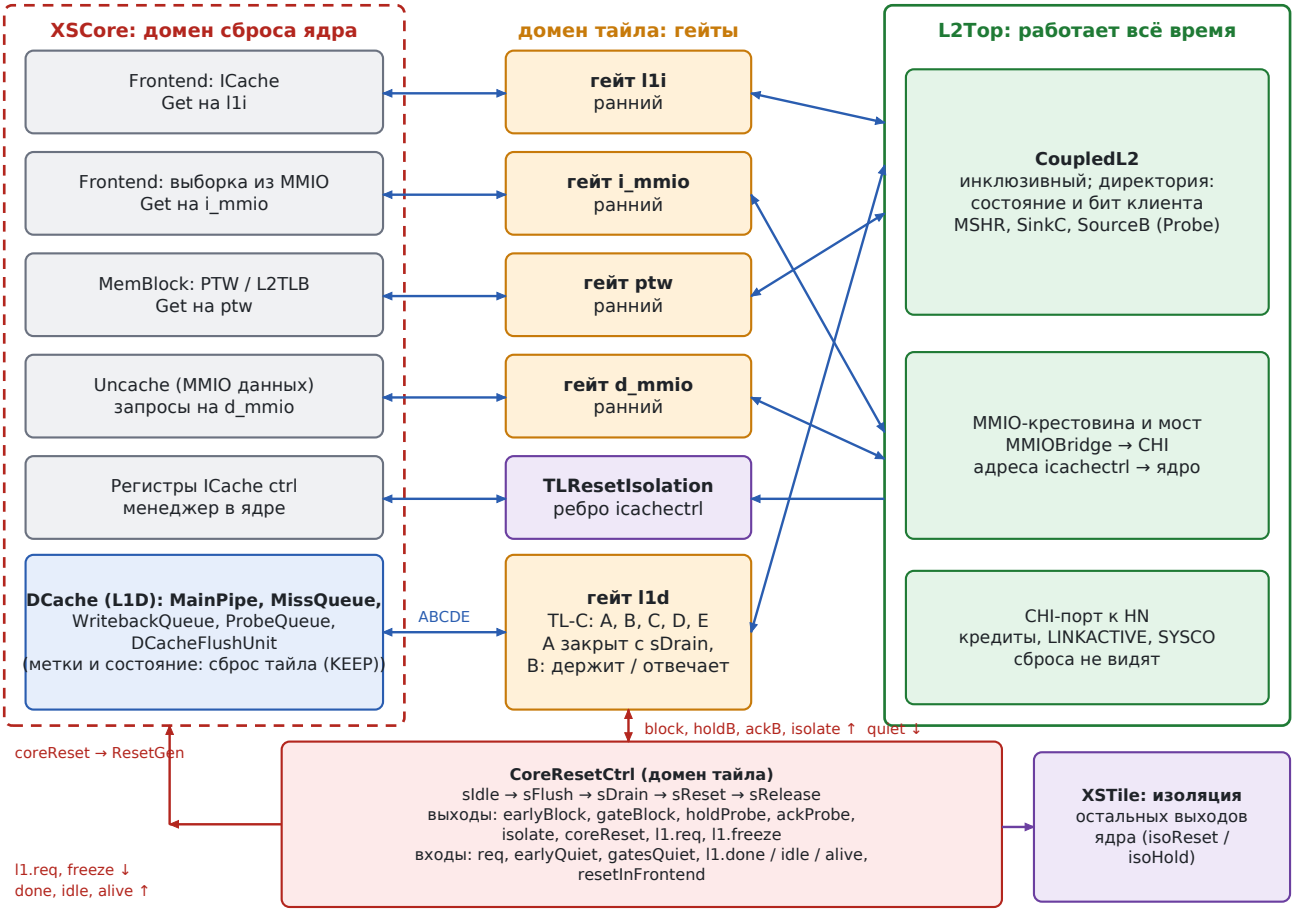


Рис. 3. XSTile с изменениями. Оранжевым — гейты (домен тайла), красным — домен сброса ядра и контроллер; двойные стрелки — TileLink-рёбра. Ребро icachectrl идёт из MMIO-крестовины L2Top обратно в ядро через `TLResetIsolation`. Остальные выходы ядра изолируются в XSTile (1.6).

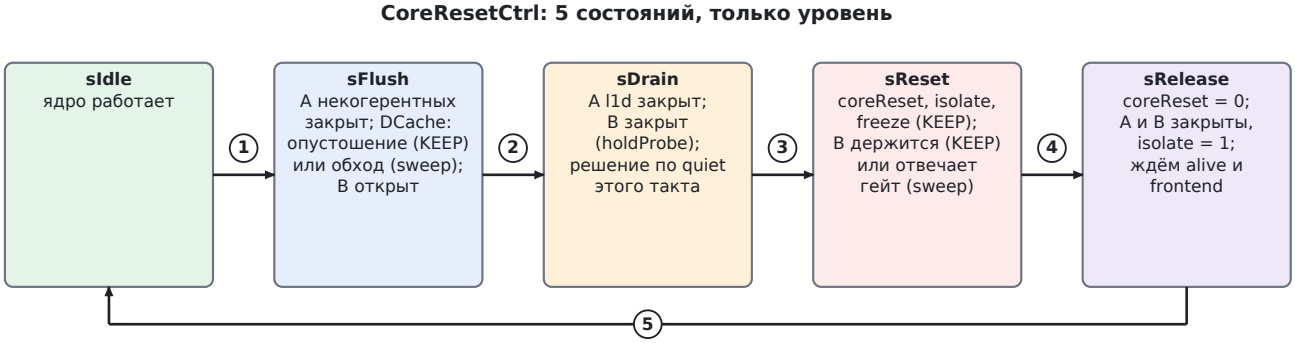
Файл (`src/main/scala/...`)	**Что изменено**	**+ / −**	**§**
`system/SoC.scala`	параметры `EnableCoreReset`, `CoreResetKeepL1`	+19 / −1	6.7
`top/Top.scala`	заглушки портов в XSTop	+2 / −0	6.6
`top/XSNoCTop.scala`	внешние порты, пробуждение такта на время последовательности	+22 / −1	6.6
`top/YamlParser.scala`	ключи YAML `EnableCoreReset`, `CoreResetKeepL1`	+12 / −0	6.7
`xiangshan/CoreReset.scala`	новый: `TLQuiesceGate` (ранний / обычный, ответчик на Probe, изоляция), `TLResetIsolation`, `CoreResetCtrl(keepL1, rstCycles = 100)`	+365 / −0	6.1–6.3
`xiangshan/XSCore.scala`	проброс `l1_quiesce`, `l1_meta_reset`	+7 / −0	6.4
`xiangshan/XSTile.scala`	порты, синхронизаторы, гейты на 5 рёбрах и `TLResetIsolation`, контроллер, изоляция всех выходов ядра, `hartIsInReset`, сбросы ядра и массивов L1D	+194 / −27	6.5
`xiangshan/XSTileWrap.scala`	порты, `hartResetReq` → `debug_reset`	+18 / −1	6.6
`xiangshan/cache/dcache/DCacheWrapper.scala`	порты `quiesce`, `metaReset`; флаги пустоты; `tagArray`, `metaArray`, `errorArray` на сбросе тайла; маски записи `freeze`; блокировки; маска WFI; две проверки	+89 / −1	5.6
`xiangshan/cache/dcache/FakeDCache.scala`	заглушка: `quiesce.done`, `idle`, `alive`	+6 / −0	5.7
`xiangshan/cache/dcache/mainpipe/FlushUnit.scala`	новый: `L1QuiesceIO` (`req`, `freeze`, `done`, `idle`, `alive`), `DCacheFlushUnit` (опустошение по флагам, обход)	+178 / −0	5.1

Файл (`src/main/scala/...`)	**Что изменено**	**+ / −**	**§**
`xiangshan/cache/dcache/mainpipe/MainPipe.scala`	запрос flush (обход): порт, арбитр, тег и состояние пути, Release и инвалидация в s3; `busy` = s1..s3	+67 / −19	5.5
`xiangshan/cache/dcache/mainpipe/MissQueue.scala`	`MissEntry.busy`, `MissQueue.empty` (с регистром приёма и CMO)	+6 / −0	5.2
`xiangshan/cache/dcache/mainpipe/Probe.scala`	новый в патче: `ProbeQueue.empty` (записи и регистр выбора)	+3 / −0	5.4
`xiangshan/cache/dcache/mainpipe/WritebackQueue.scala`	`WritebackQueue.empty`	+4 / −0	5.3
`xiangshan/mem/MemBlock.scala`	проброс `l1_quiesce`, `l1_meta_reset` до DCache	+6 / −0	6.4

Всё новое вне DCache генерируется только при `EnableCoreReset = true` (по умолчанию `false`); без него XSTile соединяет ядро и L2Top как в upstream, без масок. В DCache автомат есть всегда, но при `quiesce.req = 0` он не активен, и синтез убирает его как постоянную логику; обход (запросы flush) генерируется только при `EnableCoreReset` и `CoreResetKeepL1 = false` — иначе вход flush MainPipe получает константу 0; порт `metaReset`, перенос массивов (`metaArray`, `errorArray`, `tagArray`) в домен тайла и маски `freeze` — только при `EnableCoreReset` и `CoreResetKeepL1`. `CoreResetKeepL1` передаётся контроллеру как Scala-параметр: его ветви выбираются при генерации. В гейте неиспользуемый вход (`holdB` в SWEEP, `ackB` с KEEP) получает константу 0, и синтез убирает его логику.

4. Последовательность событий

Автомат `CoreResetCtrl` и его выходы — на рис. 4 и в таблице под ним; общая схема по фазам — на рис. 5; диаграммы KEEP и SWEEP — на рис. 6 и 7. Фазы 1–3 одинаковы во всех конфигурациях; различия по `CoreResetKeepL1` — обход L1D в фазе 2 и то, что происходит с L1D и с Probe во время сброса.

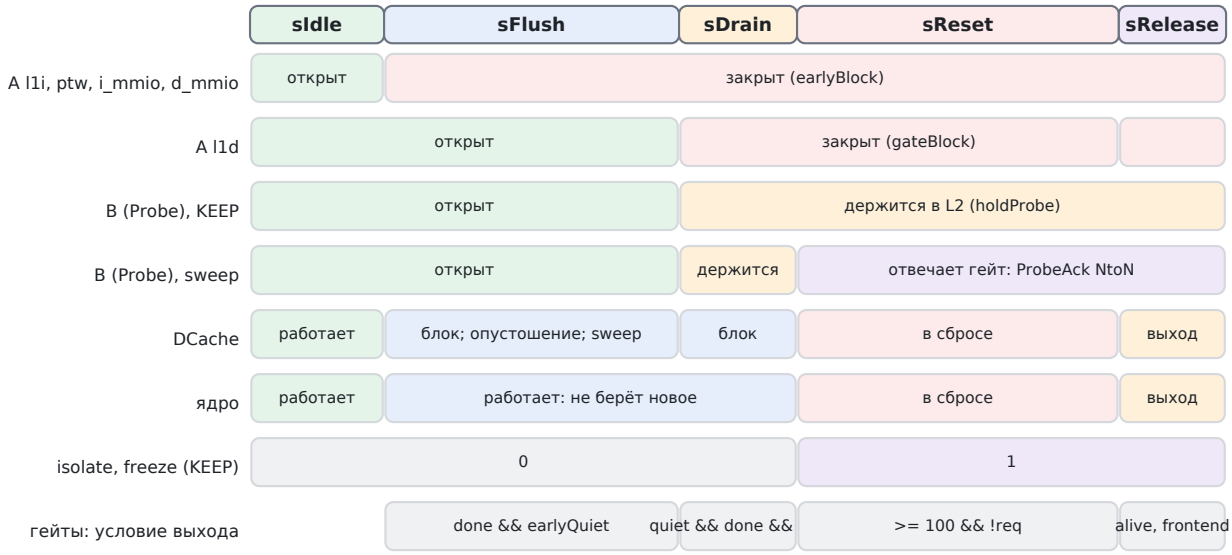


1 запрос req (синхронизированный debug_reset или core_reset_req)
2 l1Done && earlyQuiet: DCache опустошена (и обойдена), ранние гейты тихи
3 gatesQuiet && l1Done && l1Idle — в том же такте; coreReset, hartInReset, closed := 1
4 cnt >= rstCycles (100) && !req: запрос снят, сброс не короче 100 тактов; coreReset := 0
5 l1.alive && !coreInReset: DCache и frontend вне сброса; closed := 0, hartInReset := 0 (coreReset — регистр, поднимается на фронте 3)

Рис. 4. Состояния `CoreResetCtrl` (5 штук) и значения выходов. Все выходы — функции регистров состояния: `holdProbe`, `gateBlock`, `earlyBlock` меняются в том же такте, что и состояние.

Выход	sIdle	sFlush	sDrain	sReset	sRelease
`l1.req` (DCache заблокирована)	0	1	1	0	0
`earlyBlock` (A некогерентных закрыт)	0	1	1	1	1
`gateBlock` (A l1d закрыт)	0	0	1	1	1
`holdProbe` (B держится) — KEEP	0	0	1	1	1
`holdProbe` — SWEEP	0	0	1	0	0
`ackProbe` (гейт отвечает NtoN) — SWEEP	0	0	0	1	1
`l1.freeze` (массивы не пишутся) — KEEP	0	0	0	1	1
`isolate` (= `closed`)	0	0	0	1	1
`coreReset`	0	0	0	1 (регистр, с переходом)	0
`hartInReset` → `hartIsInReset`	0	0	0	1	1
`busy` → `core_reset_busy`	0	1	1	1	1

`isolate`, `ackProbe` (SWEEP), `l1.freeze` (KEEP) и `gateBlock` после sDrain — функции одного регистра `closed`: он поднимается на том же фронте, что и `coreReset`. `holdProbe` в sDrain — функция регистра `state`.



sReset длится не меньше rstCycles = 100 тактов и до снятия запроса; ширина столбцов условная

Рис. 5. Общая схема: что закрыто и открыто в каждом состоянии.

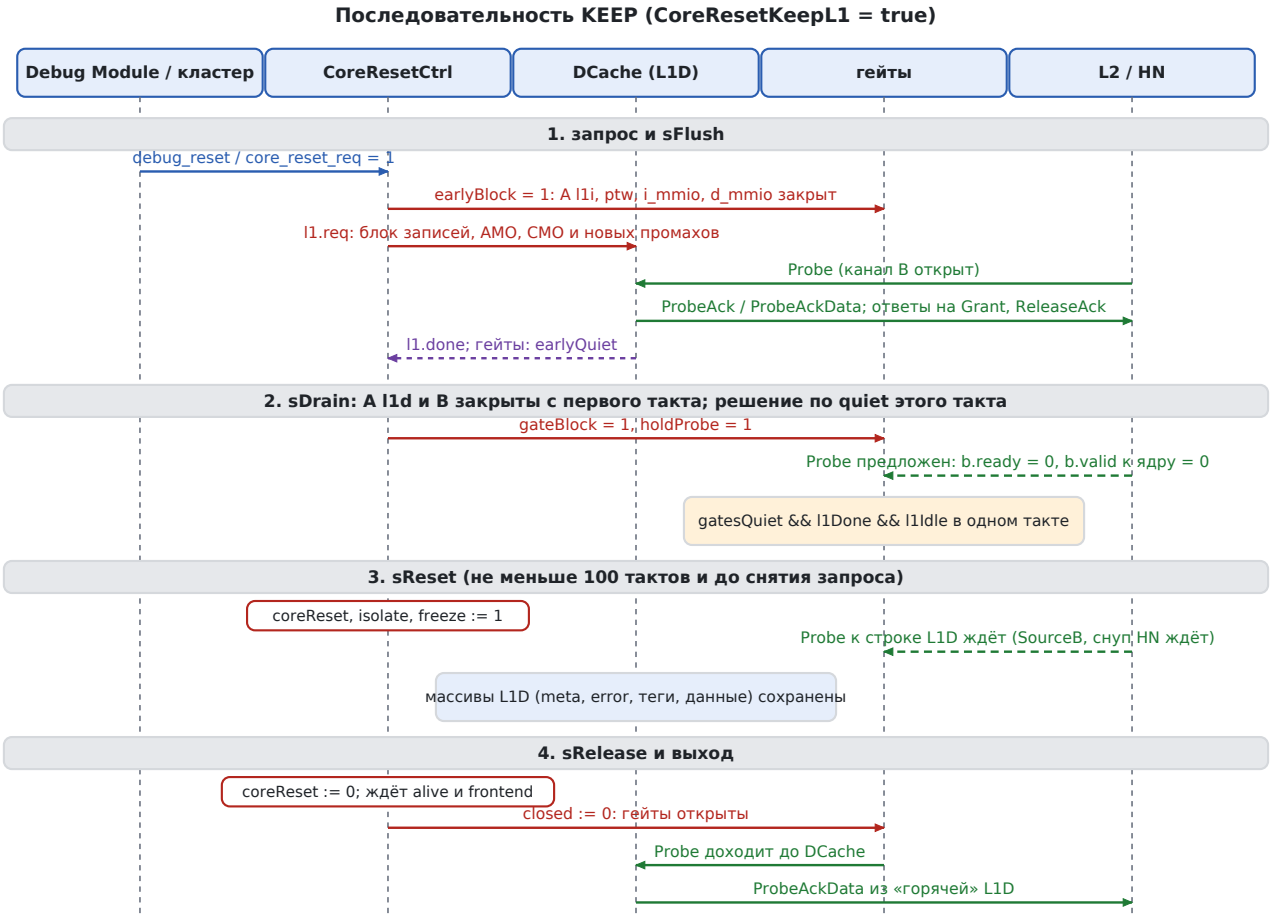


Рис. 6. Последовательность KEEP: Probe от L2 держится с первого такта sDrain и до выхода ядра из сброса.

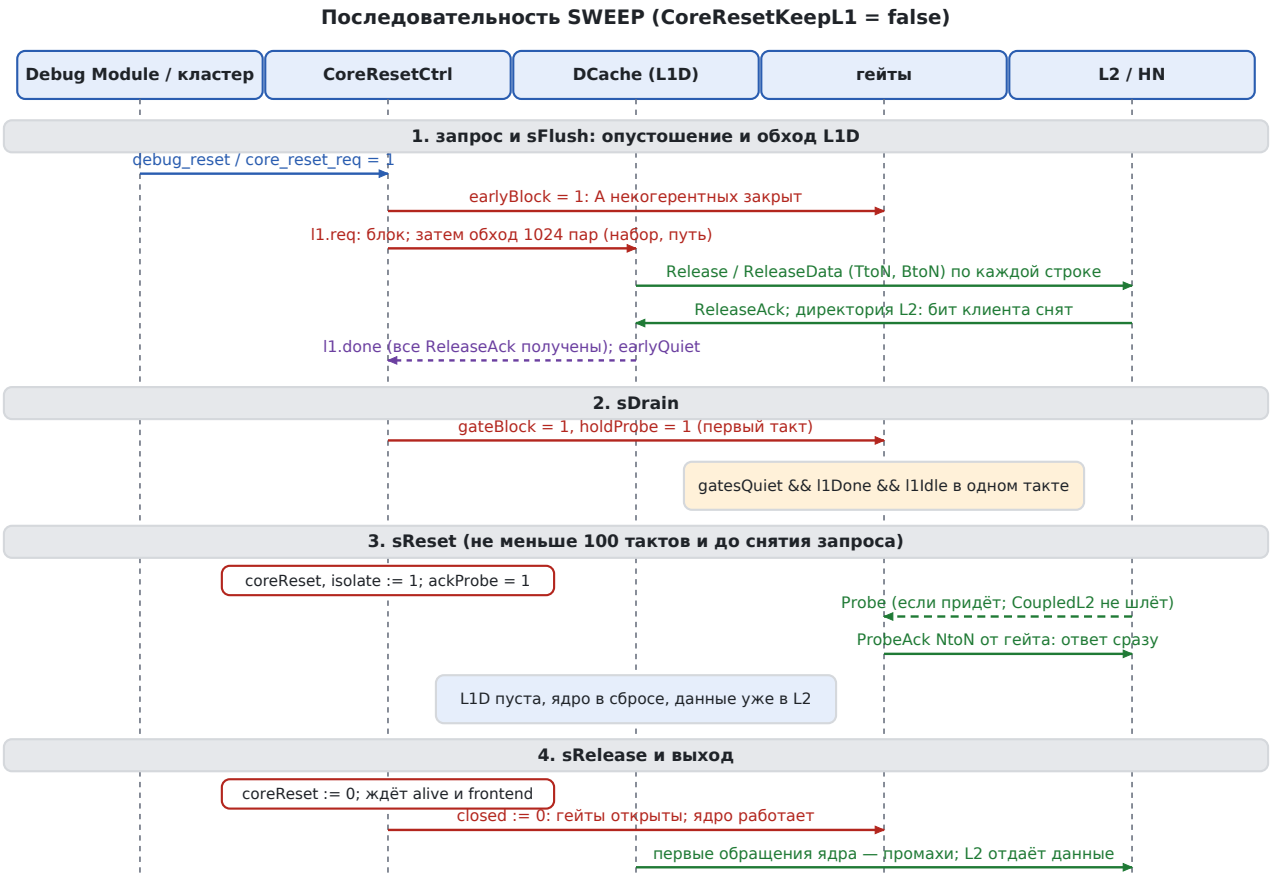


Рис. 7. Последовательность SWEEP: обход отдаёт строки L2 (Release / ReleaseData), в сбросе на Probe отвечает гейт.

4.1. Фаза 1: запрос

- Источник — `debug_reset` (DM: ndmreset или hartreset этого харта), `core_reset_req` (кластер) или `hartResetReq` в XSTileWrap (тот же hartreset в upstream-подключении). Каждый вход тайла проходит 3-триггерный синхронизатор в такт тайла (`AsyncResetSynchronizerShiftReg`); запрос контроллера — ИЛИ синхронизированных входов.
- В XSNoCTop любой запрос и `core_reset_busy` будят отключённый в WFI такт и не дают отключить его снова.
- `CoreResetCtrl` переходит sIdle → sFlush, поднимает `busy`, `l1.req` и `earlyBlock`. Пока `busy` = 1, запросы L2-предвыборщика к TLB ядра (`l2_tlb_req`) закрыты.

Почему так: синхронизаторы в XSTile, пробуждение такта, закрытие l2_tlb_req сразу

Синхронизаторы. `ndmreset` и `hartreset` формирует Debug Module в своём такте, поэтому для тайла это асинхронный уровень; без синхронизации фронт может попасть в окно setup / hold. Синхронизатор стоит в самом XSTile, чтобы порт был безопасен при любом способе подключения тайла.

Пробуждение такта. При `WFIClockGate` такт ядра и L2 отключается, пока ядро в WFI. Опустошение, счётчики гейтов, контроллер, ответы на Probe (DCache или гейт) работают от этого такта: без него последовательность встанет до выхода ядра из WFI, то есть, возможно, навсегда.

l2_tlb_req. Предвыборщик L2 переводит адреса через TLB ядра. Этот TLB неблокирующий: ответ приходит через фиксированное число тактов. Если перестать подавать запросы с первого такта последовательности, к моменту сброса ни одного ответа в пути не будет; BOP запрос без ответа просто повторит позже.

4.2. Фаза 2: sFlush — опустошение DCache и раннее закрытие A

`DCacheFlushUnit` переходит sIdle → sDrain и поднимает `block`. С этого такта:

- записи из sbuffer и атомики не входят в MainPipe, CMO не принимаются;

- новые промахи не попадают в MissQueue: загрузки переигрываются, запись, успевшая войти в MainPipe и промахнуться, возвращается в sbuffer, запросы предвыборки L1 отбрасываются;
- снято WFI-удержание промахов (`wfiReq` для MissQueue маскируется);
- загрузки-попадания продолжают читать DCache; Probe от L2 обслуживаются всё время (канал B открыт);
- в то же время гейты некогерентных агентов закрыли A (`earlyBlock`): начатые запросы получают ответы, новые ждут перед гейтом.

Автомат ждёт `idle` — четыре флага пустоты (1.5.3): MissQueue, WritebackQueue, MainPipe, ProbeQueue. С KEEP затем sDone: `l1.done` = 1. В режиме SWEEP — обход:

- `sSweep`: каждый такт, когда MainPipe принимает, автомат выдаёт запрос flush на пару (набор, путь) — сначала все наборы пути 0, затем пути 1 и т. д., всего 1024 запроса. Соседние запросы — в разные наборы, поэтому проверка конфликта набора MainPipe их не задерживает. У flush самый низкий приоритет: Probe от L2 обслуживаются раньше.
- MainPipe читает тег и состояние пути. Строка в N завершает запрос сразу. Действительная строка уходит в WritebackQueue как добровольный Release (чистая) или ReleaseData (грязная) с параметром TtoN / BtoN, а в `metaArray` в том же такте пишется N.
- `sWait`: ни одного запроса flush в MainPipe (`inflight === 0`) и `idle` — каждый Release получил ReleaseAck. Затем sDone: `l1.done` = 1. L1D пуста, директория L2 не числит за ней ничего.

Блокировка остаётся, пока `l1.req` = 1. Контроллер уходит из sFlush при `l1Done && earlyQuiet` (1.5.1).

Почему так: зачем опустошать DCache, если её всё равно сбросят

Обход отдаёт строки, но сбрасывается и управляющая логика DCache — всё, что она делала, теряется. Опасные случаи для L2: (а) запись MissQueue получила GrantData, но ещё не отправила GrantAck — MSHR L2 ждёт GrantAck вечно [TL 1.8.1 5.1, с. 29], [TL 1.8.1 9.3.9, с. 79]; (б) WritebackQueue начала ReleaseData и успела отправить один beat — SinkC ждёт второй beat вечно ([TL 1.8.1 5, с. 28]); или Release отправлен целиком, а ReleaseAck придёт в сброшенное ядро, которое этого Release уже не помнит ([TL 1.8.1 6.4, с. 43]); (в) вытеснение, запущенное промахом, отправит Release в такте сброса — гейт этого Release ещё не видел, когда разрешал сброс. Пустые очереди при заблокированных промахах и записях означают, что новый Release начаться не может, а все начатые завершены.

Опустошение перед обходом нужно и ему самому: промах, установивший строку после того, как обход прошёл её набор, оставил бы в L1D строку, о которой знает L2. С KEEP сохраняются только массивы: вытеснение уже записало в meta состояние N, а Release ещё лежит в WritebackQueue — после сброса L1 строку не держит, а L2 по-прежнему считает её TRUNK; запись из sbuffer в s3 пишет данные и состояние Dirty в одном такте — сброс посреди даст несогласованную строку, которая переживёт сброс.

Почему так: обход ждёт последнего ReleaseAck; Probe во время обхода безопасны

Release — запрос, ReleaseAck — ответ на него [TL 1.9.3 9.3.12, с. 82]. Пока ответа нет, запись WBQ занята, и сбросить ядро нельзя: ReleaseAck пришёл бы в сброшенное ядро, а source ID оставался бы «занятым» [TL 1.8.1 6.4, с. 43]. До ReleaseAck DCache и не ответит на Probe к этой строке [TL 1.9.3 9.2, с. 70].

Probe идёт в MainPipe раньше flush (более высокий приоритет) и по той же строке не встречается с ним в конвейере: MainPipe не пускает два запроса к одному набору в соседние стадии. Если Probe пришёл, когда Release строки уже в WBQ, он ждёт ReleaseAck и отвечает NtoN — обычная гонка Release и Probe [TL 1.9.3 9.2, с. 71]. Если строку раньше забрал Probe (toN), обход видит N и проходит мимо. В любом порядке L2 получает данные ровно один раз.

Зачем WritebackQueue, если Release считает гейт: гейт видит только то, что пересекло границу; Release, который сформирован и лежит в WBQ, но ещё не отправлен (канал C занят), он не видит. Пустая WBQ значит, что все Release отправлены и подтверждены, а все ProbeAck отправлены.

4.3. Фаза 3: sDrain — опустошение границы ядро / L2

Контроллер переходит sFlush → sDrain: `gateBlock` закрывает канал A гейта l1d (на границе сообщения), а `holdProbe` закрывает B в том же такте. Некогерентные гейты уже закрыты и тихи (`earlyQuiet`), DCache сообщила `l1.done`. Контроллер ждёт одного: `gatesQuiet && l1Done && l1Idle` в такте решения — в нём же переход в sReset. Обычно это первый такт sDrain. Открытым может остаться только Probe, принятый l1d-гейтом в последнем такте sFlush: его ProbeAck идёт через буфер `l1d_to_l2_buffer`, `probes` гейта ≠ 0 держит `quiet` низким, пока ответ не пройдёт.

Почему так: ждать ответов на Get и MMIO, проверка в такте перехода, Probe в ожидании не считается

Ответ, пришедший после выхода из сброса, ядро примет за ответ на новый запрос с тем же source ID и получит чужие данные или команды: идентификатор занят, пока на запрос не ответили [TL 1.8.1 6.4, с. 43]. MMIO-мост в L2 может ждать второй beat записи. Поэтому все запросы, пересёкшие границу, должны завершиться до сброса: «no unanswered TileLink requests when one half of the crossing is reset» [TL 1.8.1 3.2.3, с. 14].

Решение по `quiet` текущего такта: если переходить в sReset по значению прошлого такта, Probe, принятый l1d-гейтом в последнем такте sFlush, проскакивает — следующим тактом ядро в сбросе, ProbeAck потерян, MSHR L2 ждёт его вечно (мутант `STALE_QUIET`, раздел 7.2).

Probe, предложенный L2, но не принятый, в `quiet` не входит: он закрыт `holdProbe` с первого такта sDrain и ждёт в L2 (KEEP) либо получит ответ гейта (SWEEP). Гейты и их счётчики — в домене тайла и не сбрасываются вместе с ядром.

4.4. Фаза 4: sReset — сброс XSCore

- Контроллер переходит в sReset и поднимает `coreReset`: асинхронно сбрасывается всё внутри XSCore. С KEEP — кроме `metaArray`, `errorArray`, `tagArray` DCache; в режиме SWEEP — и они (L1D уже пуста). Данные в SRAM не меняются. `hartInReset` = 1.
- На том же фронте поднимается `isolate` (регистр `closed`): гейты и `TLResetIsolation` больше не смотрят на valid и ready ядра, XSTile подставляет вместо выходов ядра значения сброса или удержанные значения (1.6).
- Канал B к ядру закрыт: с KEEP `holdProbe` = 1 (гейты держат Probe в L2) и `l1.freeze` = 1 (запись в массивы L1D замаскирована); в режиме SWEEP `ackProbe` = 1 — гейт l1d сам принимает Probe и отвечает ProbeAck NtoN.
- **Длина:** не меньше `rstCycles` = 100 тактов и до снятия запроса: выход из sReset — при `cnt >= rstCycles && !req`. Это правило TileLink [TL 1.8.1 3.2.2] (100 тактов с valid = 0).
- L2 работает. Снуп HN к строке, которую директория числит за L1D (только KEEP): Probe ждёт в гейте. Снупы к остальным строкам — в режиме SWEEP ко всем — L2 обслуживает сама.
- В симуляции проверяется: через гейты в ядро и из ядра ничего не проходит (assert `gatesQuiet` в контроллере), а изоляция покрывает сброс (assert `!coreReset || closed`).

Почему так: KEEP — держать Probe, а не отвечать; SWEEP — ответ гейта; зачем freeze

KEEP. Отвечать некому: ProbeQueue, MainPipe и WritebackQueue в сбросе, а ответ может требовать данных, которые есть только в L1D. Если пустить Probe в ProbeQueue, которая сбрасывается, он пропадёт, и L2 будет ждать ProbeAck вечно. Пока `ready` = 0, Probe не принят [TL 1.8.1 4.1, с. 18] и ждёт в L2 (SourceB); после сброса он уходит в ядро — то есть принят за конечное время, как требует правило 3.i [TL 1.8.1 5, с. 28], если запрос снимут.

SWEEP. К сбросу L1D уже пуста: обход отдал все строки, директория L2 её не числит. Любой Probe, пришедший в sReset или sRelease, относится к строке, которой у L1D нет, поэтому честный ответ клиента — ProbeAck NtoN: Probe к клиенту без прав законен и прав не меняет [TL 1.8.1 9.3.3, с. 74], ответ — Report NtoN [TL 1.9.3 9.3.5, с. 78]. Данные L2 уже получила при обходе, поэтому её копия актуальна (1.3.3, 1.3.4).

freeze (KEEP). Записи в `metaArray`, `errorArray` и SRAM тегов и данных приходят из s3 MainPipe, а MainPipe в сбросе. Опустошение гарантирует, что в момент сброса записи нет. Но асинхронная установка сброса — момент, когда выходы регистров домена ядра переключаются не по такту (1.6), а разрешение записи SRAM у `BankedDataArray` выходит из регистра этого домена. Маска `freeze` — тот же регистр `closed`, что и `isolate`, — гарантирует, что ни такты установки, ни такты снятия сброса ничего не запишут в массивы, которые должны пережить сброс.

4.5. Фаза 5: sRelease — выход из сброса

- `coreReset` = 0 при `cnt >= rstCycles && !req`; ResetGen снимает сброс ядра синхронно, по цепочке MemBlock → Backend → Frontend.
- Контроллер в sRelease ждёт `l1.alive` и выход frontend из сброса (`!coreInReset`, где `coreInReset` = `resetInFrontend` — последний в цепочке ResetGen). `alive` — регистр в `DCacheFlushUnit`, равный 0, пока DCache в сбросе, и 1 с первого такта после него. Каналы A и B к ядру закрыты всё это время, `isolate` = 1. Оба сигнала контроллер смотрит только в sRelease: снятие сброса синхронное, и в этом состоянии выходы ядра меняются только по фронту.
- Затем `closed := 0`, `hartInReset := 0`, sIdle: гейты открыты, `core_reset_busy` = 0.

Почему так: зачем ждать alive и frontend

Сброс внутри XSCore расходится по цепочкам ResetGen, и DCache выходит из сброса через неизвестное заранее число тактов после `coreReset = 0`. Если открыть канал B раньше, ожидавший Probe (или, в режиме SWEEP, новый Probe) попадёт в ProbeQueue, которая ещё в сбросе, и пропадёт. `alive` — прямое свидетельство, что DCache работает. Frontend выходит последним: пока он в сбросе, ядро ещё не готово; кроме того, ядро, открытое до выхода frontend, могло бы послать первую выборку в закрытый гейт. Мутант `NO_FE_WAIT` ловится свойством P20 и симуляцией.

4.6. Фаза 6: ядро работает

- sIdle: гейты открыты, `busy` = 0, `hartIsInReset` = 0.
- Ядро выбирает команды с текущего reset vector. С KEEP первые загрузки к сохранённым строкам — попадания в L1D, Probe, ждавший в L2, получает ответ из «горячей» L1D. В режиме SWEEP — промахи; L2 отдаёт данные, которые получила при обходе.
- Если отладчик просил остановку на сбросе (`resethaltreq` или `haltreq`), прерывание отладки ждёт ядро, и харт останавливается на первой команде.
- Если запрос снова поднят в sRelease, последовательность начинается заново из sIdle (один сброс на запрос).

4.7. Сводка: чего ждём и что сломается без этого

Чего ждём / что делаем	Где	Что сломается без этого
флаги пустоты DCache (MissQueue, WBQ, MainPipe, ProbeQueue)	DCacheFlushUnit, sDrain; `l1.idle` в контроллере	MSHR L2 ждёт GrantAck; ReleaseData оборвана; (KEEP) строка наполовину установлена, неотправленный Release пропал с данными
обход всех (набор, путь), Release с данными грязных строк (SWEEP)	DCacheFlushUnit, MainPipe, sSweep	грязные данные L1D потеряны; после AcquirePerm L2 отдаёт данные другого адреса
последний ReleaseAck (SWEEP)	DCacheFlushUnit, sWait	ReleaseAck в сброшенное ядро; Release, ещё лежащий в WBQ, пропал вместе с данными
A некогерентных закрыт с sFlush; `earlyQuiet` при выходе	гейты ранние, CoreResetCtrl, sFlush	запросы продолжают приходить (длиннее опустошение); без тишины B закроется раньше, чем L2 дождётся Probe для Get — Get без ответа
B закрыт с первого такта sDrain	гейт l1d, `holdProbe`	Probe, принятый после решения, теряется сбросом; MSHR L2 ждёт вечно
`gatesQuiet && l1Done && l1Idle` текущего такта	CoreResetCtrl, sDrain	MSHR L2 ждёт GrantAck / ProbeAck вечно; Probe последнего такта sFlush потерян; старый ответ примется за новый
isolate с того же фронта, что и сброс	CoreResetCtrl `closed`, гейты, TLResetIsolation, XSTile	выход ядра, меняющийся не по такту, защёлкнут в тайле: метастабильность, ложный valid, лишний beat
канал B закрыт: держится (KEEP) или ответ гейта (SWEEP)	гейт, sReset / sRelease	Probe в сбрасываемую ProbeQueue пропал, L2 ждёт вечно
запись в массивы замаскирована (KEEP)	DCacheWrapper, `freeze`	meta, тег или данные испорчены в такт установки или снятия сброса
теги на сбросе тайла (KEEP)	DCacheWrapper, `tagArray.reset`	теги стёрты при сбросе ядра, meta сохранена — порча данных
сброс не короче 100 тактов	CoreResetCtrl, sReset	нарушение правила TileLink 3.2.2 для связей внутри ядра
сброс до снятия запроса	CoreResetCtrl, sReset	ядро выходит из сброса, пока его держат
`alive` и выход frontend	CoreResetCtrl, sRelease	Probe ушёл в DCache, ещё не вышедшую из сброса; ядро открыто, пока frontend в сбросе
запись массивов: нулевой запас между решением и `coreReset`	CoreResetCtrl, DCacheFlushUnit	запись s3 предыдущего такта попала бы под сброс

5. Изменения в DCache

Файлы в каталоге `src/main/scala/xiangshan/cache/dcache/`. Все фрагменты взяты из патча как есть (`git diff 7b16188 HEAD -- src/`); зелёным — добавленные строки, красным — удалённые, серым — контекст; в левой колонке — номер строки в новом файле. Автомат генерируется всегда, но неактивен, пока `quiesce.req = 0`; без `EnableCoreReset` XSTile подаёт на него константу 0, и синтез убирает автомат. Обход генерируется только в режиме SWEEP: иначе вход flush MainPipe получает константу 0.

5.1. mainpipe/FlushUnit.scala — интерфейс, автомат и обход (новый файл)

Интерфейс контроллер ↔ DCache и автомат. Заголовочный комментарий файла описывает флаги пустоты и последовательность; ниже — код.

L1QuiesceIO: пять сигналов

```
 48 + class L1QuiesceIO extends Bundle {
 49 +   // level: drain L1D (and sweep it, sweep implementation) and keep it blocked (no store, atomic, CMO, miss)
 50 +   val req    = Output(Bool())
 51 +   // the core reset is asserted or being released: no write into the arrays that survive it
 52 +   // (CoreResetKeepL1 only, 0 otherwise)
 53 +   val freeze = Output(Bool())
 54 +   // L1D has no miss, no release and no store in flight (and is empty with the sweep); high while req is high
 55 +   val done   = Input(Bool())
 56 +   // the control logic of L1D is idle at this moment: the emptiness flags of all its queues and of MainPipe
 57 +   // (with done high only a Probe can make it busy again)
 58 +   val idle   = Input(Bool())
 59 +   // the DCache control logic is out of reset (0 while it is held in reset)
 60 +   val alive  = Input(Bool())
```

- `req` — уровень от контроллера: опустошать (и обойти) L1D и держать её заблокированной; `freeze` — сброс ядра установлен или снимается: массивы, переживающие сброс, не писать (только KEEP, иначе 0); `done` — нет промахов, Release и записей в полёте (и L1D пуста при обходе); `idle` — флаги пустоты всех очередей и MainPipe сейчас; `alive` — управляющая логика DCache вышла из сброса.

DCacheFlushUnit: флаги пустоты, автомат, обход

```
 77 +   val sIdle :: sDrain :: sSweep :: sWait :: sDone :: Nil = Enum(5)
 78 +   val state = RegInit(sIdle)
 79 +
 80 +   val set = RegInit(0.U(idxBits.W))
 81 +   val way = RegInit(0.U(log2Up(nWays).W))
 82 +   // flush requests between MainPipe s0 fire and s3 fire (pipeline depth is 3)
 83 +   val inflight = RegInit(0.U(3.W))
 84 +
 85 +   inflight := inflight + io.pipe_req.fire.asUInt - io.pipe_resp.asUInt
 86 +
 87 +   // the control logic of the DCache holds nothing (see the emptiness flags above)
 88 +   val idle = io.mq_empty && io.wbq_empty && io.mp_empty && io.pq_empty
 89 +
 90 +   switch (state) {
 91 +     is (sIdle) {
 92 +       when (io.flush.req) { state := sDrain }
 93 +     }
 94 +     is (sDrain) {
 95 +       // a store or atomic accepted just before the block is still in MainPipe (mp_empty);
 96 +       // a miss or a Release keeps its queue busy until its Grant / ReleaseAck (mq_empty, wbq_empty).
 97 +       // idle does not cover the one registered write stage of the data and meta arrays (BankedDataArray,
 98 +       // AsynchronousMetaArray): the write of an s3 fire in the previous cycle is committed at the edge that
 99 +       // ends the cycle of the decision, the same edge at which coreReset rises (CoreResetCtrl), before the
100 +       // asynchronous reset assertion. No delay may be added between the decision and coreReset.
101 +       when (!io.flush.req) {
102 +         state := sIdle
103 +       }.elsewhen (idle) {
104 +         state := (if (sweep) sSweep else sDone)
105 +         set := 0.U
106 +         way := 0.U
107 +       }
108 +     }
109 +     is (sSweep) {
110 +       // CoreResetCtrl keeps quiesce.req high until l1.done, so the sweep always runs to its end
111 +       when (io.pipe_req.fire) {
112 +         set := set + 1.U
113 +         when (set === (nSets - 1).U) {
114 +           set := 0.U
115 +           way := way + 1.U
116 +           when (way === (nWays - 1).U) { state := sWait }
117 +         }
```

```
118 +       }
119 +     }
120 +     is (sWait) {
121 +       // the last flush requests leave MainPipe, the last Releases of the sweep get their ReleaseAck
122 +       when (inflight === 0.U && idle) { state := sDone }
123 +     }
124 +     is (sDone) {
```

- `idle` = `mq_empty && wbq_empty && mp_empty && pq_empty`: точное состояние всей управляющей логики. В sDrain автомат ждёт `idle` (запись, принятая перед блокировкой, ещё в MainPipe — `mp_empty`; промах или Release держат свои очереди до Grant / ReleaseAck). Счётчика тихих тактов нет.
- В sSweep на каждый принятый MainPipe запрос flush счётчики (`set`, `way`) идут way-major: соседние запросы — в разные наборы. sWait ждёт `inflight === 0 && idle`: последний запрос flush покинул MainPipe, каждый Release получил ReleaseAck.
- `alive` — регистр, который сбрасывается вместе с DCache: 0 в сбросе, 1 с первого такта после. Контроллер ждёт его в sRelease.
- В KEEP состояния sSweep и sWait недостижимы (assert); запросы flush не выдаются (`io.pipe_req.valid` — константа 0).

Запрос flush: поля MainPipeReq

```
143 +   val r = io.pipe_req.bits
144 +   r := DontCare
145 +   r.miss := false.B
146 +   r.miss_id := 0.U
147 +   r.miss_param := 0.U
148 +   r.miss_dirty := false.B
149 +   r.occupy_way := 0.U
150 +   r.miss_fail_cause_evict_btot := false.B
151 +   r.probe := false.B
152 +   r.probe_param := 0.U
153 +   r.probe_need_data := false.B
154 +   r.source := LOAD_SOURCE.U          // neither a store nor an AMO
155 +   r.cmd := M_XRD
156 +   r.vaddr := Cat(set, 0.U(blockOffBits.W))   // set index (with the alias bits of the VIPT cache), offset 0
157 +   r.addr := 0.U                      // not used: MainPipe builds the line address from the tag it reads in s1
158 +   r.store_data := 0.U
159 +   r.store_mask := 0.U
160 +   r.word_idx := 0.U
161 +   r.amo_data := 0.U
162 +   r.amo_mask := 0.U
163 +   r.amo_cmp := 0.U
164 +   r.error := false.B
165 +   r.replace := false.B
166 +   r.replace_way_en := UIntToOH(way, nWays)
167 +   r.flush := true.B
```

Запрос flush помечен `r.flush`, адрес набора идёт через `vaddr`, путь — one-hot в `replace_way_en`; адрес строки MainPipe берёт сам из тега, который читает на s1. Остальные поля — нейтральные (не store, не AMO, не Probe, не промах).

DCacheFlushUnit: 5 состояний; idle = mq_empty && wbq_empty && mp_empty && pq_empty



5.2. mainpipe/MissQueue.scala — признак «MissQueue пуста»

MissEntry: запись занята

```
469       val wfi = Flipped(new WfiReqBundle)
470 +     // this entry holds a miss (used by DCacheFlushUnit to wait until all misses are done)
471 +     val busy = Output(Bool())
```

```
659 +   io.busy := req_valid
```

MissQueue.empty: порт и выражение

```
1056       val wfi = Flipped(new WfiReqBundle)
1057 +     // no miss entry in use, nothing in the enqueue pipeline register and no CMO in progress (DCacheFlushUnit)
1058 +     val empty = Output(Bool())
```

```
1245 +   io.empty := !Cat(entries.map(_.io.busy)).orR && !miss_req_pipe_reg.reg_valid() && cmo_unit.io.req.ready
```

`empty` выражает три места, где может находиться промах: запись (`busy` = `req_valid`), регистр приёма `miss_req_pipe_reg` (запрос, разрешённый в этом такте и ещё не занявший запись, — раньше это окно закрывали 16 тактами) и CMO-блок (`cmo_unit.io.req.ready`: ready = нет CMO в процессе).

5.3. mainpipe/WritebackQueue.scala — признак «WBQ пуста»

```
 324       val block_miss_req = Vec(LoadPipelineWidth + 2, Output(Bool()))
 325 +     // no Release / ProbeAck in progress: every voluntary Release got its ReleaseAck (DCacheFlushUnit)
 326 +     val empty = Output(Bool())
```

```
 390 +   io.empty := !Cat(entries.map(_.io.block_addr.valid)).orR
```

Запись WBQ занята (`state =/= s_invalid`) от приёма Release / ProbeAck до его завершения: Release — до ReleaseAck, ProbeAck — до отправки последнего beat. Пуста — значит все Release подтверждены, а все ProbeAck ушли.

5.4. mainpipe/Probe.scala — признак «ProbeQueue пуста» (новый изменённый файл)

```
 134       val update_resv_set = Input(Bool())
 135 +     // no Probe in an entry or in the register in front of MainPipe (DCacheFlushUnit, core-only reset)
 136 +     val empty = Output(Bool())
```

```
 207     when(pipe_req_arb.io.out.fire){
 208       selected_req_valid := true.B
 209     }
 210 +   io.empty := !Cat(entries.map(_.io.block_addr.valid)).orR && !selected_req_valid
```

Probe находится либо в одной из записей (`block_addr.valid`), либо в регистре выбора `selected_req_valid` перед MainPipe, куда его кладёт арбитр. Без этого флага Probe, ещё не вошедший в MainPipe, остался бы невидимым для `mp_empty` и для `done`.

5.5. mainpipe/MainPipe.scala — запрос flush (обход) и признак busy

Обход идёт тем же путём, что вытеснение при промахе: MainPipe читает тег и состояние указанного пути, и если строка действительна, отправляет её в WritebackQueue и в том же такте пишет в `metaArray` состояние N.

Поле запроса

```
  70     val replace_way_en = UInt(DCacheWays.W)
  71 +   // flush: voluntarily release the line in (set of vaddr, way replace_way_en) and invalidate it.
  72 +   // Issued only by DCacheFlushUnit before a reset of XSCore (core + L1) alone.
```

Порты flush_req, flush_resp, busy

```
 143 +     // L1 flush-all sweep (DCacheFlushUnit): one request per (set, way); flush_resp pulses when
 144 +     // a flush request leaves s3 (its Release, if any, has been handed to the writeback queue)
 145 +     val flush_req = Flipped(DecoupledIO(new MainPipeReq))
 146 +     val flush_resp = Output(Bool())
 147 +     // a request (of any kind) is in s1, s2 or s3: DCacheFlushUnit waits until MainPipe is empty
 148 +     val busy = Output(Bool())
 149       // find matched refill data in missentry
```

Арбитр: flush — самый низкий приоритет

```
 261 +   // only the flush port may carry a flush request (other producers leave the field DontCare)
 262 +   def withFlush(in: DecoupledIO[MainPipeReq], isFlush: Boolean): DecoupledIO[MainPipeReq] = {
 263 +     val w = Wire(DecoupledIO(new MainPipeReq))
 264 +     w.valid := in.valid
 265 +     w.bits := in.bits
 266 +     w.bits.flush := isFlush.B
 267 +     in.ready := w.ready
 268 +     w
 269 +   }
 270 +   val probe_req_in  = withFlush(io.probe_req, false)
 271 +   val refill_req_in = withFlush(io.refill_req, false)
 272 +   val atomic_req_in = withFlush(io.atomic_req, false)
 273 +   val flush_req_in  = withFlush(io.flush_req, true)
 274 +
 275     // s0: read meta and tag
 276     val req = Wire(DecoupledIO(new MainPipeReq))
 277     arbiter(
```

```
278       in = Seq(
    -       io.probe_req,
    -       io.refill_req,
279 +       probe_req_in,
280 +       refill_req_in,
281         store_req, // Note: store_req.ready is now manually assigned for better timing
    -       io.atomic_req,
282 +       atomic_req_in,
283 +       flush_req_in // lowest priority: the sweep only runs while stores, atomics and misses are blocked
```

- У четырёх других источников (Probe, refill, store, atomic) поле `flush` принудительно 0 (`withFlush`), чтобы мусорное значение DontCare не превратило их в flush.

s0: flush всегда читает всю строку

```
315 +   // dirtiness is only known in s1, so a flush always reads the whole line
316 +   val flush_need_data = s0_req.flush
317
```

s1: тег и состояние пути берутся из пути запроса

```
422 +   // flush: the way is given by the request, tag and coherence state are taken from that way
423 +   val s1_flush = s1_req.flush
424 +   val s1_flush_tag = ParallelMux(s1_req.replace_way_en.asBools, (0 until nWays).map(w => tag_resp(w)))
425 +   val s1_flush_coh = ParallelMux(s1_req.replace_way_en.asBools, (0 until nWays).map(w => meta_resp(w))).asTypeOf
        ↪ (new ClientMetadata)
426 +
427 +   val s1_way_en = Mux(
428 +     s1_flush,
429 +     s1_req.replace_way_en,
430 +     Mux(io.pseudo_error.valid || s1_need_replacement, s1_repl_way_en, s1_tag_ecc_match_way)
431 +   )
432     assert(!RegNext(s1_fire && PopCount(s1_way_en) > 1.U))
433
    -   val s1_tag = s1_hit_tag
    -   val s1_coh = s1_hit_coh
434 +   val s1_tag = Mux(s1_flush, s1_flush_tag, s1_hit_tag)
```

s2: flush идёт в s3, как replace

```
    -   val s2_can_go_to_s3 = (s2_sc || s2_req.replace || s2_req.probe ||
```

s3: инвалидация строки

```
605 +   // flush of a valid line: invalidate it (new_coh falls through to ClientMetadata.onReset = Nothing)
606 +   val flush_update_meta = s3_req.flush && s3_coh.state =/= ClientStates.Nothing
607 +   val update_meta = (miss_update_meta || probe_update_meta || store_update_meta || amo_update_meta || flush_upda
        ↪ te_meta) &&
```

s3: Release / ReleaseData строки

```
772 +   val flush_wb = s3_req.flush && s3_coh.state =/= ClientStates.Nothing
```

s3: когда flush уходит из s3

```
794 +   // flush: an invalid way finishes at once; a valid one needs the writeback queue and the meta write port
795 +   val s3_flush_can_go = s3_req.flush && (!flush_wb || io.wb.ready && io.meta_write.ready)
```

flush_resp и busy

```
951 +   io.flush_resp := s3_fire && s3_req.flush
952 +   io.busy := s1_valid || s2_valid || s3_valid
```

Запись в WritebackQueue

```
1033 +     // flush: the line is invalidated in the same cycle (meta write)
1034 +     s3_req.flush && io.meta_write.ready ||
```

- Недействительный путь (`state === Nothing`) завершается в s3 сразу: Release не создаётся, `meta_write` не нужен.
- Действительный путь: `flush_wb` включает запись в WBQ, `voluntary` = 1 (Release, а не ProbeAck), `hasData` — для грязной строки (в `alwaysReleaseData` — для любой действительной); параметр `miss_shrink_param` — TtoN / BtoN из состояния строки. Одновременно `meta_write` пишет N (`new_coh` проваливается в `ClientMetadata.onReset`).
- `flush_resp` — импульс на выходе s3 запроса flush: по нему `inflight` во FlushUnit уменьшается.
- `busy` = s1_valid || s2_valid || s3_valid: флаг пустоты MainPipe `mp_empty` — отрицание этого выхода. Он нужен не обходу, а любой причине, по которой в конвейере может оказаться запрос (запись, принятая перед блокировкой, Probe).
- Тот же проход по MainPipe используется во всех режимах как защита: запись в массивы данных и meta из s3 после блокировки исключена флагами пустоты, остальное — assert-ы в DCacheWrapper.

5.6. DCacheWrapper.scala — подключение, флаги, теги на сбросе тайла, блокировки

Порты DCacheIO: quiesce и metaReset

```
 855 +   // drain before a reset of XSCore alone (see DCacheFlushUnit)
 856 +   val quiesce = Flipped(new L1QuiesceIO)
 857 +   // CoreResetKeepL1: reset of the arrays that survive a reset of XSCore alone: coherence meta, error flags
 858 +   // and the tag array with its initialisation counter (tile reset)
 859 +   val metaReset = Option.when(p(SoCParamsKey).EnableCoreReset && p(SoCParamsKey).CoreResetKeepL1)(Input(AsyncRes
         ↪ et()))
 860   }
```

Экземпляр DCacheFlushUnit (sweep = EnableCoreReset && !CoreResetKeepL1)

```
1055 +   // CoreResetKeepL1 = false: before the core reset the flush unit releases every line (sweep)
1056 +   val flushUnit    = Module(new DCacheFlushUnit(sweep = p(SoCParamsKey).EnableCoreReset && !p(SoCParamsKey).Core
         ↪ ResetKeepL1))
1057 +   // while the flush unit is active: no store, atomic, CMO or new miss enters the DCache
1058 +   val flushBlock   = flushUnit.io.block
1059
```

Маска WFI, флаги пустоты, обход через MainPipe

```
1072 +   // A WFI request holds unsent Acquires in the miss queue; the flush must be able to drain it,
1073 +   // so the WFI request is masked while the flush unit is active.
1074 +   missQueue.io.wfi.wfiReq := io.wfi.wfiReq && !flushBlock
1075 +   io.wfi.wfiSafe := missQueue.io.wfi.wfiSafe
1076 +
1077 +   // L1 quiesce before a core-only reset
1078 +   flushUnit.io.flush <> io.quiesce
1079 +   flushUnit.io.mq_empty := missQueue.io.empty
1080 +   flushUnit.io.wbq_empty := wb.io.empty
1081 +   flushUnit.io.mp_empty := !mainPipe.io.busy
1082 +   flushUnit.io.pq_empty := probeQueue.io.empty
1083 +   // sweep: flush requests through MainPipe (lowest priority, see MainPipe.flush_req)
1084 +   mainPipe.io.flush_req <> flushUnit.io.pipe_req
1085 +   flushUnit.io.pipe_resp := mainPipe.io.flush_resp
```

- `mp_empty` = `!mainPipe.io.busy`, `pq_empty` = `probeQueue.io.empty`, `mq_empty` = `missQueue.io.empty`, `wbq_empty` = `wb.io.empty`. Это единственные связи флагов; больше в контроллере ничего не ждёт «16 тактов».
- WFI-удержание промахов снимается, пока автомат активен: иначе промах, придержанный в MissQueue по `wfiReq`, никогда бы не ушёл и очередь не опустела бы; ROB при этом остаётся в WFI.

KEEP: массивы на сбросе тайла — теги, meta, error

```
1180 +   // Reset of XSCore alone that keeps L1D (EnableCoreReset, CoreResetKeepL1): the data SRAMs have no reset at
1181 +   // all (shouldReset = false, no initialisation), the tag array is initialised only by the reset of its own
1182 +   // domain, and the coherence state of every line (metaArray) and its error flags (errorArray) are flops.
1183 +   // They take the tile reset instead of the core reset, so L1D still holds its lines after a core
1184 +   // reset. While the core reset is asserted or released (freeze) nothing may be written into them.
1185 +   // Without CoreResetKeepL1 they stay on the core reset (as upstream): the flush unit has released every line
1186 +   // before (sweep), L1D starts empty after a core reset.
1187 +   // The tag array is a SRAM, but its wrapper (TagSRAMBank) has a reset counter (rst_cnt) that writes zeros into
1188 +   // every set after reset, so it takes the tile reset as well (otherwise every core reset would clear the tags)
         ↪ .
1189 +   io.metaReset.foreach { r =>
1190 +     metaArray.reset := r
1191 +     errorArray.reset := r
1192 +     tagArray.reset := r
```

`io.metaReset` — асинхронный сброс тайла (порт `l1_meta_reset`, формируется в XSTile тем же `ResetGen`, что и сброс ядра, но без `coreReset`). Три строки присваивания — самое важное: `metaArray`, `errorArray` и (исправление, найденное ревью) `tagArray` переводятся с сброса ядра на сброс тайла. Без `tagArray.reset := r` счётчик `rst_cnt` обёртки `TagSRAMBank` стирал бы теги при каждом сбросе ядра.

freeze: запись в meta

```
1193 +     when (io.quiesce.freeze) {
1194 +       metaArray.io.write.foreach(_.valid := false.B)
1195 +     }
1196 +   }
```

freeze: запись в теги и данные

```
1369 +   // is on the tile reset, see metaReset above). While the core reset is asserted
1370 +   // or released (freeze, a register of the tile reset domain) no write enable reaches them,
1371 +   // whatever the control logic of the core reset domain drives at that time (isolation of the write enables;
1372 +   // nothing should write then anyway, see the assertion below).
1373 +   io.metaReset.foreach { _ =>
```

```
1374 +     when (io.quiesce.freeze) {
1375 +       tagArray.io.write.valid := false.B
1376 +       bankedDataArray.io.write.valid := false.B
1377 +       bankedDataArray.io.write_dup.foreach(_.valid := false.B)
```

Блокировки: новые промахи и CMO

```
1593 +   // flush: no new miss (load/store/prefetch/AMO) and no CMO is accepted; requesters replay
1594 +   when (flushBlock) {
1595 +     missQueue.io.req.valid := false.B
1596 +     missReqArb.io.out.ready := false.B
1597 +     missReadyGen.io.queryMQ.foreach(_.ready := false.B)
1598 +     missQueue.io.cmo_req.valid := false.B
1599 +     io.cmoOpReq.ready := false.B
```

Блокировка записей из sbuffer

```
1653 +   // flush: stores wait in the sbuffer. A store writes data (SRAM) and coherence meta (survives the core
1654 +   // reset with CoreResetKeepL1) in MainPipe s3, and a store miss may start an eviction: none may be in
1655 +   // MainPipe when the core reset hits it. With the sweep a store would also re-dirty a released line.
1656 +   when (flushBlock) {
1657 +     mainPipe.io.store_req.valid := false.B
1658 +     io.lsu.store.req.ready := false.B
```

Блокировка атомиков

```
1666 +   // flush: atomics (AMO, LR/SC) wait as well, for the same reason
1667 +   when (flushBlock) {
1668 +     mainPipe.io.atomic_req.valid := false.B
1669 +     io.lsu.atomics.req.ready := false.B
```

Две проверки (assert)

```
1692 +   // core-only reset with CoreResetKeepL1: freeze masks the writes into every L1D array that survives the
1693 +   // reset. Nothing should even try to write then: the DCache was drained, channel A is closed (no refill, and
1694 +   // no instruction runs: the ICache is empty after its reset) and Probes are held in XSTile.
1695 +   assert(!(io.quiesce.freeze && (tag_write_arb.io.out.valid || dataWriteArb.io.out.valid)),
1696 +     "DCache: L1D array write while the core reset is in progress")
1697 +   // after the drain no voluntary Release may start: the reset point must find the WritebackQueue idle
1698 +   assert(!(io.quiesce.done && wb.io.req.fire && wb.io.req.bits.voluntary),
1699 +     "DCache: Release started after the drain reported done")
```

- Первая проверка: пока `freeze`, ни арбитр записи тегов, ни арбитр записи данных не должны даже пытаться писать: опустошение гарантирует, что MainPipe пуст. Вторая: после `done` DCache не начинает добровольных Release.
- Блокировки гарантируют, что после `block` ничто новое не войдёт: загрузки-попадания читают как раньше, а всё, что пишет, меняет meta или отправляет Release, — ждёт в sbuffer / LSU, откуда переигрывается после сброса.

5.7. FakeDCache.scala — заглушка

```
  29 +   // no cache, nothing to drain or flush
  30 +   io.quiesce.done := io.quiesce.req
  31 +   io.quiesce.idle := true.B
  32 +   val alive = RegInit(false.B)
  33 +   alive := true.B
  34 +   io.quiesce.alive := alive
  35     // to LoadUnit
```

Для конфигурации без DCache (`FakeDCache`) нечего опустошать: `done` = `req`, `idle` = 1, `alive` — регистр, поднятый на первом такте. Контроллер не ждёт напрасно.

6. Изменения в ядре, тайле и верхнем уровне

6.1. xiangshan/CoreReset.scala — TLQuiesceGate (новый файл)

Гейт — TileLink-адаптер на ребре ядро → L2Top. Он пропускает сообщения как есть, кроме случаев, когда закрыт канал A (`block`), Probe держится (`holdB`) или отвечается самим гейтом (`ackB`), либо ядро изолировано (`isolate`); он считает транзакции, пересекающие его, и сообщает `quiet`. Вместе с `CoreResetCtrl` и опустошением DCache это адаптер [TL 1.8.1 3.2.3, с. 14].

QuiesceGateIO: четыре входа и выход quiet

```
+ class QuiesceGateIO extends Bundle {
+   val block   = Input(Bool())   // stop new requests on channel A (only between messages)
+   val holdB   = Input(Bool())   // hold Probes in L2: B is neither accepted from L2 nor offered to the core
+   val ackB    = Input(Bool())   // answer Probes here with ProbeAck NtoN, the core sees none (it holds no line)
+   // XSCore reset asserted or being released (a register of CoreResetCtrl): every signal that comes from the
+   // core reset domain (A, C, E valid, D ready; B ready through holdB / ackB, which are high then) is ignored,
+   // nothing is offered to the core. The outputs of the core change asynchronously when its reset is asserted;
+   // this keeps them away from L2 and from the counters below (reset domain crossing isolation).
+   val isolate = Input(Bool())
+   // nothing outstanding across the boundary, no message in transfer (a Probe offered by L2 but not taken
+   // does not count: it is held or answered by the gate)
+   val quiet   = Output(Bool())
```

- `quiet` = ничего не открыто через границу и нет сообщения на середине; Probe, предложенный L2, но не принятый, в `quiet` не входит — он держится или отвечается гейтом. Поля `idle` с `b.valid` (v4) больше нет.

Класс гейта: gateName, early

```
+ class TLQuiesceGate(val gateName: String, val early: Boolean)(implicit p: Parameters) extends LazyModule {
+   val node = TLAdapterNode()
+   lazy val module = new TLQuiesceGateImp(this)
```

Реализация: gname и require

```
+   private val gname = outer.gateName
+
+   val quiets = (outer.node.in zip outer.node.out).map { case ((in, edgeIn), (out, edgeOut)) =>
+     val hasBCE = edgeOut.manager.anySupportAcquireB && edgeOut.client.anySupportProbe
+     // a request of a coherent agent cannot simply be stopped at the start of the sequence: the DCache still
+     // releases lines and answers Probes then
+     require(!(outer.early && hasBCE), s"TLQuiesceGate($gname): an early gate must be on a TL-UL edge")
+
+     out.a <> in.a
```

`gateName` **вместо** `name`: `LazyModule` уже имеет член `name`, поэтому поле v4 `val name` не компилировалось. `early` — гейт некогерентного агента (TL-UL), его канал A закрыт с начала последовательности (`earlyBlock`); иначе — с sDrain (`gateBlock`). `require` запрещает ранний гейт на ребре TL-C: когерентный запрос нельзя остановить в начале, DCache ещё делает Release и отвечает на Probe.

Каналы B и C: holdB и ackB

```
+       // holdB: a Probe waits in L2 (Probe is a single-beat message). B ready to L2 and B valid to the core
+       // drop in the cycle holdB rises: a Probe offered but not taken may be withdrawn from the core
+       // (TileLink 1.8.1 section 4.1: valid may fall while the beat has not been accepted), L2 keeps offering it
+       when (io.holdB) {
+         in.b.valid := false.B
+         out.b.ready := false.B
+       }
+       // ackB: the gate takes the Probe and answers it on behalf of the core: ProbeAck NtoN, the report of a
+       // client that holds no copy (CoreResetKeepL1 = false: L1D released every line before the reset and is
+       // reset with the core). One Probe at a time; C from the core stays closed meanwhile (it is isolated
+       // while the core is in reset, and the core has nothing to send on C then).
+       val ackPending = RegInit(false.B)
+       val ackProbe   = Reg(new TLBundleB(edgeOut.bundle))
+       val takeB      = io.ackB && !io.holdB && !ackPending
+       when (io.ackB || ackPending) {
+         in.b.valid := false.B
+         out.b.ready := takeB
+       }
+       when (out.b.fire && takeB) {
+         ackPending := true.B
+         ackProbe := out.b.bits
+       }
+       when (ackPending) {
+         out.c.valid := true.B
+         out.c.bits := edgeOut.ProbeAck(ackProbe, TLPermissions.NtoN)
+         in.c.ready := false.B
+         when (out.c.ready) { ackPending := false.B }
+       }
```

```
147 +       assert(!(io.ackB && io.holdB), s"TLQuiesceGate($gname): Probes both held and answered")
```

- `holdB`: `in.b.valid` и `out.b.ready` падают в том же такте, где поднимается `holdB`: непринятый beat можно снять [TL 1.8.1 4.1, с. 19]; L2 продолжает предлагать Probe (SourceB). Probe — однобитное сообщение.
- `ackB`: гейт принимает один Probe за раз (`ackPending`), запоминает его (`ackProbe`) и отвечает `edgeOut.ProbeAck(…, NtoN)`: source, адрес и размер копируются из Probe, как требует [TL 1.9.3 9.3.5, с. 78]. С ядра в это время закрыт (`in.c.ready := false`).
- assert-ы: Probe не держится и не отвечается одновременно; пока `isolate`, канал B закрыт.

Изоляция C и E

```
110 +       // isolation of C and E: C and E valid of the core are not looked at (a ProbeAck of the gate itself is
111 +       // still sent, below). B needs nothing more: whenever isolate is high, holdB (keep) or ackB (sweep) is hig
        ↪ h
112 +       // too and closes B (no Probe to the core, its B ready is not looked at; asserted below).
113 +       when (io.isolate) {
114 +         in.c.ready := false.B
115 +         out.c.valid := false.B
116 +         in.e.ready := false.B
117 +         out.e.valid := false.B
```

Блокировка A на границе сообщения и изоляция A, D

```
158 +     // message boundaries (each firstlast call instantiates one beat counter per channel)
159 +     val (aFirst, _, _)     = edgeIn.firstlast(in.a)
160 +     val (dFirst, dLast, _) = edgeIn.firstlast(in.d)
161 +
162 +     // block A only at a message boundary, so a multi-beat Put is never cut
163 +     when (io.block && aFirst) {
164 +       out.a.valid := false.B
165 +       in.a.ready := false.B
166 +     }
167 +     // isolation of A and D (the drain left nothing in flight: no A message is cut, no D is pending)
168 +     when (io.isolate) {
169 +       out.a.valid := false.B
170 +       in.a.ready := false.B
171 +       in.d.valid := false.B
```

Счётчики и quiet

```
175 +     def counter(inc: Bool, dec: Bool, what: String): UInt = {
176 +       val c = RegInit(0.U(8.W))
177 +       c := c + inc.asUInt - dec.asUInt
178 +       assert(!(dec && !inc && c === 0.U), s"TLQuiesceGate($gname): $what underflow")
179 +       assert(!(inc && !dec && c.andR), s"TLQuiesceGate($gname): $what overflow")
180 +       c
181 +     }
182 +
183 +     val dIsReleaseAck = in.d.bits.opcode === ReleaseAck
184 +     val dIsGrant      = in.d.bits.opcode === Grant || in.d.bits.opcode === GrantData
185 +
186 +     // every request on A gets exactly one response message on D
187 +     val reqs = counter(in.a.fire && aFirst, in.d.fire && dLast && !dIsReleaseAck, "A->D")
188 +
189 +     val quietBCE = if (hasBCE) {
190 +       val (cFirst, cLast, _) = edgeIn.firstlast(in.c)
191 +       val cIsProbeAck = in.c.bits.opcode === ProbeAck || in.c.bits.opcode === ProbeAckData
192 +       val cIsRelease  = in.c.bits.opcode === Release || in.c.bits.opcode === ReleaseData
193 +       // Grant -> GrantAck, Probe -> ProbeAck, Release -> ReleaseAck. The counters see the messages where they
194 +       // cross the gate, so a message still in the L1D buffer (l1d_to_l2_buffer, between the DCache and the
195 +       // gate) is covered too: a ProbeAck on its way keeps probes above 0, a GrantAck keeps grants above 0.
196 +       val grants   = counter(in.d.fire && dFirst && dIsGrant, in.e.fire, "Grant->GrantAck")
197 +       val probes   = counter(in.b.fire && in.b.bits.opcode === Probe, in.c.fire && cLast && cIsProbeAck, "Probe-
        ↪ >ProbeAck")
198 +       val releases = counter(in.c.fire && cLast && cIsRelease, in.d.fire && dIsReleaseAck, "Release->ReleaseAck"
        ↪ )
199 +       grants === 0.U && probes === 0.U && releases === 0.U && cFirst
200 +     } else {
201 +       true.B
202 +     }
203 +
204 +     reqs === 0.U && aFirst && dFirst && quietBCE
205 +   }
206 +   io.quiet := quiets.reduce(_ && _)
```

- Блокировка A — только при `aFirst` (на границе сообщения): многобитное Put не обрывается (формальное свойство P4).
- `quiet = reqs === 0 && aFirst && dFirst && (grants === 0 && probes === 0 && releases === 0 && cFirst)` для TL-C; для TL-UL — без блока BCE. Счётчики считают сообщения там, где они пересекают гейт: сообщение, ещё лежащее в `l1d_to_l2_buffer`, тоже учтено — поэтому нужен

`quiet` гейта, а не только флаг DCache.

6.2. xiangshan/CoreReset.scala — TLResetIsolation (новое)

Адаптер на ребре, где менеджер находится внутри ядра (регистры ICache ctrl, путь: MMIO ядра → гейт → буфер MMIO-крестовины L2Top → `TLResetIsolation` → ядро). Пока ядро в сбросе или выходит из него, ему ничего не предлагается и ни один его сигнал (`A ready`, `D valid`) не доходит до тайла. Достижимы здесь только собственные обращения ядра к этим регистрам, которые уже проходят гейты `i_mmio` / `d_mmio`; `quiet` сообщает контроллеру то же, что гейт.

TLResetIsolation: изоляция A, D и счётчик запросов

```
219 + class TLResetIsolationImp(outer: TLResetIsolation) extends LazyModuleImp(outer) {
220 +   val io = IO(new Bundle {
221 +     val isolate = Input(Bool())
222 +     val quiet   = Output(Bool())   // no request to the core is waiting for its response
223 +   })
224 +   val quiets = (outer.node.in zip outer.node.out).map { case ((in, edgeIn), (out, edgeOut)) =>
225 +     require(!(edgeOut.manager.anySupportAcquireB && edgeOut.client.anySupportProbe),
226 +       "TLResetIsolation: TL-UL edges only")
227 +     out.a <> in.a
228 +     in.d <> out.d
229 +     in.b.valid := false.B
230 +     in.c.ready := true.B
231 +     in.e.ready := true.B
232 +     out.b.ready := true.B
233 +     out.c.valid := false.B
234 +     out.e.valid := false.B
235 +     when (io.isolate) {
236 +       out.a.valid := false.B   // no request into the core while it is in reset (it would be lost)
237 +       in.a.ready := false.B    // the ready of the core is not looked at
238 +       in.d.valid := false.B    // nor its responses
239 +       out.d.ready := false.B
240 +     }
241 +     // requests on A (first beat) and their responses on D (last beat), seen on the tile side
242 +     val (aFirst, _, _) = edgeIn.firstlast(in.a)
243 +     val (_, dLast, _)  = edgeIn.firstlast(in.d)
244 +     val reqs = RegInit(0.U(8.W))
245 +     reqs := reqs + (in.a.fire && aFirst).asUInt - (in.d.fire && dLast).asUInt
246 +     assert(!(in.d.fire && dLast && !(in.a.fire && aFirst) && reqs === 0.U), "TLResetIsolation: A->D underflow")
247 +     assert(!(io.isolate && reqs =/= 0.U), "TLResetIsolation: request in flight while XSCore is in reset")
248 +     reqs === 0.U && aFirst
249 +   }
250 +   io.quiet := quiets.foldLeft(true.B)(_ && _)
```

- Выход `quiet` переименован из `idle` (v4). Assert `!(isolate && reqs =/= 0)` проверяет, что к сбросу ничего не висит.
- Мутант `ISO_MUTANT_ALWAYS_QUIET` (quiet всегда 1) эквивалентен: запрос к этим регистрам уже учтён гейтом клиента (7.2).

6.3. xiangshan/CoreReset.scala — CoreResetCtrl

Интерфейс контроллера

```
253 + class CoreResetCtrlIO extends Bundle {
254 +   val req         = Input(Bool())   // level, tile clock: reset XSCore; it is held in reset while high
255 +   val busy        = Output(Bool())  // a core reset sequence is in progress
256 +   val inReset     = Output(Bool())  // XSCore reset is asserted
257 +   val hartInReset = Output(Bool())  // from the core reset until the core runs again (Debug Module)
258 +   val l1          = new xiangshan.cache.L1QuiesceIO
259 +   // reset of the frontend, the last of the reset chain of XSCore (core reset domain; looked at in sRelease)
260 +   val coreInReset = Input(Bool())
261 +   val earlyBlock  = Output(Bool())  // channel A of the non-coherent agents closed (from sFlush)
262 +   val gateBlock   = Output(Bool())  // channel A of L1D closed (from sDrain)
263 +   val holdProbe   = Output(Bool())  // channel B held in L2 (sDrain; and in sReset, sRelease with CoreResetKeepL
        ↪ 1)
264 +   val ackProbe    = Output(Bool())  // Probes answered NtoN by the gate (CoreResetKeepL1 = false: L1D swept)
265 +   // XSCore reset asserted or being released (sReset, sRelease): every signal that leaves the core reset
266 +   // domain must be isolated on the tile side (gates, XSTile). A register, glitch-free; it rises on the same
267 +   // clock edge as coreReset, so the first edge that could sample an output of the core changed by the
268 +   // (asynchronous) reset assertion already sees the isolation.
269 +   val isolate     = Output(Bool())
270 +   val earlyQuiet  = Input(Bool())   // every gate of a non-coherent agent is quiet
271 +   val gatesQuiet  = Input(Bool())   // every gate and every TLResetIsolation is quiet
272 +   val coreReset   = Output(Bool())  // to XSCore reset (synchronised by the tile)
```

Контроллер: параметры, регистры

```
282 + class CoreResetCtrl(keepL1: Boolean, rstCycles: Int = 100) extends Module {
283 +   val io = IO(new CoreResetCtrlIO)
284 +
285 +   val sIdle :: sFlush :: sDrain :: sReset :: sRelease :: Nil = Enum(5)
```

```
286 +   val state = RegInit(sIdle)
287 +   val cnt = RegInit(0.U(8.W))
288 +   require(rstCycles >= 100 && rstCycles < 255, "TileLink 1.8.1, 3.2.2: at least 100 cycles of reset")
289 +
290 +   val coreReset = RegInit(false.B)
291 +   val hartInReset = RegInit(false.B)
292 +   // sReset or sRelease, as a register (see CoreResetCtrlIO.isolate)
293 +   val closed = RegInit(false.B)
```

Входы из домена сброса ядра: маскирование регистром closed

```
295 +   // the inputs from the DCache and the frontend come from the core reset domain: done and idle are used only
296 +   // while the core runs (sFlush, sDrain), alive and the frontend reset only when the core leaves reset
297 +   // (sRelease, the release is synchronous); in sReset, when the core reset is asserted asynchronously, none
298 +   // of them reaches the state machine
299 +   val l1Done = io.l1.done && !closed
300 +   val l1Idle = io.l1.idle && !closed
301 +   val coreUp = io.l1.alive && !io.coreInReset && state === sRelease
```

Автомат и assert-ы

```
303 +   switch (state) {
304 +     is (sIdle) {
305 +       cnt := 0.U
306 +       when (io.req) { state := sFlush }
307 +     }
308 +     is (sFlush) {
309 +       // channel A of the non-coherent agents is closed (earlyBlock); DCache drain (and sweep, without keepL1);
310 +       // L2 may still probe the DCache. A Get of a non-coherent agent may wait in L2 for a Probe of L1D, so B
311 +       // stays open until every early gate is quiet.
312 +       when (l1Done && io.earlyQuiet) { state := sDrain }
313 +     }
314 +     is (sDrain) {
315 +       // channel A of L1D closed, B closed from the first cycle (holdProbe). The decision looks at quiet of this
316 +       // cycle (the counters already include a Probe taken in the last cycle of sFlush), so nothing that
317 +       // crossed a gate before can be lost.
318 +       // The emptiness flags do not cover the one registered write stage of the L1D data and meta arrays
319 +       // (BankedDataArray, AsynchronousMetaArray): a write of an s3 fire in the previous cycle is committed
320 +       // into the array at the edge that ends this cycle, the same edge at which coreReset rises, so it lands
321 +       // before the asynchronous reset assertion. No delay may be added between this decision and coreReset.
322 +       when (io.gatesQuiet && l1Done && l1Idle) {
323 +         state := sReset; cnt := 0.U; coreReset := true.B; hartInReset := true.B; closed := true.B
324 +       }
325 +     }
326 +     is (sReset) {
327 +       // at least rstCycles, and until the request drops (Probes held or answered by the gate meanwhile)
328 +       when (cnt < rstCycles.U) { cnt := cnt + 1.U }
329 +       when (cnt >= rstCycles.U && !io.req) { state := sRelease; coreReset := false.B }
330 +     }
331 +     is (sRelease) {
332 +       // the core leaves reset through its reset synchronisers; channels A and B stay closed (and the core
333 +       // isolated) until the DCache (with keepL1 it takes the held Probes) and the frontend are out of reset
334 +       when (coreUp) {
335 +         state := sIdle; closed := false.B; hartInReset := false.B
336 +       }
337 +     }
338 +   }
339 +
340 +   // nothing may cross the core/L2 boundary while XSCore is in reset (a message would be lost and
341 +   // L2 would wait for its answer forever); a Probe may be offered, it is held or answered by the gate
342 +   assert(!(state === sReset && !io.gatesQuiet),
343 +     "CoreResetCtrl: TileLink traffic between XSCore and L2 while XSCore is in reset")
344 +   // the isolation covers the whole time the core reset is asserted
345 +   assert(closed === (state === sReset || state === sRelease), "CoreResetCtrl: isolation out of step with the sta
        ↪ te")
346 +   assert(!coreReset || closed, "CoreResetCtrl: XSCore reset without isolation")
```

Выходы — функции регистров

```
348 +   io.busy := state =/= sIdle
349 +   io.inReset := state === sReset
350 +   io.hartInReset := hartInReset
351 +   // keep the DCache drained and blocked until the core is reset
352 +   io.l1.req := state === sFlush || state === sDrain
353 +   io.l1.freeze := (if (keepL1) closed else false.B)
354 +   // channel A: the non-coherent agents from the start, L1D from sDrain (it releases lines in sFlush)
355 +   io.earlyBlock := state =/= sIdle
356 +   io.gateBlock := state === sDrain || closed
357 +   // channel B, a function of the state register: closed in the first cycle of sDrain. With keepL1 the Probe
358 +   // waits in L2 until the DCache is out of reset (L1D holds the line, maybe dirty); without it the gate answers
359 +   // NtoN while the core is in reset (L1D released everything before), and holds it in sDrain (the DCache is
360 +   // still there, but no new Probe may reach it any more)
361 +   io.holdProbe := (if (keepL1) state === sDrain || closed else state === sDrain)
362 +   io.ackProbe := (if (keepL1) false.B else closed)
363 +   io.isolate := closed
364 +   io.coreReset := coreReset
```

- `l1Done`, `l1Idle` маскируются `!closed`: они приходят из домена сброса ядра и в sReset (когда сброс установлен асинхронно) могли бы быть произвольными; `coreUp` (`alive && !coreInReset && sRelease`) смотрится только в sRelease, когда снятие синхронно.
- `sFlush → sDrain` — `l1Done && earlyQuiet`; `sDrain → sReset` — `gatesQuiet && l1Done && l1Idle` в том же такте (с `coreReset := 1`, `hartInReset := 1`, `closed := 1`); `sReset → sRelease` — `cnt >= rstCycles && !req`; `sRelease → sIdle` — `coreUp`.
- Комментарий в sDrain фиксирует инвариант нулевого запаса (1.5.3): между решением и `coreReset` ничего добавлять нельзя.
- `require(rstCycles >= 100 && rstCycles < 255)`: счётчик 8-битный; 100 — правило TileLink 3.2.2.
- Assert-ы: `gatesQuiet` в sReset; `closed ≡ sReset || sRelease`; `coreReset ⇒ closed`. Их проверяют симуляция и формально (P1, P16).

6.4. mem/MemBlock.scala, xiangshan/XSCore.scala — проброс до DCache

XSCore: порты

```
121 +     // L1 DCache quiesce before a reset of XSCore alone (driven by XSTile)
122 +     val l1_quiesce = Flipped(new L1QuiesceIO)
123 +     // tile reset for the L1 DCache arrays that survive a reset of XSCore alone (CoreResetKeepL1)
124 +     val l1_meta_reset = Option.when(p(SoCParamsKey).EnableCoreReset && p(SoCParamsKey).CoreResetKeepL1)(Input(As
              ↪ yncReset()))
```

XSCore: соединение с MemBlock

```
216 +   memBlock.io.l1_quiesce <> io.l1_quiesce
217 +   memBlock.io.l1_meta_reset.foreach(_ := io.l1_meta_reset.get)
```

MemBlock → DCache

```
640 +   dcache.io.quiesce <> io.l1_quiesce
641 +   dcache.io.metaReset.foreach(_ := io.l1_meta_reset.get)
```

Два порта прокладываются без изменений логики: `l1_quiesce` (L1QuiesceIO, с `Flipped`) и `l1_meta_reset` (асинхронный сброс тайла, существует только при `EnableCoreReset && CoreResetKeepL1`).

6.5. xiangshan/XSTile.scala — гейты, изоляция, контроллер, сбросы

coreGate: гейт на каждое ребро

```
 73 +   // One TLQuiesceGate on every TileLink edge from XSCore to L2Top: it can stop channel A, hold channel B
 74 +   // (Probes) or answer them itself (NtoN), isolates the core side while XSCore is in reset, and tells when
 75 +   // no transaction is outstanding across the core/L2 boundary (see CoreReset.scala).
 76 +   // early: a non-coherent agent (TL-UL), its channel A is closed from the start of the sequence (sFlush);
 77 +   // the coherent L1D keeps A open until sDrain (it releases lines and answers Probes in sFlush)
 78 +   val coreGates = scala.collection.mutable.ListBuffer[TLQuiesceGate]()
 79 +   private def coreGate(name: String, early: Boolean) = {
 80 +     if (EnableCoreReset) {
 81 +       val g = LazyModule(new TLQuiesceGate(name, early))
 82 +       coreGates += g
 83 +       g.node
 84 +     } else {
 85 +       TLTempNode()
```

Вставка гейтов в рёбра (l1d — early = false, остальные — true)

```
101     // L1 to l1_xbar
102     coreParams.dcacheParametersOpt.map { _ =>
    -     l2top.inner.misc_l2_pmu := l2top.inner.l1d_logger := memBlock.dcache_port :=
103 +     l2top.inner.misc_l2_pmu := l2top.inner.l1d_logger := coreGate("l1d", early = false) := memBlock.dcache_port
              ↪ :=
104         memBlock.l1d_to_l2_buffer.node := memBlock.dcache.clientNode
105     }
106
    -   l2top.inner.misc_l2_pmu := l2top.inner.l1i_logger := memBlock.frontendBridge.icache_node
107 +   l2top.inner.misc_l2_pmu := l2top.inner.l1i_logger := coreGate("l1i", early = true) := memBlock.frontendBridge.
          ↪ icache_node
108     if (!coreParams.softPTW) {
    -     l2top.inner.misc_l2_pmu := l2top.inner.ptw_logger := l2top.inner.ptw_to_l2_buffer.node := memBlock.ptw_to_l2
              ↪ _buffer.node
```

Контроллер и связи с гейтами

```
198 +     // core-only reset sequencer (tile clock and tile reset domain)
199 +     val coreResetCtrl = Option.when(EnableCoreReset)(Module(new CoreResetCtrl(CoreResetKeepL1)))
200 +     coreResetCtrl match {
201 +       case Some(ctl) =>
202 +         // asynchronous requests: 3-flop synchronisers, cleared by the tile reset
203 +         val debugReq = AsyncResetSynchronizerShiftReg(io.debug_reset.get, 3, 0)
204 +         val clusterReq = AsyncResetSynchronizerShiftReg(io.core_reset_req.get, 3, 0)
```

```
+         ctl.io.req := debugReq || clusterReq
+         io.core_reset_busy.get := ctl.io.busy
+         core.module.io.l1_quiesce <> ctl.io.l1
+         coreGates.foreach { g =>
+           // channel A: non-coherent agents from sFlush, L1D from sDrain
+           g.module.io.block := (if (g.early) ctl.io.earlyBlock else ctl.io.gateBlock)
+           g.module.io.holdB := ctl.io.holdProbe
+           g.module.io.ackB := ctl.io.ackProbe
+           g.module.io.isolate := ctl.io.isolate
+         }
+         coreIsolations.foreach(_.module.io.isolate := ctl.io.isolate)
+         ctl.io.earlyQuiet := coreGates.filter(_.early).map(_.module.io.quiet).foldLeft(true.B)(_ && _)
+         ctl.io.gatesQuiet := (coreGates.map(_.module.io.quiet) ++ coreIsolations.map(_.module.io.quiet))
+           .foldLeft(true.B)(_ && _)
+         // the frontend is the last part of XSCore to leave reset (ResetGen chain memBlock -> backend ->
+         // frontend); CoreResetCtrl looks at it in sRelease only
+         ctl.io.coreInReset := core.module.io.resetInFrontend
+       case None =>
+         core.module.io.l1_quiesce.req := false.B
+         core.module.io.l1_quiesce.freeze := false.B
```

Для каждого гейта: `block` = `earlyBlock` (если `g.early`) или `gateBlock`; `holdB` = `holdProbe`, `ackB` = `ackProbe`, `isolate` = `isolate`. `earlyQuiet` — И `quiet` ранних гейтов, `gatesQuiet` — всех гейтов и `TLResetIsolation`. `coreInReset` = `resetInFrontend` ядра: последний в цепочке ResetGen (MemBlock → Backend → Frontend), он маскируется структурно, как `l1.*`.

isoReset и isoHold

```
+     // ---- isolation of the outputs of XSCore (EnableCoreReset) ----
+     // The core reset is asserted asynchronously (ResetGen synchronises only its release), so the outputs of
+     // XSCore change at no particular clock edge then (reset domain crossing). isolate (CoreResetCtrl: a registe
          ↪ r
+     // raised on the same clock edge as the core reset request, high in sReset and sRelease) replaces each
+     // control output on the tile side by a value that does not depend on the core:
+     //   isoReset(x, r): r, the value x takes in reset (valid signals 0, levels of a handshake, status levels):
+     //                   the tile sees what a reset of the core shows it, only at a clock edge;
+     //   isoHold(x)    : the value x had before the reset (configuration levels with no handshake).
+     // A data field qualified by an isolated valid needs nothing (it is not looked at without its valid).
+     // TileLink is isolated in the gates and in TLResetIsolation, the DCache handshake (l1_quiesce.done, alive)
+     // in CoreResetCtrl, the L1D arrays that survive the reset by l1_quiesce.freeze; the inputs of XSCore need
+     // nothing. The interrupts reach the core through IntBuffer (tile side) and are inputs too.
+     val iso = coreResetCtrl.map(_.io.isolate)
+     def isoReset[T <: Data](x: T, r: T): T = iso.map(i => Mux(i, r, x)).getOrElse(x)
+     def isoHold[T <: Data](x: T): T = iso.map(i => Mux(i, RegEnable(x, !i), x)).getOrElse(x)
```

Изоляция выходов: состояние, MSI, hartIsInReset

```
+     // status levels, 0 in reset
+     l2top.module.io.cpu_halt.fromCore := isoReset(core.module.io.cpu_halt, false.B)
      io.cpu_halt := l2top.module.io.cpu_halt.toTile
-     l2top.module.io.cpu_critical_error.fromCore := core.module.io.cpu_critical_error
+     l2top.module.io.cpu_critical_error.fromCore := isoReset(core.module.io.cpu_critical_error, false.B)
      io.cpu_crtical_error := l2top.module.io.cpu_critical_error.toTile
-     l2top.module.io.msiAck.fromCore := core.module.io.msiAck
+     // MSI handshake with the IMSIC bus bridge (vld_req / vld_ack levels; the bridge waits for a rising edge of
+     // ack): ack is 0 in reset. Not a hold: a held 1 would hide the 0 of the core's reset and the next rising
+     // edge, and the bridge would wait for it forever
+     l2top.module.io.msiAck.fromCore := isoReset(core.module.io.msiAck, false.B)
      io.msiAck := l2top.module.io.msiAck.toTile
      l2top.module.io.teemsiAck zip core.module.io.teemsiAck foreach { case (l2top_teemsiAck, core_teemsiAck) =>
-       l2top_teemsiAck.fromCore := core_teemsiAck
+       l2top_teemsiAck.fromCore := isoReset(core_teemsiAck, false.B)
      }
      io.teemsiAck zip l2top.module.io.teemsiAck foreach { case (io_teemsiAck, l2top_teemsiAck) =>
        io_teemsiAck := l2top_teemsiAck.toTile
      }

-     l2top.module.io.hartIsInReset.resetInFrontend := core.module.io.resetInFrontend
-     io.hartIsInReset := l2top.module.io.hartIsInReset.toTile
```

Сбросы ядра и массивов L1D

```
+         // XSCore reset = tile reset (via the L2Top reset chain, if any) OR core-only reset;
+         // asserted asynchronously, released synchronously to the tile clock
+         val coreBaseReset = if (debugOpts.ResetGen && enableL2) l2top.module.reset_core.asBool else reset.asBool
+         core.module.reset := withClockAndReset(clock, (coreBaseReset || ctl.io.coreReset).asAsyncReset) {
+           ResetGen(io.dft_reset)
+         }
+         // CoreResetKeepL1: the L1D arrays that survive a core-only reset (coherence meta, error flags) follow
+         // the tile reset only, with the same synchroniser, so they are released together with the core on a
+         // tile reset. Without it they are reset with the rest of XSCore.
+         core.module.io.l1_meta_reset.foreach { _ := withClockAndReset(clock, coreBaseReset.asAsyncReset) {
+           ResetGen(io.dft_reset)
+         }.asAsyncReset }
+       case None =>
+         if (debugOpts.ResetGen && enableL2) {
+           core.module.reset := l2top.module.reset_core
```

- Сброс ядра = (сброс тайла через цепочку L2Top, если есть) ИЛИ `coreReset`, асинхронная установка, синхронное снятие (`ResetGen`). Сброс массивов L1D (`l1_meta_reset`) — только сброс тайла, тот же синхронизатор.
- Мосты предвыборки разрезаны на приёмник и источник (`BundleBridgeSink` / `BundleBridgeSource`) ради изоляции `addr_valid`.
- Запросы L2-предвыборщика к TLB ядра (`l2_tlb_req.req.valid`) закрыты, пока `busy`; ответы изолируются.

6.6. xiangshan/XSTileWrap.scala, top/XSNoCTop.scala, top/Top.scala

XSTileWrap: порты

```
104 +       // reset of XSCore (core + L1) alone, L2 and CHI keep running (EnableCoreReset; L1D kept, or swept into
105 +       // L2 and reset with the core: CoreResetKeepL1):
106 +       //   debug_reset    - level, asynchronous: Debug Module ndmreset / hartreset of this hart
107 +       //   core_reset_req - level, asynchronous: reset controller of the cluster
108 +       //   core_reset_busy - tile clock domain: high from the request until the core runs again
109 +       // XSCore is held in reset while a request is high (at least 100 cycles) and starts when it drops.
110 +       // With EnableCoreReset, hartResetReq (Debug Module hartreset) is a debug_reset too.
111 +       val debug_reset = Option.when(EnableCoreReset) (Input(Bool()))
112 +       val core_reset_req = Option.when(EnableCoreReset) (Input(Bool()))
113 +       val core_reset_busy = Option.when(EnableCoreReset) (Output(Bool()))
```

hartResetReq больше не сбрасывает тайл

```
119 +     // hartResetReq resets the whole tile, or only XSCore with EnableCoreReset (a reset of the whole
120 +     // tile in the middle of CHI traffic breaks the CHI link, see CoreReset.scala)
121 +     val hartResetsTile = if (EnableCoreReset) false.B else io.hartResetReq
122 +     val reset_sync = withClockAndReset(clock, (reset.asBool || hartResetsTile).asAsyncReset)(ResetGen(io.dft_res
        ↪ et))
```

Запросы

```
181 +     // core-only reset requests: synchronised in XSTile
182 +     tile.module.io.debug_reset.foreach { _ := io.debug_reset.get || io.hartResetReq }
183 +     tile.module.io.core_reset_req.foreach { _ := io.core_reset_req.get }
184 +     io.core_reset_busy.foreach { _ := tile.module.io.core_reset_busy.get }
```

XSNoCTop: пробуждение такта

```
167 +       // A core-only reset needs the Core+L2 clock (the drain runs on it, the reset is counted on it, and
168 +       // without CoreResetKeepL1 the gate in front of L1D answers Probes while the core is in reset): a request
169 +       // wakes the gated clock like a flit and keeps it from being gated again until the sequence is over.
170 +       val coreResetReq = if (socParams.EnableCoreReset) {
171 +         tileio.debug_reset.get || tileio.hartResetReq || tileio.core_reset_req.get
172 +       } else false.B
173 +       val coreResetWake = withClockAndReset(clock, cpuReset_sync) {
174 +         AsyncResetSynchronizerShiftReg(
175 +           coreResetReq || core_with_l2.module.io.core_reset_busy.getOrElse(false.B),
176 +           3, 0)
177 +       }
178 +       wfiState := withClockAndReset(clock, cpuReset_sync){
179 +         WfiStateNext(wfiState, isWFI && !coreResetWake, isNormal, flitpend || coreResetWake, false.B)
```

XSNoCTop: порты

```
262 +     // reset of XSCore alone, L2 and CHI keep running (see XSTileWrap, CoreReset.scala):
263 +     // debug_reset from the Debug Module (ndmreset / hartreset), core_reset_req from the cluster
264 +     val debug_reset = Option.when(socParams.EnableCoreReset)(Input(Bool()))
265 +     val core_reset_req = Option.when(socParams.EnableCoreReset)(Input(Bool()))
266 +     val core_reset_busy = Option.when(socParams.EnableCoreReset)(Output(Bool()))
```

XSTop: заглушки

```
495 +       core.module.io.debug_reset.foreach(_ := false.B)
496 +       core.module.io.core_reset_req.foreach(_ := false.B)
```

Порты тайла и XSNoCTop не изменились относительно v4. В XSNoCTop запрос или `core_reset_busy` будят отключённый в WFI такт и не дают его отключить, пока последовательность не закончилась.

6.7. system/SoC.scala, top/YamlParser.scala — параметры

SoCParameters

```
145 +   // Reset of XSCore (core + L1) only, while L2 and the CHI port keep running (Debug Module ndmreset /
146 +   // hartreset, or a cluster reset controller): L1 DCache is drained, the core<->L2 TileLink boundary is
147 +   // drained (see CoreReset.scala). XSCore stays in reset as long as the request is high (at least 100
148 +   // cycles), and no Probe reaches L1D meanwhile.
149 +   EnableCoreReset: Boolean = false,
150 +   // true : L1D keeps its lines across the core reset (coherence meta and error flags on the tile reset,
151 +   //        and the tag array, which has an initialisation counter, on the tile reset; the data SRAMs have no
152 +   //        reset; tag and data are not written meanwhile); L2 takes dirty data from the "hot" L1D with an
153 +   //        ordinary Probe. A Probe offered while the core is in reset waits in L2 until the core is out of
```

```
154 +   //        reset (and so does a snoop from the HN that needs it)
155 +   // false: L1D is swept before the reset: every valid line is released to L2 (Release, or ReleaseData for
156 +   //        a dirty line, the eviction path of MainPipe), then L1D is reset with the core (meta and error
157 +   //        flags on the core reset, as without EnableCoreReset). No data is lost, L2's directory lists no
158 +   //        L1D copy afterwards; the sweep takes about nSets * nWays cycles plus the ReleaseAcks. A Probe
159 +   //        offered while the core is in reset is answered by the gate in front of L1D (ProbeAck NtoN)
160 +   CoreResetKeepL1: Boolean = true
```

YamlConfig

```
 63 +   EnableCoreReset: Option[Boolean],
 64 +   CoreResetKeepL1: Option[Boolean],
```

`CoreResetLevel` удалён из `SoCParameters` и `YamlConfig` (комментарий влит в описание `CoreResetKeepL1`). YAML-ключ `CoreResetLevel` в старых конфигурациях игнорируется (circe `withDefaults`): старые YAML не ломаются.

7. Проверка

Проверка идёт четырьмя независимыми путями: компиляция и elaboration Chisel (7.1), формальное доказательство гейтов, изоляции и контроллера в произвольном окружении (7.2), симуляция моделей с проверяющим тестбенчем (7.3) и временные диаграммы из VCD (7.4). Все числа — из файлов результатов (`core-reset/verification/results_*`, `coverage_*.json`); если файла нет, документ пишет «нет данных». Это проверка **моделей, которые повторяют Scala-код один в один, и RTL трёх модулей через эквивалентность**, а не симуляция RTL всего ядра.

7.1. Chisel: компиляция, elaboration, эквивалентность RTL ↔ моделям

Компиляция. `mill xiangshan.compile` проходит на коммитах v5 (Scala 2.13.15, Chisel 6.7, firtool 1.62.1). Версия v4 не компилировалась (конфликт `TLQuiesceGate.name` и `LazyModule.name`).

Elaboration модулей. Стенд `verification/elab/CoreResetElabHarness.scala` (object `xiangshan.CoreResetElab`) собирает отдельные модули в Verilog: `CoreResetCtrl` (KEEP и SWEEP), гейты `TLQuiesceGate` (TL-C и TL-UL), `TLResetIsolation`; а также `DCache` (`dcache_keep`, `dcache_sweep`: DCacheWrapper с менеджером и клиентом ICache ctrl). Из вывода первых трёх целей `scripts/extract_rtl.py` вырезает RTL, на котором работают миттеры:

Файл RTL (`verification/rtl/`)	Модуль	Есть
`CoreResetCtrl_keep.sv`	CoreResetCtrl, KEEP	да (216 строк)
`CoreResetCtrl_sweep.sv`	CoreResetCtrl, SWEEP	да (216 строк)
`TLQuiesceGate.sv`	TLQuiesceGate, TL-C (l1d)	да (436 строк)
`TLQuiesceGate_1.sv`	TLQuiesceGate, TL-UL (ранний)	да (226 строк)
`TLResetIsolation.sv`	TLResetIsolation	да (221 строк)

Результат elaboration целей `dcache_keep` / `dcache_sweep` в пакет результатов не записан: нет данных.

Эквивалентность RTL ↔ модели. Пять миттеров (`formal/eq_chisel.sv`, `formal/run_equiv.sh`) сравнивают RTL, сгенерированный Chisel / firtool, с моделями (`model/cr_ctl_models.sv`), на которых построены формальная проверка и симуляция. Все входы свободны, каждый выход сравнивается в каждом такте, кроме первого (там сброс). Ограничения входов: коды операций выбираются из малого свободного селектора, так что встречаются только допустимые для канала (A: PutFull, Get, AcquireBlock, AcquirePerm; C: ProbeAck(Data), Release(Data); D: AccessAck(Data), Grant(Data), ReleaseAck; B: Probe); размер сообщения — 16 байт (2 beat-а по 8 Б), адрес с нулевыми старшими битами, маска единицы, corrupt / denied = 0, данные не сравниваются. Проверка: BMC глубины 30 и доказательство (abc pdr).

Миттер	BMC 30	Время	Доказательство (prove)	Время
CoreResetCtrl, KEEP	PASS	1,0 с	PASS	1,2 с
CoreResetCtrl, SWEEP	PASS	1,1 с	PASS	2,6 с
TLQuiesceGate, TL-C	PASS	115,5 с	PASS	4,1 с
TLQuiesceGate, TL-UL	PASS	8,5 с	PASS	1,0 с
TLResetIsolation	PASS	9,8 с	PASS	1,3 с

Итог: 5 из 5 миттеров эквивалентны; неудачных миттеров: 0.

Полная elaboration XSNoCTop

Полная elaboration XSNoCTop **не выполнена**: файла `results_elab.txt` нет. Полная elaboration — единственное, что проверило бы соединения XSTile на реальных модулях (гейты на пяти рёбрах, изоляция, контроллер, сбросы). Для неё нужен YAML только с ключами сброса (`verification/elab/xsnoctop_keep.yml`, `xsnoctop_sweep.yml`): конфигурация XSNoCTop из `Default.yml` (с `tp: true`) ломает elaboration независимо от сброса.

7.2. Формальное доказательство

SymbiYosys (движок abc pdr — без ограничения глубины; BMC и cover — smtbmc, решатель yices) проверяет свойства для всех достижимых состояний трёх гейтов (TL-C для DCache и два TL-UL ранних), `TLResetIsolation` и `CoreResetCtrl` (`formal/fv_boundary.sv`; модели один в один с `CoreReset.scala`, их RTL доказан эквивалентным в 7.1). Окружение произвольно в рамках правил TileLink: запрос сброса приходит и снимается в любой такт и держится сколько угодно.

Допущения среды (вне допущений — всё свободно):

- TileLink: beat-ы данных принадлежат одному сообщению; ответ — только на открытый запрос (Grant → Acquire, ReleaseAck → Release, AccessAck → Put / Get, GrantAck → Grant, ProbeAck → Probe); L2 может предложить Probe в любой такт; до 5 открытых транзакций каждого вида. Устойчивость valid **не** предполагается: [TL 1.8.1 4.1, с. 19] разрешает снять valid непринятого beat-a.
- Выходы ядра в сбросе: пока `CoreResetCtrl` в sReset — **все** выходы ядра произвольны в каждом такте, в том числе `l1.done`, `l1.idle`, `l1.alive` и признак сброса frontend; пока сброс снимается через синхронизатор (sRelease) — значения сброса (valid 0, `alive` 0, `done` 0). Вне сброса ядро подчиняется TileLink.
- Сброс frontend: `fe_rst = core_rst || fe_q2`, frontend выходит на 2 такта позже остального ядра.
- Drain DCache: после перехода контроллера из sIdle / sFlush DCache не начинает новых Release (проверяет monitor в симуляции и assert в DCacheWrapper); `l1.done` только при `l1.req`; `l1.idle` свободен (и в сбросе); `l1.alive` = 0 в сбросе и поднимается не позже чем через 3 такта после него (нужно только для границы P8B).

Независимые счётчики-эталоны (`fv_boundary.sv`) считают транзакции отдельно от счётчиков гейта: A, D, E — на стороне L2; Probe и C — на стороне ядра; Probe, на который гейт ответил сам, отслеживается отдельно.

Свойство	Что утверждает	Зачем
P1	в sReset через границу ничего не открыто (включая запросы к ICache ctrl), ни одно сообщение не на середине	главное свойство: сброс не обрывает транзакцию и не оставляет L2 в ожидании
P2	в sReset и sRelease в ядро и из ядра не проходит ни одного beat-a — ни через гейты, ни через `TLResetIsolation`	граница закрыта на всё время сброса и выхода
P3	Probe не предлагается ядру, пока оно в сбросе, и раньше, чем гейты открылись	Probe не пропадёт в сбрасываемой ProbeQueue
P4	гейт не обрывает многобитное сообщение на A	PutFull не повиснет в мосту L2
P5	`quiet` гейтов верен: ничего не открыто и нет сообщения на середине	решение контроллера опирается на `quiet`
P6	`coreReset` только в sReset	сброс ядра не приходит вне процедуры
P7	Probe попадает в DCache только после её `alive`	ProbeQueue вышла из сброса
P8	B закрыт (держится или отвечает гейт) только в процедуре (sDrain, sReset, sRelease); P8B — ограничено по времени после снятия запроса	B не остаётся закрытым навсегда; взаимной блокировки на стороне гейта нет
P8N	SWEEP: Probe держится только в sDrain; пока гейт отвечает, предложенный Probe принимается сразу (если прошлый ответ ушёл), а готовый ответ всегда предложен L2	ответ гейта не задерживает L2
P9	с начала процедуры по A ранних гейтов не проходит новых запросов	ранняя блокировка A некогерентных работает
P10	в sDrain некогерентные агенты тихи (ничего не открыто, нет сообщения на середине ни на одном канале)	зависимость Get → Probe: B закрывается только после их тишины
P11	`hartIsInReset` = 1 от первого сброса ядра до запуска ядра	Debug Module видит харт «в сбросе»
P12	собственные ответы гейта: один Probe за раз, ровно один ProbeAck NtoN на каждый, никогда внутри сообщения C ядра; только в SWEEP	ответ гейта законен и однозначен

Свойство	Что утверждает	Зачем
P13	ядро выходит из сброса только после снятия запроса	сброс — уровень
P14	каждый сброс ядра не короче `rstCycles` (100 тактов с valid = 0)	правило TileLink 3.2.2
P15	изоляция: пока `isolate`, ни один сигнал стороны тайла гейтов и `TLResetIsolation` не зависит от ядра (A, C, E valid и D ready замаскированы, B держится или отвечает), и ядру ничего не предлагается	выходы ядра в сбросе произвольны
P16	изоляция покрывает сброс ядра (установку и синхронизатор) и только sReset / sRelease	нет окна без изоляции
P17	B держится или отвечается, пока ядро изолировано	общая маска для B
P18	B держится с первого такта sDrain (в этом такте Probe не предлагается ядру)	быстрое закрытие B
P19	сброс ядра начинается только после `l1.done` и `l1.idle` в предыдущем такте	флаги пустоты вместо 16 тактов
P20	ядро запускается (sIdle после sRelease) только когда frontend вне сброса	sRelease ждёт самый поздний узел цепочки

Проверено 20 свойств (P8 включает P8B, P8N — только SWEEP, P12 — только SWEEP).

Результаты. `results_formal_k.txt` (KEEP), `results_formal_s.txt` (SWEEP) — вывод `formal/run_formal.sh`: доказательство (unbounded) при `rstCycles` = 100 и 16, BMC глубины 40 (допущения выполнимы), cover и мутанты (BMC глубины 60 при `rstCycles` = 16 — короче трассы).

	KEEP	SWEEP
prove, rstCycles = 100	PASS, 122,02 с	нет данных
prove, rstCycles = 16	PASS, 8,71 с	нет данных
BMC 40	нет данных	нет данных
cover	нет данных	нет данных
мутанты	нет данных	нет данных
эквивалентные мутанты (должны доказываться)	нет данных	нет данных

Достижимость (cover). Каждое сценарное условие достижимо при допущениях среды:

Cover	Что достигается	KEEP: шаг	SWEEP: шаг
C1 `inflight_at_start_then_reset`	в момент запроса открыты Probe, Release, Acquire, сообщение на середине C — и сброс состоялся	нет данных	нет данных
C2 `more_inflight_at_start`	в момент запроса открыты также GrantAck, Get и сообщение на середине A	нет данных	нет данных
C3 `probe_held_then_delivered`	Probe, предложенный в сбросе, ждёт и после сброса доставлен	нет данных	нет данных
C4 `back_to_idle`	после сброса контроллер вернулся в sIdle	нет данных	нет данных
C5 `probe_in_first_drain_cycle_held_then_delivered`	KEEP: Probe, предложенный в первом такте sDrain, держится весь сброс и доставлен	нет данных	нет данных
C6 `reset_held_long_then_run`	запрос держат дольше `rstCycles` + 24 такта, затем ядро запускается	нет данных	нет данных
C7 `probe_answered_by_gate_in_reset_then_run`	SWEEP: гейт ответил на Probe в сбросе, затем ядро запущено	нет данных	нет данных
C8 `icachectrl_inflight_at_start_then_reset`	запрос к регистрам ICache ctrl открыт в момент запроса сброса	нет данных	нет данных
C9 `core_garbage_in_reset_then_run`	выходы ядра в сбросе — мусор, затем ядро запущено	нет данных	нет данных

Мутанты. Намеренно испорченные варианты модели; каждый, применимый к конфигурации, должен дать контрпример (BMC глубины 60, `rstCycles` = 16). В таблице — какие свойства нарушены и на каком такте; «—» — мутант к этой конфигурации не применяется.

Таблица мутантов: нет данных.

7.3. Симуляция

Тестбенч (`tb/cr_tb.sv`, мониторы `tb/cr_monitors.sv`) собирает Verilator 5 (бинарь, `--timing`) из моделей, которые повторяют Scala-код: контроллер и гейты (`model/cr_ctl_models.sv`, те же, что в формальной части), DCache и окружение (`model/cr_dcache.sv`, `model/cr_env_models.sv`).

- **Модель DCache** (`cr_dcache`): MainPipe с тремя стадиями (запросы Probe, store, flush), проверка конфликта набора; MissQueue с одной записью (в том числе AcquirePerm на запись всей строки); WritebackQueue — две записи Release и слот ProbeAck; ProbeQueue — запись и регистр выбора; `DCacheFlushUnit` (sweep по всем парам, sWait); флаги пустоты — те же четыре, что в RTL; флаги `busy_true` — настоящее состояние для проверки самого стенда. Размер модели уменьшен: 4 набора × 2 пути, 16 адресов (в RTL — 256 × 4); задержка записи в s3 — 0..2 такта (0..6 с `+SLOW`).
- **Буферы B и C** (`cr_buf1`): одноместные буферы в домене сброса ядра между DCache и гейтом — модель `l1d_to_l2_buffer`; в сбросе на выходе мусор.
- **L2** (`cr_l2`): инклюзивная, с директорией, снупами HN и ответчиком TL-UL; AcquirePerm не даёт данных и оставляет L2 без действительной копии (poison); **зависимость Get → Probe**: Get некогерентного клиента к строке, которую L1D держит в T, ждёт Probe toB (как CoupledL2). L2 иногда нарочно шлёт Probe строке, которой в директории нет (кроме `+NOSPUR`).
- Каждая модель ядра после установки сброса 1–4 такта выдаёт случайные выходы (и случайные разрешения записи массивов): изоляция должна их скрыть.

Сценарии.

Сценарий	Что проверяет
1, 2	сбросы при непрерывном трафике ядра (загрузки, записи, AcquirePerm) и снупах HN: три запроса (1), двенадцать подряд с случайными паузами (2)
3	запрос случайной длины (2..2000 тактов): ядро в сбросе, пока держат запрос
4	наивный сброс без последовательности: воспроизводит проблему (ошибки ожидаются)
5	процедура начинается ровно тогда, когда в полёте транзакция одного из классов C1..C12 (открытые Probe, Release, Acquire, GrantAck, сообщение на середине, MSHR с неотправленным Acquire, Probe, предложенный в этом такте)
6	направленные Probe строк L1D (грязная / чистая T / B; toN / toB), шесть случаев: 0 — в sReset, 1 — в sRelease, 2 — сразу после последовательности, 3 — в начале sFlush, 4 — в первом такте sDrain, 5 — в последнем такте sFlush (гейт принял, sDrain ждёт его ProbeAck)
7	долгий запрос (2000..20000 тактов, как `ndmreset` / `hartreset` у отладчика) со снупами во время удержания (KEEP: Probe горячей строки ждёт в L2 до снятия запроса; SWEEP: ответ гейта сразу)
+SLOW	длинные случайные задержки в L2, паузы ready и между beat-ами (по умолчанию в сценариях 5, 6, 7)

Проверки в каждой точке сброса и при выходе: в L2 для ядра ничего не висит; управляющая логика DCache и буферы B / C пусты (`busy_true` = 0); директория L2 совпадает с meta DCache построчно (SWEEP: L1D пуста, L2 ничего не числит); текущее значение каждой строки (DCache, если грязная, иначе L2) равно эталону; после начала процедуры ни одного нового некогерентного запроса; в sDrain ничего некогерентного открытого; ни одного Probe через гейт в sDrain; конец последовательности только когда frontend вне сброса; выход из сброса только после снятия запроса; один сброс на запрос; во время сброса ничего не проходит ни по одному каналу, при KEEP meta не меняется; Probe не ждёт более 200 тактов (KEEP — вне сброса). Каждая загрузка-попадание сверяется с эталоном, каждый некэшируемый ответ — с клиентом. В конце — глобальная согласованность и отсутствие зависания.

Регрессия (`scripts/regress.sh` → `results_regress.txt`): сценарии 1–3 × 15 seed-ов, с `+SLOW` × 5, сценарий 5 × 10, сценарий 6 × 5, сценарий 7 × 5, сценарий 4 × 3 (наивный сброс) — для каждой конфигурации; затем мутанты. Все прогоны дизайна должны пройти, наивный сброс и каждый мутант — дать ошибку, эквивалентные (`+EQUIV`) — пройти.

	KEEP	SWEEP
прогонов дизайна: PASS / всего	80 / 80	80 / 80
наивный сброс: проблема воспроизведена	3 / 3	3 / 3
прогонов мутантов: пойманы / всего	110 / 110	101 / 101
ожидаемо эквивалентные (+EQUIV): PASS / всего	9 / 9	18 / 18

Группа прогонов дизайна	KEEP: PASS / всего	SWEEP: PASS / всего
сценарий 1	15 / 15	15 / 15
сценарий 1 (+SLOW)	5 / 5	5 / 5
сценарий 2	15 / 15	15 / 15
сценарий 2 (+SLOW)	5 / 5	5 / 5
сценарий 3	15 / 15	15 / 15
сценарий 3 (+SLOW)	5 / 5	5 / 5
сценарий 5	10 / 10	10 / 10
сценарий 6	5 / 5	5 / 5
сценарий 7	5 / 5	5 / 5

Мутанты в симуляции. Каждый мутант — отдельная сборка (`build/obj_<конфигурация>+<мутант>`); «пойман» — прогон завершился ошибкой проверки. Редкие в симуляции мутанты (попадание зависит от seed-а) запускаются на seed-ах, подобранных `scripts/seedsearch.sh` (`scripts/rare_seeds.txt`; сборка Verilator 5.050 вместо 5.049, поэтому seed-ы подобраны заново).

Мутант	Что ломает	KEEP	SWEEP	Сценарии
`CTRL_MUTANT_ISO_EARLY_DROP+NO_CTRL_ASSERT`	`isolate` снимается вместе с `coreReset`, до конца sRelease (assert-ы контроллера выключены)	3 / 3	3 / 3	2
`CTRL_MUTANT_ISO_LATE+NO_CTRL_ASSERT`	`isolate` поднимается на такт позже `coreReset` (assert-ы контроллера выключены)	3 / 3	3 / 3	2
`CTRL_MUTANT_LATE_HOLDB`	B закрывается на такт позже (со второго такта sDrain)	3 / 3	3 / 3	6
`CTRL_MUTANT_NOKEEP_HOLDB`	SWEEP: Probe держится вместо ответа гейта	—	3 / 3	2
`CTRL_MUTANT_NO_DRAIN`	сброс без ожидания флагов и тишины: sDrain → sReset сразу	3 / 3	3 / 3	2, 6
`CTRL_MUTANT_NO_DRAIN+NO_CTRL_ASSERT`	сброс без ожидания флагов и тишины: sDrain → sReset сразу (assert-ы контроллера выключены)	3 / 3	3 / 3	2, 6
`CTRL_MUTANT_NO_EARLY_BLOCK`	A некогерентных закрывается только с sDrain (как в v4)	6 / 6	6 / 6	2, 5
`CTRL_MUTANT_NO_EARLY_QUIET`	выход из sFlush без тишины ранних гейтов: B закрывается, пока Get может ждать Probe	6 / 6	3 / 3	2, 5
`CTRL_MUTANT_NO_FE_WAIT`	sRelease не ждёт выхода frontend из сброса	3 / 3	3 / 3	2
`CTRL_MUTANT_NO_HOLD_IN_RELEASE+NO_CTRL_ASSERT`	Probe открывается вместе со снятием `coreReset`, до конца sRelease (assert-ы контроллера выключены)	3 / 3	3 / 3	6
`CTRL_MUTANT_RESET_EARLY`	сброс заканчивается при `cnt >= rstCycles` даже при поднятом запросе	6 / 6	6 / 6	3, 7
`CTRL_MUTANT_STALE_QUIET`	решение sDrain по `quiet` предыдущего такта (Probe последнего такта sFlush проскакивает)	3 / 3	3 / 3	6
`FLUSH_MUTANT_NO_BLOCK`	записи, атомики и промахи не блокируются на время опустошения	5 / 5	5 / 5	2
`FLUSH_MUTANT_NO_BLOCK+NO_CTRL_ASSERT`	записи, атомики и промахи не блокируются на время опустошения (assert-ы контроллера выключены)	5 / 5	5 / 5	2
`FLUSH_MUTANT_NO_MP_WAIT`	опустошение не ждёт MainPipe	3 / 3	—	2, 5
`FLUSH_MUTANT_NO_SWEEP`	обхода нет: L1D сбрасывается со строками (удалённый полный сброс)	—	6 / 6	2, 6

Мутант	Что ломает	KEEP	SWEEP	Сценарии
`FLUSH_MUTANT_NO_WBQ_WAIT+NO_CTRL_ASSERT`	опустошение не ждёт WritebackQueue (Release, ProbeAck) (assert-ы контроллера выключены)	5 / 5	5 / 5	2, 5
`FLUSH_MUTANT_SWEEP_NO_DATA`	грязная строка отдаётся без данных	—	3 / 3	2
`FLUSH_MUTANT_SWEEP_NO_WAIT`	`done` сразу после последнего flush, Release ещё в очереди	—	3 / 3	2
`FLUSH_MUTANT_SWEEP_NO_WAIT+NO_CTRL_ASSERT`	`done` сразу после последнего flush, Release ещё в очереди (assert-ы контроллера выключены)	—	3 / 3	5
`FLUSH_MUTANT_SWEEP_ONE_WAY`	обход останавливается после пути 0	—	3 / 3	2
`FREEZE_MUTANT_NO_SRAM_MASK`	KEEP: запись в SRAM тегов и данных не замаскирована `freeze`	6 / 6	—	2, 6
`GATE_MUTANT_ACK_PARAM`	SWEEP: гейт отвечает TtoN вместо NtoN	—	3 / 3	5
`GATE_MUTANT_ALWAYS_QUIET`	`quiet` гейта всегда 1	3 / 3	3 / 3	2
`GATE_MUTANT_ALWAYS_QUIET+NO_CTRL_ASSERT`	`quiet` гейта всегда 1 (assert-ы контроллера выключены)	3 / 3	3 / 3	5
`GATE_MUTANT_CUT_A+NO_CTRL_ASSERT`	A закрывается посреди многобитного Put (assert-ы контроллера выключены)	5 / 5	5 / 5	2
`GATE_MUTANT_NO_ACKB`	SWEEP: гейт не отвечает на Probe, Probe проходит к ядру	—	3 / 3	2
`GATE_MUTANT_NO_HOLDB`	KEEP: гейт не держит B (Probe проходит к ядру в сбросе)	6 / 6	—	2, 6
`GATE_MUTANT_NO_ISO_C`	нет изоляции канала C	3 / 3	3 / 3	2
`GATE_MUTANT_NO_ISO_E`	нет изоляции канала E	3 / 3	3 / 3	2
`ISO_MUTANT_NO_ISO`	`TLResetIsolation` ничего не изолирует	3 / 3	3 / 3	2
`META_MUTANT_CORE_RESET`	KEEP: `metaArray` сбрасывается сбросом ядра	3 / 3	—	2
`META_MUTANT_CORE_RESET+NO_FREEZE_CHECK`	KEEP: `metaArray` сбрасывается сбросом ядра (проверка массивов под freeze выключена)	9 / 9	—	2, 6, 7
`TAG_MUTANT_CORE_RESET`	KEEP: теги стираются сбросом ядра (дефект v4), meta сохранена	6 / 6	—	2, 6

«a / b»: пойманных прогонов / запущенных. Любой мутант, пойманный не во всех прогонах, — «немой» на части seed-ов; немой на всех — дыра в проверке (в таблице таких не должно быть).

Ожидаемо эквивалентные мутанты в симуляции (прогон проходит, мутант не наблюдаем):

Мутант	Что ломает	PASS / всего	Почему не наблюдаем
`CTRL_MUTANT_NO_ALIVE`	sRelease не ждёт `alive` DCache	6 / 6	frontend — последний в цепочке ResetGen, после DCache: ожидание выхода frontend покрывает `alive`; формально ловится (P7), потому что окружение допускает любой порядок
`CTRL_MUTANT_NO_L1_IDLE`	контроллер игнорирует `l1.idle`	12 / 12	флаг `l1.idle` избыточен с `l1.done` и `quiet` гейтов (в sDone занят может быть только путь Probe, а каждый Probe считает гейт); формально ловится (P19), потому что окружение допускает произвольный `idle`
`FLUSH_MUTANT_NO_MP_WAIT`	опустошение не ждёт MainPipe	6 / 6	в SWEEP после обхода MainPipe пуст (ожидание `inflight === 0` в sWait), поэтому флаг `mp_empty` ничего не добавляет; в KEEP мутант ловится
`NOKEEP_MUTANT_META_KEPT`	SWEEP: `metaArray` переживает сброс	3 / 3	после обхода у каждой строки состояние N, поэтому сохранённая meta содержит только N

Длинный прогон (`scripts/soak.sh` → `results_soak_<конфигурация>.log`): сценарии 1–3 на seed-ах 31..130 (300 прогонов), с `+SLOW` на seed-ах 11..40 (90), сценарий 5 на seed-ах 21..70 (50), сценарии 6 и 7 на seed-ах 11..35 (по 25) — 490 прогонов на конфигурацию.

	KEEP	SWEEP
прогонов: PASS / всего	490 / 490	490 / 490

Покрытие (`coverage_*.json`, сумма по прогонам длинного прогона):

Показатель	KEEP	SWEEP
сбросов (resets)	5869	5869
операций DCache	940169	921875
Probe всего	220297	247116
Probe лишних (строки нет в директории)	38824	73453
снупов, которые L2 обслужила сама	124455	216285
AcquirePerm	83473	88519
Release во время процедуры	603	37403
из них с данными	309	20214
Probe удержано (KEEP)	5021	315
Probe, отвеченных гейтом (SWEEP)	0	29297
фаз sDrain	5869	5869
самая длинная sDrain, тактов	85	84
Probe удержано в sDrain	354	315
Get, ждавших Probe (get-probes)	68129	65798
Probe в 1-м такте sDrain: предложен в sDrain	250	250
Probe в 1-м такте sDrain: позже	0	0
Probe в последнем такте sFlush: попал	249	249
Probe в последнем такте sFlush: позже	1	0
Probe в начале sFlush: в sFlush	249	250

Попадания направленных запусков сценария 5 в классы транзакций (`triggers hit`; классы C1..C10 — открытые состояния `cr_txmon`, C11 и C12 — условия в тестбенче):

Класс сценария 5	Описание	Попаданий KEEP	Попаданий SWEEP
C1	Acquire принят, Grant ещё нет	150	150
C2	GrantData: первый beat передан, второй нет	150	150
C3	Grant передан, GrantAck ещё нет	150	150
C4	Release(Data) передан, ReleaseAck ещё нет	150	150
C5	ReleaseData: передан только первый beat	150	150
C6	Probe доставлен, ProbeAck ещё нет	150	150
C7	ProbeAckData: только первый beat	150	150
C8	Get / PutFull передан, AccessAck ещё нет	150	150
C9	PutFull: только первый beat	150	150
C10	AccessAckData: только первый beat	150	150
C11	MSHR выделена, Acquire не отправлен	150	150
C12	L2 предлагает Probe в этом такте	150	150

Seed-ы редких мутантов (`scripts/rare_seeds.txt`):

Конфигурация	Мутант	seed-ы (сценарий:seed)
KEEP	`GATE_MUTANT_CUT_A+NO_CTRL_ASSERT`	2:1 2:4 2:5 2:6 2:8
CFG_SWEEP	`GATE_MUTANT_CUT_A+NO_CTRL_ASSERT`	2:2 2:3 2:4 2:5 2:6
KEEP	`FLUSH_MUTANT_NO_WBQ_WAIT+NO_CTRL_ASSERT`	5:1 5:2 5:3 5:4 5:5
CFG_SWEEP	`FLUSH_MUTANT_NO_WBQ_WAIT+NO_CTRL_ASSERT`	2:1 2:2 2:3 2:4 2:5

Конфигурация	Мутант	seed-ы (сценарий:seed)
KEEP	`FLUSH_MUTANT_NO_BLOCK`	2:2 2:4 2:6 2:8 2:9
CFG_SWEEP	`FLUSH_MUTANT_NO_BLOCK`	2:3 2:5 2:6 2:7 2:9
KEEP	`FLUSH_MUTANT_NO_BLOCK+NO_CTRL_ASSERT`	2:2 2:4 2:6 2:8 2:9
CFG_SWEEP	`FLUSH_MUTANT_NO_BLOCK+NO_CTRL_ASSERT`	2:3 2:5 2:6 2:7 2:9
KEEP	`FLUSH_MUTANT_NO_MP_WAIT`	2:14 5:5 5:8

Что нашла сама проверка

(а) Ревью Chisel: дефект v4 — теги L1D стирались при сбросе ядра (KEEP): `TagSRAMBank.rst_cnt` в домене сброса ядра обнулял SRAM тегов при каждом сбросе, а meta сохранялась. Исправлено (`tagArray` на сбросе тайла), добавлен мутант `TAG_MUTANT_CORE_RESET`.

(б) Определение мутанта `STALE_QUIET` было эквивалентным (мутант не менял поведения) — исправлено: теперь он решает по `quiet` прошлого такта и в первом такте sDrain, и Probe последнего такта sFlush проскакивает.

(в) В модели DCache апгрейд BtoT не блокировался flush-ем, в отличие от RTL — исправлено.

(г) Класс C12 сценария 5 был недостижим в v5 (Probe, предложенный в такте запроса, принимается ещё в sIdle) — класс переопределён: «L2 предлагает Probe в этом такте».

(д) Триггер случая 5 сценария 6 в SWEEP не попадал в последний такт sFlush — добавлено условие пустоты (`u_dc.idle`).

(е) Конфигурация XSNoCTop из `Default.yml` (`tp: true`) ломает elaboration независимо от сброса — взяты YAML только с ключами сброса.

Кроме того, стенд дал ошибку, найденную длинным прогоном: ожидание тишины в сценарии 6 не учитывало MainPipe, и эталон строки B запоминался, пока store в s3 ждал BtoT-апгрейда; исправление — ждать `dc_busy_true` и буферы B и C.

7.4. Временные диаграммы из VCD

Диаграммы нарисованы по VCD-файлам реальных прогонов тестбенча (`core-reset/doc/make_waves.sh`: сборка Verilator с трассировкой `TRACE_BUILD=1 scripts/run.sh ...`, разбор VCD — `core-reset/doc/vcd.py`, отрисовка — `reportlab`). Окна по тактам найдены автоматически по событиям (состояние контроллера, handshake Probe), не заданы числами. Горизонтальная ось — такты тестбенча; зелёный — условие выполнено, серая линия — 0; красная штриховка в строке B — Probe предложен и не принят. Модель DCache уменьшена (4 набора × 2 пути), поэтому sFlush и обход в ней короче, чем в RTL.

Диаграмма 1. SWEEP, сценарий 1, seed 1: вся последовательность

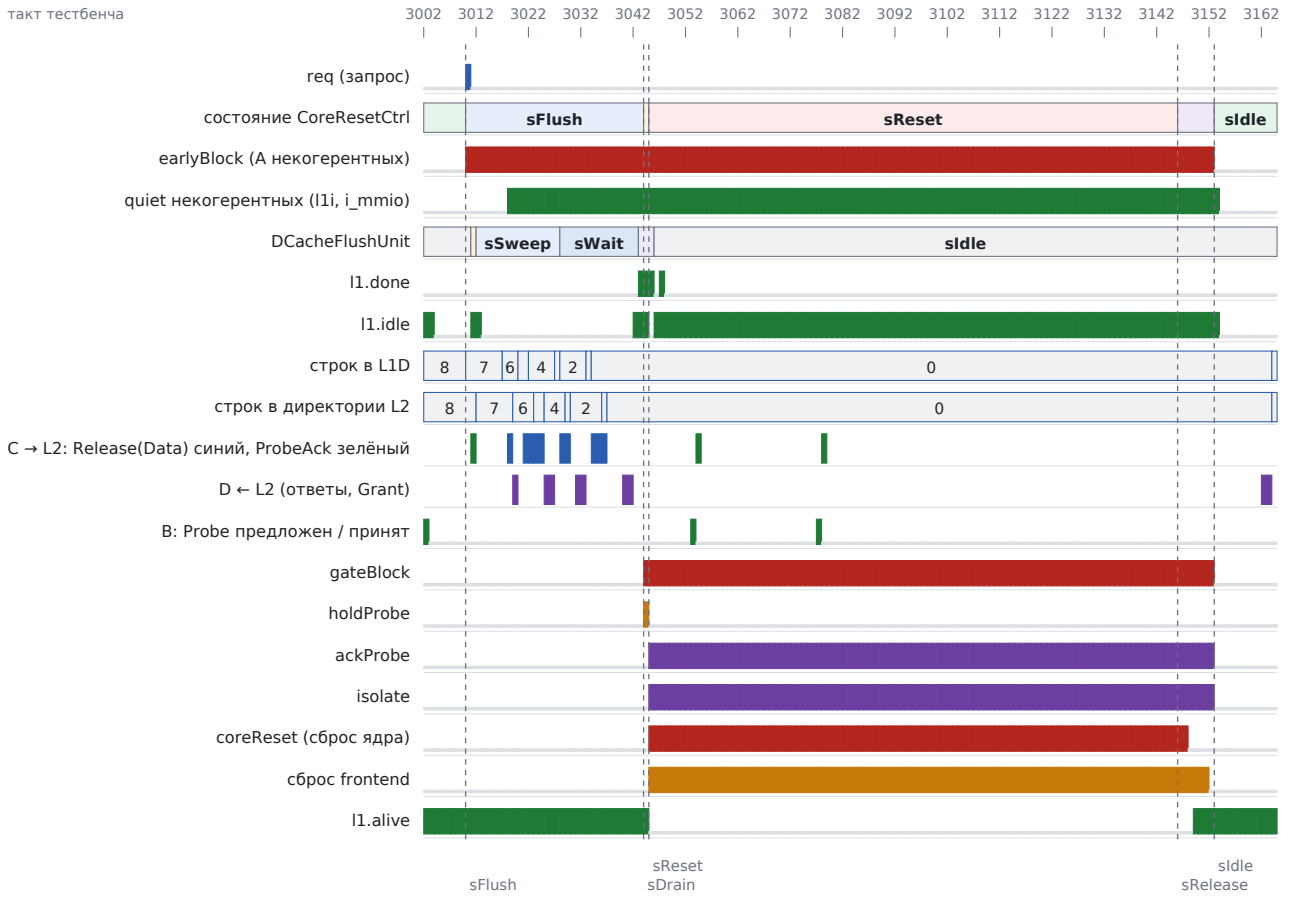


Рис. 8. Запрос поднят в такте тестбенча; sFlush — 34 тактов (обход L1D модели: Release / ReleaseData по строкам, синие метки на C; ReleaseAck — на D), sDrain — 1 такт(а), sReset — 101 тактов (>= 100, запрос снят сразу после), sRelease — 7 тактов; весь цикл — 143 тактов. Число строк в L1D падает до 0, в директории L2 — до 0: после обхода L2 ничего не числит за L1D. `earlyBlock` поднят с первого такта sFlush, `quiet` некогерентных — к его концу; `holdProbe` — только в sDrain; `ackProbe`, `isolate` и сброс ядра — с sReset; сброс frontend снимается позже `alive`.

Диаграмма 2. KEEP, сценарий 6, seed 1, случай 5: Probe принят в последнем такте sFlush

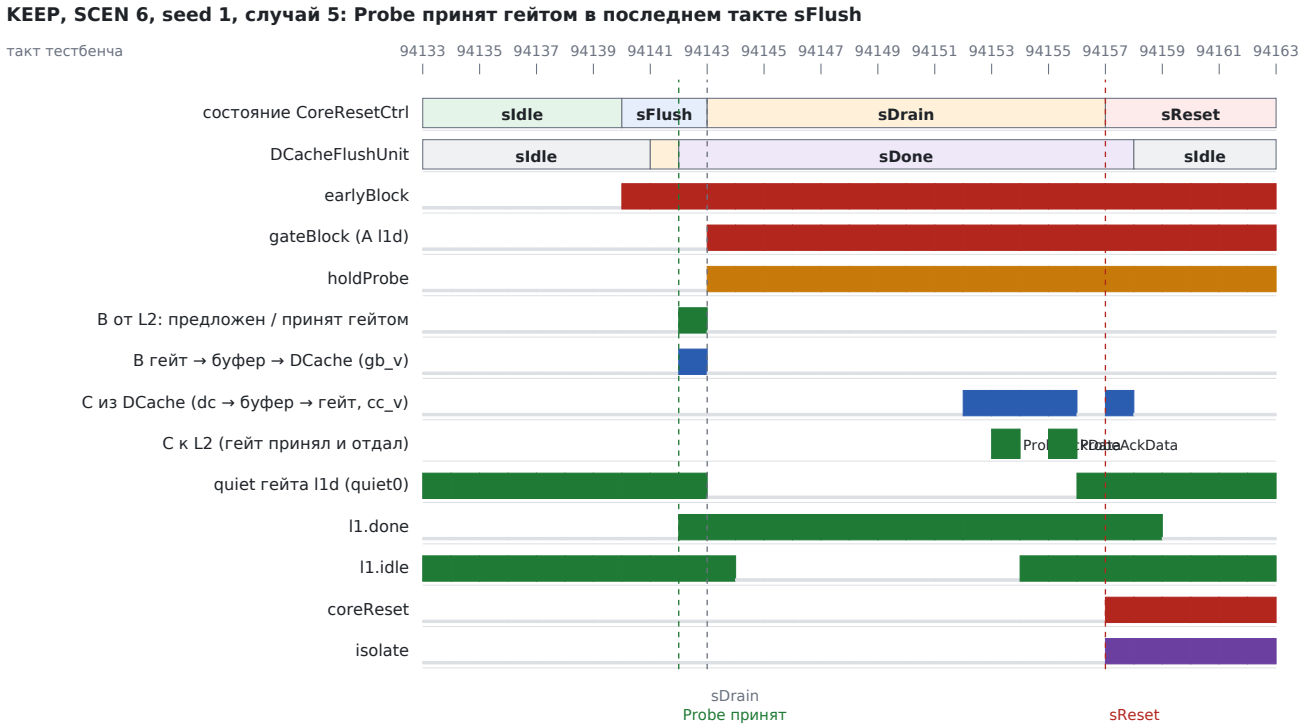


Рис. 9. Гейт l1d принял Probe (зелёный в строке B) в последнем такте sFlush; в sDrain `holdProbe` закрывает B, а sDrain ждёт: DCache отвечает ProbeAck(Data), он идёт через буфер C (`cc_v`) в гейт и дальше к L2 (`C к L2`); `quiet` гейта l1d (`probes` ≠ 0) остаётся 0, пока ответ не пройдёт, и только затем — `gatesQuiet && l1Done && l1Idle` и переход в sReset. sDrain длится 14 тактов (в v4 — 16 тактов подряд плюс проверка).

Диаграмма 3. KEEP, сценарий 6, seed 1, случай 4: Probe toN грязной строки в первом такте sDrain

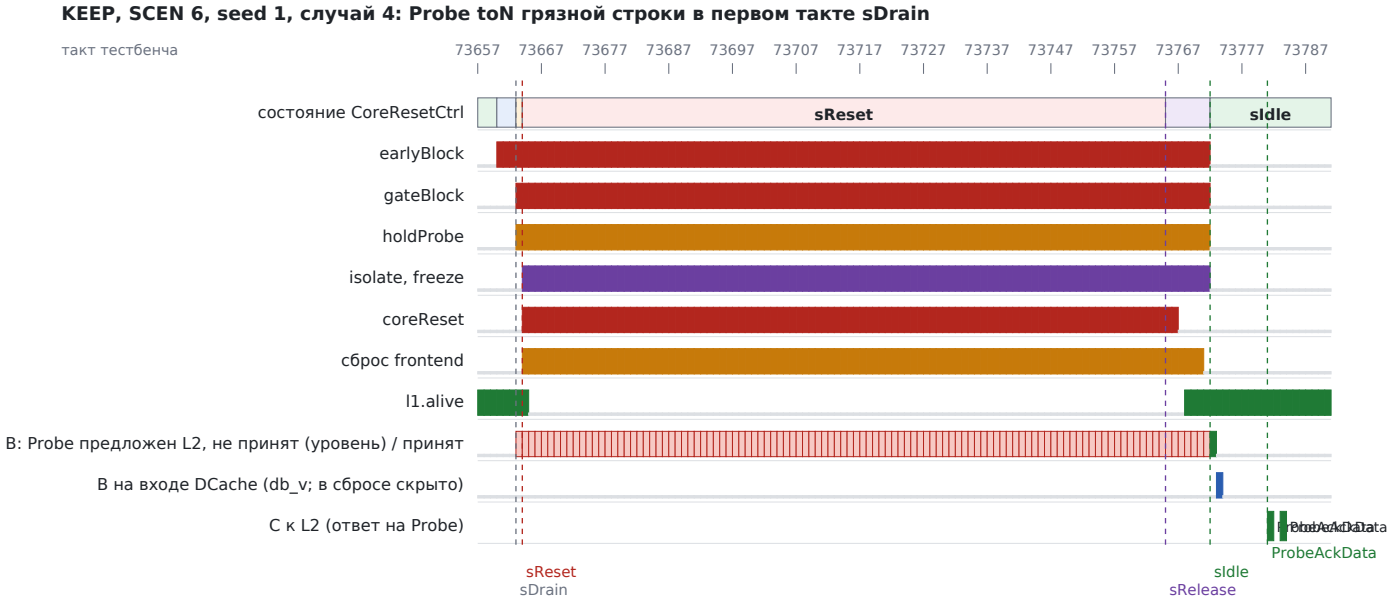


Рис. 10. L2 предлагает Probe в первом такте sDrain: `holdProbe` уже 1 (функция регистра состояния), `b.ready` = 0. Probe не принят все 118 тактов до ответа (красная штриховка: предложен, не принят — рисуется уровнем), включая sReset (101 тактов), sRelease и выход ядра; после запуска DCache получает Probe и отвечает ProbeAckData из «горячей» L1D. Выход буфера B внутри домена сброса ядра в сбросе — мусор: строка `db_v` скрыта на время `coreReset`.

Диаграмма 4. Момент установки сброса (тот же сброс, случай 4)

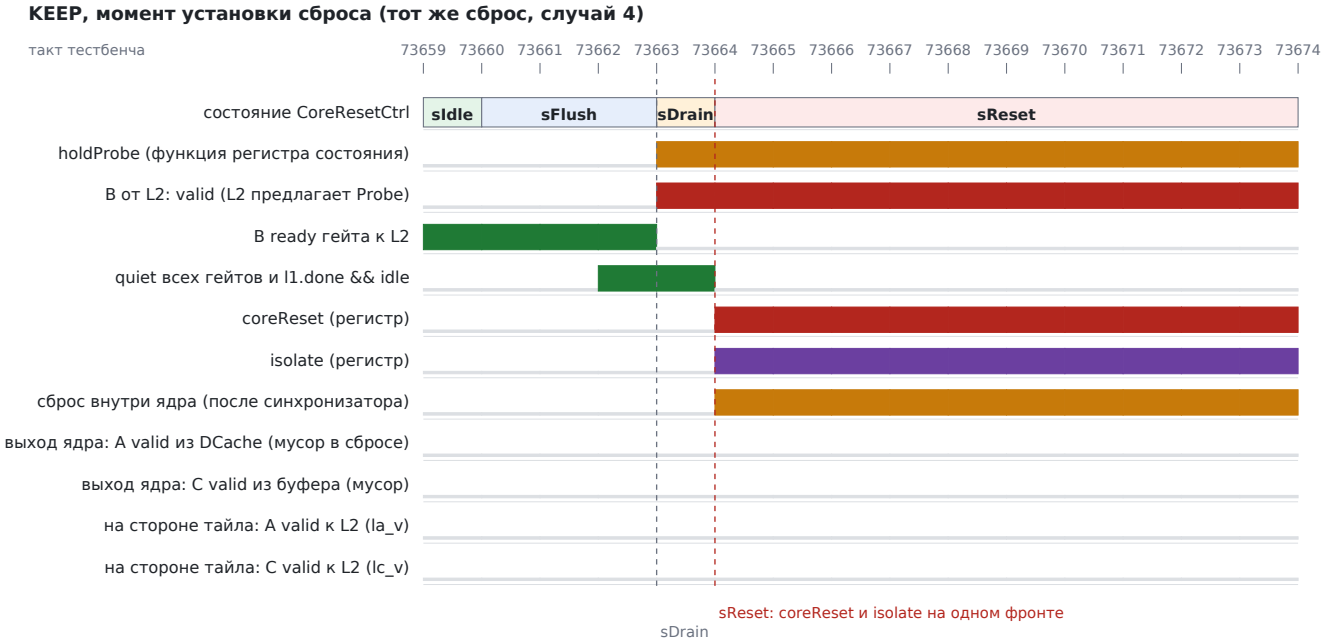


Рис. 11. Крупно: `holdProbe` с первого такта sDrain, L2 предлагает Probe (`B valid`), `B ready` гейта = 0. В такте решения `quiet` всех гейтов, `l1.done` и `l1.idle` одновременно высоки; на следующем фронте поднимаются регистры `coreReset` и `isolate` вместе (E0). Внутри ядра сброс приходит позже (после синхронизатора), а выходы ядра (`A valid` из DCache, `C valid` из буфера) в первые такты сброса случайны (мусор, оранжевый); на стороне тайла `A valid` и `C valid` к L2 остаются 0: изоляция их замаскировала.

8. Подключение и выбор конфигурации

- **Включение.** `EnableCoreReset = true` и при необходимости `CoreResetKeepL1` в `SoCParameters` (например, `case SoCParamsKey => up(SoCParamsKey).copy(EnableCoreReset = true, CoreResetKeepL1 = false)`) или те же ключи в YAML-конфигурации (раздел 6.7). Параметр `CoreResetLevel` удалён; в старых YAML он игнорируется.
- **Порты XSNoCTop / XSTileWrap / XSTile.** `debug_reset` — вход, асинхронный уровень; `core_reset_req` — вход, асинхронный уровень; `core_reset_busy` — выход в такте тайла. `hartResetReq` — существующий вход upstream: с `EnableCoreReset` действует как `debug_reset`. `hartIsInReset` — существующий выход: теперь 1 от первого такта сброса до запуска ядра. Набор портов одинаков во всех конфигурациях и **не изменился** относительно v4.
- **Debug Module.** `debug_reset[i] = ndmreset | hartreset[i]` для тайла с хартом i (или `hartreset[i]` на `hartResetReq`, `ndmreset` на `debug_reset`); `hartIsInReset[i]` — на вход DM. `ndmreset` не подавать на основной сброс тайла: он сбросит L2 и CHI-порт.
- **Кластер.** `core_reset_req` — для контроллера сброса кластера; можно привязать к 0. Если XSTile подключён напрямую и у вас своё отключение такта: такт тайла должен идти, пока запрос или `core_reset_busy` = 1.
- **Протокол.** Поднять запрос; ядро останавливается после опустошения (и обхода), сбрасывается и остаётся в сбросе, пока держат запрос (не меньше 100 тактов); после снятия запроса ядро выходит из сброса и стартует. `core_reset_busy = 0` — ядро снова работает.
- **CHI и HN.** Ничего делать не нужно. Снупы к строкам L1D: KEEP — ждут Probe до конца сброса (весь сброс), SWEEP — не задерживаются (строк за L1D нет).
- **Выходы тайла во время сброса ядра:** `cpu_halt`, `cpu_crtical_error`, `msiAck` — 0, `l2_flush_en` — 0, трасса — без valid, `hartIsInReset` — 1 (1.6).

Выбор KEEP или SWEEP.

Если нужно	Конфигурация	Почему
сброс по debug reset, программа перезапускается; отладчик пишет память через SBA, пока держит сброс	SWEEP	снупы не ждут (L1D за собой ничего не числит, ответ гейта NtoN); L1D после сброса чистая, как после обычного сброса, а её данные — в L2; взаимной блокировки нет
сохранить «горячую» L1D (сброс ядра как восстановление после зависания программы, с последующим анализом памяти через кэш)	KEEP (по умолчанию)	строки и данные остаются в L1D; цена — снупы к строкам L1D ждут конца сброса, и при удержанном сбросе нельзя обращаться к памяти, кэшированной этим ядром (раздел 9)
сброс как реакция на подозрение на испорченные строки L1	SWEEP	L1D не переживает сброс; но обход отдаёт в L2 и испорченную строку (флаги ошибок идут с ReleaseData)

Как это выглядит для отладчика (OpenOCD, reset halt через `ndmreset`): запись `dmcontrol` с `ndmreset = 1`, `haltreq = 1` — опустошение (и обход) и сброс ядра, `hartIsInReset = 1`, DM выставляет `havereset`; отладчик может в это время писать память через SBA (SWEEP; с KEEP — не в кэшируемую этим ядром память); запись `ndmreset = 0` — ядро стартует, видит прерывание отладки и останавливается на первой команде; отладчик видит `allhavereset` и `allhalted`, пишет `ackhavereset`. Reset run — то же без `haltreq`.

Время последовательности

Ожидания по 16 тактов нет, поэтому время определяется только реальной занятостью. Составляющие (после подъёма запроса):

Состояние	KEEP	SWEEP
sFlush	пока DCache не станет пустой (`idle`) и ранние гейты не затихнут: от 1–2 тактов при пустом DCache до длительности самых долгих начатых промахов и Release	то же + обход: не менее 1024 тактов (по запросу flush на такт, если MainPipe принимает; Probe имеют приоритет) + последний ReleaseAck
sDrain	обычно 1 такт	обычно 1 такт

Состояние	KEEP	SWEEP
sReset	≥ 100 тактов и до снятия запроса	≥ 100 тактов и до снятия запроса
sRelease	до выхода DCache (`alive`) и frontend из сброса: единицы тактов (в модели — 7)	то же
после снятия запроса	sRelease: единицы тактов до запуска ядра	то же

Измерено в модели (уменьшенный DCache: 4 набора × 2 пути)	Значение
SWEEP, сценарий 1, seed 1 (рис. 8): sFlush / sDrain / sReset / sRelease, тактов	34 / 1 / 101 / 7 (всего 143)
самая длинная sDrain в длинном прогоне, KEEP / SWEEP	85 / 84

В реальном DCache (256 × 4) обход SWEEP — порядка 1024 тактов плюс задержки Release, ReleaseAck и L2, поэтому суммарно порядка 1100–2000 тактов при быстрых устройствах (зависит от числа грязных строк и задержки L2); KEEP — десятки тактов подготовки + сброс не короче 100 тактов. В v4 к этому добавлялось 16 тактов в sDrain плюс 8 в sRelease и ещё 16 в каждом ожидании DCache; в v5 их нет.

9. Ограничения и что проверить при интеграции

- **Полная elaboration XSNoCTop:** не выполнена (файла `results_elab.txt` нет). Соединения XSTile, пять гейтов на реальных рёбрах diplomacy, изоляция выходов, мосты предвыборки (`BundleBridgeSink` / `BundleBridgeSource`) и сброс массивов L1D (`metaArray.reset := ...`: тип сброса должен получиться асинхронным) проверены компиляцией и ревью, не прогоном всего дизайна. Собрать обе конфигурации.
- **KEEP — взаимная блокировка**, если отладчик при удержанном сбросе пишет через SBA в строку, которая есть в L1D: снуп к ней ждёт Probe, Probe ждёт выхода ядра из сброса, а отладчик ждёт запись и не снимает сброс. Для отладки через SBA — SWEEP (1.4, раздел 8).
- **У контроллера нет таймаута**: если L2 или MMIO-устройство не отвечает, опустошение не заканчивается, сброс ядра не наступает, `core_reset_busy` остаётся 1 (L2 без ответа — последовательность висит, `busy` держит такт). Это сделано намеренно: сбросить ядро с незавершённой транзакцией нельзя. Кластеру нужен запасной путь: сторожевой таймер на `core_reset_busy` и полный сброс тайла.
- **MMIO-запросы**, не прошедшие гейт до sFlush, теряются со сбросом ядра (включая закоммиченный MMIO store); так же теряются зафиксированные записи в sbuffer и некэшируемые записи, ещё не отправленные Uncache.
- **Новые места состояния в DCache** (новая очередь, регистр между очередью и конвейером) требуют своего флага пустоты и включения его в `idle`: иначе точный `idle` перестанет быть точным (1.5.3). Формальные свойства P19 и модельный флаг `busy_true` ловят расхождение только если модель обновлена.
- **Инвариант нулевого запаса записи массивов:** между решением sDrain → sReset и поднятием `coreReset` нельзя добавлять задержку (регистр, ожидание): одна регистровая стадия записи в массивы данных и meta фиксируется на том же фронте (1.5.3). Проверяется латентность `SRAMTemplate`: если она вырастет, инвариант надо пересмотреть.
- **MBIST-цепочка массива тегов** частично на сбросе тайла (регистры, перенесённые вместе с `tagArray`): режим MBIST — при общем сбросе; описать это в правилах DFT.
- **Проверка на моделях, не на RTL всего ядра.** Формальная часть и симуляция работают на моделях, повторяющих Scala-код; для контроллера и гейтов RTL доказан эквивалентным модели (7.1). DCache и XSTile проверены ревью и компиляцией, а их поведение — моделью DCache.
- **MainPipe изменён** (запрос flush) — самое чувствительное место патча: прогнать штатные тесты DCache XiangShan и RTL-симуляцию со SWEEP на реальной CoupledL2.
- **Инструмент RDC.** Все сигналы из домена сброса ядра в домен тайла изолированы регистром `isolate` (таблица 1.6); описать в правилах инструмента сигнал изоляции, его домен и значения. С KEEP — ещё `metaArray`, `errorArray`, `tagArray`: домен тайла, читаются и пишутся логикой домена ядра, запись под `freeze`.
- **DFT.** Новые ResetGen получают тот же `io.dft_reset`, что и сброс ядра; регистры изоляции — в домене тайла.

- **IMSIC ядра:** файл прерываний — часть ядра и сбрасывается с ним: MSI, принятые до сброса, теряются; рукопожатие с мостом IMSIC остаётся согласованным (1.6).
- **Изменилось действие** `hartResetReq`**:** с `EnableCoreReset` он больше не сбрасывает L2.
- **Другой L2:** обход использует только Release / ReleaseData, ответ гейта — ProbeAck NtoN; оба законны для любого менеджера TileLink. С KEEP L2 должен уметь ждать Probe (держать B) весь сброс; кроме того, зависимость Get → Probe — свойство CoupledL2: для другого L2 `earlyQuiet` остаётся безопасным.
- **Версия Verilator.** Использован Verilator 5.050 вместо 5.049, на которой подбирались seed-ы редких мутантов: они подобраны заново (`scripts/seedsearch.sh`, `rare_seeds.txt`).

10. Отличия от v4 и состав пакета

Что	v4	v5
Режим сброса	импульс (sHold, второй импульс) или уровень; два параметра	только уровень; один параметр `CoreResetKeepL1`
Компиляция Chisel	не компилировалась (`name`)	компилируется (`gateName`)
Канал A	все гейты — с sDrain	некогерентные — с sFlush (`early`); `earlyQuiet` при выходе из sFlush
Канал B	держался с sReset	закрыт с первого такта sDrain (`holdProbe` — функция состояния)
Ожидание перед сбросом	16 тактов подряд (DCache и контроллер), `idle` гейта с `b.valid`	флаги пустоты, решение по `quiet` текущего такта
Состояния контроллера	6 (с sHold)	5
Выход из sRelease	≥ 8 тактов и `alive`	`alive` и выход frontend (`resetInFrontend`)
Новые изменённые файлы	15 файлов	16 файлов (+ `Probe.scala`; `FakeDCache`, `MainPipe`, `DCacheWrapper` — флаги)
Теги L1D (KEEP)	стирались сбросом ядра	на сбросе тайла
Проверка	четыре конфигурации (KEEP / SWEEP × импульс / уровень), свойства P1–P17	две конфигурации (KEEP, SWEEP), свойства P1–P20, эквивалентность RTL ↔ модели (5 миттеров), мутанты, сценарии 1–7, временные диаграммы из VCD
Порты, параметры	`CoreResetKeepL1`, `CoreResetLevel`	порты без изменений; `CoreResetLevel` удалён
Генератор PDF	потерян	в ветке: `core-reset/doc/make_pdf.py`

Состав пакета (ветка `core-reset-v5`):

Путь	Что это
`src/main/scala/...`	16 изменённых и новых файлов XiangShan (раздел 3); CoupledL2 не менялся
`core-reset/xscore_reset_v5.pdf`	этот документ
`core-reset/doc/`	генератор документа: `make_pdf.py` (+ `content_*.py`, `pdflib.py`, `diagrams.py`, `waves.py`, `results.py`), `vcd.py`, `make_waves.sh`, `SPEC_PDF.md`
`core-reset/verification/model/`, `tb/`	модели гейтов, изоляции, контроллера, DCache и L2; тестбенч и мониторы
`core-reset/verification/formal/`	`fv_boundary.sv` (P1–P20, cover), `run_formal.sh`, `eq_chisel.sv`, `run_equiv.sh`
`core-reset/verification/rtl/`, `elab/`	RTL модулей, вырезанный из elaboration; стенд elaboration и YAML для XSNoCTop
`core-reset/verification/scripts/`	`run.sh`, `regress.sh`, `soak.sh`, `coverage.py`, `regress_summary.py`, `seedsearch.sh`, `extract_rtl.py`, `env.sh`
`core-reset/verification/results_*`, `coverage_*.json`	результаты формальной проверки, эквивалентности, регрессии, длинных прогонов
`core-reset/HANDOFF.md`, `CLAUDE.md`	требования, дизайн, ловушки, состояние работы